Documentation of pyOM3

Contents

Overview **2**

Practical tips for programming **2**

pe_simple **6**

pe_realistic **19**

nh_simple **29**

nh_realistic **38**

wim_simple **46**

wim_realistic **71**

The exact S_{nl} of wim_simple **78**

Overview

pyOM3 is a set of ocean and wave models in Python. Each model is a package in the directory `pyOM3`. This document collects the documentation of all packages. Each documentation is a section of its own and can also be compiled alone.

Package	Model	Page
`pe_simple`	primitive equations on a uniform Cartesian grid	6
`pe_realistic`	pe_simple with a variable grid, the sphere and isoneutral mixing	19
`nh_simple`	non-hydrostatic Boussinesq equations on a uniform grid, periodic in x and y, with LES closures	29
`nh_realistic`	pe_realistic with a prognostic w and a non-hydrostatic pressure, Cartesian grid	38
`wim_simple`	spectral wave model on a Cartesian grid	46
`wim_realistic`	wim_simple on the sphere	71
—	the exact scattering integral S_{nl} of wim_simple	78

A realistic package is documented by its differences to the simple package. The documentation of pe_realistic thus needs the one of pe_simple. The same holds for nh_realistic, which needs pe_realistic and pe_simple, and for wim_realistic.

The routine tests and their results are described in `pyOM3/test/README_test_suite.md`. The coupled configurations are described in `setups/README.md`.

Practical tips for programming

This section lists the few things that are needed to write a configuration. Everything else is in the documentation of the packages.

The model object

A model is a Python class. A configuration derives a class from the class `model` of a package. It overloads the methods it needs, for instance `set_coriolis`, `set_topography`, `set_initial_conditions` and `set_forcing`. Additional parts such as the TKE closure of pe_realistic or the LES closures of nh_simple are further base classes in front, for instance `class model(tke, pe_realistic.model.model)`.

The object is created with the dictionary of parameters and the number of processes in x and y,

```
m = model(parameter, n_pes_x = 2, n_pes_y = 2)
```

Every entry of the dictionary becomes an attribute of the object. All fields, grid arrays and parameters of the model are attributes of this one object. In `setups/coupled_common/run.py` the object is called `M`, in the tests `m`.

The time stepping is started with `m.loop(n)` for n time steps. Before that, the pe family needs `m.prepare_for_run()` after the topography is set. nh_simple and the wave models need `m.setup()` after the object is created. The snapshot classes take the place of `model` when output is wanted.

Arrays and their dimensions

All fields carry halo points on every side of every axis. The size of an axis is therefore $N = n + 2h$ with n cells and $h =$ `halo` halo points. The interior of a three-dimensional field `a` is `a[h:-h, h:-h, h:-h]`.

The order of the axes differs between the packages.

Packages	Order	Shape
pe_simple, pe_realistic, nh_realistic	z, y, x	`(Nz, Ny, Nx)`
nh_simple	x, y, z	`(Nx, Ny, Nz)`
wim_simple, wim_realistic	direction, frequency, y, x	`(Nd, Nf, Ny, Nx)`

Two-dimensional fields of the pe and wim families have the shape `(Ny, Nx)`. nh_simple stores its arrays in Fortran order, since its pressure solver needs that. All other packages use the order of numpy.

In z the index `h` is the lowest cell and the index `-h-1` is the surface cell, in all packages with z. The tendencies have a first axis of length 3 for the time levels of the Adams–Bashforth step. The tendency of u at the current level is `du[tau]`.

The velocities sit on the faces of the cells, the tracers in the centres. u with index i sits at the eastern face of cell i. v with index j sits at the northern face of cell j. w with index k sits at the top face of cell k. The shift `rollx(a,-1)` gives the value at $i + 1$, and `rollx(a,1)` the value at $i - 1$. The same holds for `rolly` and `rollz`.

The main variables

Package	Variables
pe_simple	`u`, `v`, `w`, `temp`, `salt`, `rho`, the hydrostatic pressure `p_h`, the surface pressure `p_s`. The masks `maskT`, `maskU`, `maskV`, `maskW`, the bottom index `kbot`, the Coriolis parameter `coriolis_t`. The grid spacings are the numbers `dx`, `dy`, `dz`
pe_realistic	as pe_simple. The grid spacings are one-dimensional arrays `dxt`, `dxu`, `dyt`, `dyu`, `dzt`, `dzw` and the coordinates `xt`, `xu`, `yt`, `yu`, `zt`, `zw`, all with halo. `dxt_x` has the shape `(1,1,Nx)`, `dyt_y` the shape `(1,Ny,1)` and `dzt_z` the shape `(Nz,1,1)`, for use with three-dimensional fields. `dx`, `dy` and `dz` are None

Package	Variables
nh_realistic	as pe_realistic, and the non-hydrostatic pressure `p_nh`, the tendency `dw` and the mask `maskWc`
nh_simple	`u`, `v`, `w`, the buoyancy `b`, the pressure `p`. The grid spacings are the numbers `dx`, `dy`, `dz`
wim_simple, wim_realistic	the wave action density `A`

Writing to arrays

`OM.np` is numpy or jax.numpy, after `import pyOM3 as OM`. The choice is made by the environment variable `PYOM3_BACKEND`. Arrays of JAX cannot be changed in place. `a[...] = b` therefore fails with JAX. `a = OM.modify_array(a, index, b)` works with both and replaces `a[index]` by `b`. `OM.add_to_array` adds instead. The result must always be assigned.

After a field is set, `m.apply_bc(a)` fills its halo points. It exchanges them with the neighbouring processes or wraps them in a periodic direction. Without it the halo points keep wrong values.

Initial conditions with domain decomposition

With MPI each process holds the block of its cells and the halo around it. With JAX one array holds the whole domain, and every shard carries its own halo points inside that array. A field that is set by hand must follow this layout.

The simplest way in the pe family is `m.set_initial_conditions(u, v, temp, salt, p_s)`. It takes the fields of the whole domain without halo points, of the shape `(nz, ny, nx)`. It cuts out the block of the process with MPI and inserts the halo points with JAX. `p_s` is None for the rigid lid. Each process then holds the whole field once, which may need too much memory for a large model.

In pe_realistic and nh_realistic the coordinates `xt`, `yt` and `zt` already have the layout of the process. A field built from them is therefore correct with MPI and with JAX,

```
T = 10 + 1e-3*m.zt[:,None,None] + 1e-6*m.xt[None,None,:] \
       + 1e-6*m.yt[None,:,None]
m.temp = m.apply_bc(OM.np.asarray(T))*m.maskT
```

The three terms together give the shape `(Nz, Ny, Nx)`. This is the recommended way.

In pe_simple and wim_simple there are no coordinate arrays. With MPI the local index i has the global index $i-h+$`xs_pe`-1 of the interior cells, counted from 0. The same holds in y with `ys_pe`. The number of interior cells of the process is `Nx` $-\,2h$. In the pe family `xe_pe` should not be used, since it is `Nx` on a single process. With JAX the global index of a point depends on its shard. The function `global_index` of `pyOM3/test/pe/config.py` gives it for both cases.

nh_simple divides the domain in y and z and never in x. The local index i has the global index $i-h+$`xs_pe`-1, and the same holds with `ys_pe` and `zs_pe`. `pyOM3/test/nh/config.py` builds its initial fields in this way.

After the fields are set, `apply_bc` fills the halo points. In the pe family the density follows from `get_rho` of `pyOM3/density.py`. `set_initial_conditions` does both. nh_simple makes the initial velocity free of divergence with `m.pressure_solver()`. nh_realistic does this with `m.nh_projection()`.

The simple Primitive equation model

Consider the Boussinesq equations in hydrostatic approximation

$$\partial_t \boldsymbol{u} + f\underset{\neg}{\boldsymbol{u}} + \boldsymbol{\nabla} p = -\boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u} - w\partial_z \boldsymbol{u} + \boldsymbol{F}(\boldsymbol{u}) \quad , \quad \partial_t b = -\boldsymbol{u}\cdot\boldsymbol{\nabla} b - w\partial_z b + Mb \tag{1}$$

with the diagnostic relations $\partial_z p = b$ and $\boldsymbol{\nabla}\cdot\boldsymbol{u} + \partial_z w = 0$. Vectors are two-dimensional and $\underset{\neg}{\boldsymbol{u}}$ denotes anticlockwise rotation by 90°. Pressure is scaled with background density ρ_0 and thermodynamics have been simplified to a buoyancy equation. $\boldsymbol{F}$ and M are the friction and mixing operators. At the free surface $z = \eta$, we also have the condition

$$\partial_t p_s = -g\boldsymbol{\nabla}\cdot\int_{-H}^{\eta} \boldsymbol{u} dz \tag{2}$$

with $p_s = p|_{z=\eta} = g\eta$. Alternatively, we use a rigid lid with $w = 0$ at $z = 0$ and $p_s = p|_{z=0}$. We split full pressure p into hydrostatic part and surface pressure.

$$p = p_h(z) + p_s \ , \ p_h = \int_{-h}^{z} b dz' \tag{3}$$

where p_s is to be determined by the external mode solvers.

Simple implicit external solver for rigid lid

Velocity and buoyancy and pressure are all co-located in time in this scheme. The buoyancy equation is integrated as

$$b^{n+1} - b^n = \Delta t \left(\overline{\delta b}^{n+1/2} + M(b^n) \right) \tag{4}$$

with the advective buoyancy tendency

$$\delta b^n = -\boldsymbol{\nabla}\cdot\boldsymbol{u}^n b^n - \partial_z w^n b^n \quad , \quad \overline{\delta b}^{n+1/2} = A\delta b^n + B\delta b^{n-1} + C\delta b^{n-2} \tag{5}$$

interpolated with the Adam-Bashforth (AB3) scheme to time level $n+1/2$. The biharmonic horizontal mixing (attribute `Khbi`) is treated with an Euler forward step. Vertical mixing follows the options described in the section on vertical mixing and friction below.

An intermediate velocity $\boldsymbol{u}^*$ is calculated from

$$\boldsymbol{u}^* - \boldsymbol{u}^n = \Delta t \left(\overline{\delta \boldsymbol{u}}^{n+1/2} + \boldsymbol{F}(\boldsymbol{u}^n) \right) \tag{6}$$

with the momentum tendency $\delta\boldsymbol{u}$ excluding the surface pressure gradient and friction

$$\delta\boldsymbol{u}^n = -f\underset{\neg}{\boldsymbol{u}}^n - \boldsymbol{\nabla} p_h^n - \boldsymbol{u}^n\cdot\boldsymbol{\nabla}\boldsymbol{u}^n - w^n\partial_z\boldsymbol{u}^n \tag{7}$$

These tendencies are extrapolated with AB3 to time level $n + 1/2$

$$\overline{\delta\boldsymbol{u}}^{n+1/2} = A\delta\boldsymbol{u}^n + B\delta\boldsymbol{u}^{n-1} + C\delta\boldsymbol{u}^{n-2} \tag{8}$$

Friction is treated with an Euler forward step. From the vertically integrated continuity equation follows the condition

$$\boldsymbol{\nabla} \cdot \int_{-h}^{0} \boldsymbol{u}^* = \Delta t \boldsymbol{\nabla} \cdot h \boldsymbol{\nabla} p_s^{n+1} \tag{9}$$

which is solved with a conjugate-gradient iteration for p_s^{n+1} (attributes `epsilon` for the convergence threshold and `max_iterations`; a BiCGStab variant is implemented as well and is selected by editing the solver call in `pressure.py`). The iteration count and the estimated error are available per step as `congr_itts` and `estimated_error`. The intermediate velocity is then corrected as

$$\boldsymbol{u}^{n+1} = \boldsymbol{u}^* - \Delta t \boldsymbol{\nabla} p_s^{n+1} \tag{10}$$

It is possible to include an implicit free surface in the time stepping. Vertical velocity is calculated from continuity

$$w^{n+1} = -\int_{-h}^{z} \boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1} \tag{11}$$

This scheme is also used as default in the MITgcm.

Simple split-explicit external mode solver

Velocity is known in this scheme at $t = n + 1/2$, while b and p at level n. The momentum equation is discretised as

$$\boldsymbol{u}^* - \boldsymbol{u}^{n-1/2} = \Delta t \left(-f \underline{\overline{\boldsymbol{u}}}^{n} + \overline{\delta \boldsymbol{u}}^{n} - \boldsymbol{\nabla} p_h^n - \boldsymbol{\nabla} p_s^n + \boldsymbol{F}(\boldsymbol{u}^{n-1/2}) \right) \tag{12}$$

with the intermediate velocity $\boldsymbol{u}^*$ and AB3 extrapolation of Coriolis force and advection terms tendencies

$$\underline{\overline{\boldsymbol{u}}}^{n} = A \underline{\boldsymbol{u}}^{n-1/2} + B \underline{\boldsymbol{u}}^{n-3/2} + C \underline{\boldsymbol{u}}^{n-5/2} \quad , \quad \overline{\delta \boldsymbol{u}}^{n} = A \delta \boldsymbol{u}^{n-1/2} + B \delta \boldsymbol{u}^{n-3/2} + C \delta \boldsymbol{u}^{n-5/2} \tag{13}$$

and

$$\delta \boldsymbol{u}^{n-1/2} = -\boldsymbol{u}^{n-1/2} \cdot \boldsymbol{\nabla} \boldsymbol{u}^{n-1/2} - w^{n-1/2} \partial_z \boldsymbol{u}^{n-1/2} \tag{14}$$

For the hydrostatic and surface pressure $p_h + p_s$ no extrapolation is made, since they are at level n anyways, and friction is treated with Euler forward.

Now the momentum tendencies are vertically integrated but without surface pressure and Coriolis contributions.

$$\boldsymbol{R}^n = \int_{-h}^{0} \left(\overline{\delta \boldsymbol{u}}^{n} - \boldsymbol{\nabla} p_h^n + \boldsymbol{F}(\boldsymbol{u}^{n-1/2}) \right) dz \tag{15}$$

The external mode is integrated with sub-cycling over $m = 1, ...M$ (M set by the attribute `sub_cycling`) from n to $n+1$ with a simple, dissipative scheme proposed by Demange et al (2019) using the forcing $\boldsymbol{R}^n$

$$\boldsymbol{U}^{n+(m+1)/M} - \boldsymbol{U}^{n+m/M} = \Delta t/M \left(-f \overline{\boldsymbol{U}}^{n+(m+1/2)/M} - h \boldsymbol{\nabla} p_s^{n+m/M} + \boldsymbol{R}^n \right) \quad (16)$$

$$p_s^{n+(m+1)/M} - p_s^{n+m/M} = -g\Delta t/M \left((1+\theta) \boldsymbol{\nabla} \cdot \boldsymbol{U}^{n+(m+1)/M} - \theta \boldsymbol{\nabla} \cdot \boldsymbol{U}^{n+m/M} \right) \quad (17)$$

with the transport $\boldsymbol{U} = \int_{-h}^{0} \boldsymbol{u}\, dz$, and $\overline{\boldsymbol{U}}^{n+(m+1/2)/M}$ interpolated with AB2. Demange et al (2019) found $\theta = 0.14$ to be the best choice in terms of phase error and damping. In the model θ is the attribute `fb_theta` with default 0.14. Larger values damp grid-scale barotropic waves more strongly.

The sub cycle starts from the transport $\boldsymbol{U}^n$ and the surface pressure p_s^n which the sub cycle of the previous time step ended with. This transport is model state (`U_bt`, `V_bt`) and part of the restart. It is computed from the vertical integral of $\boldsymbol{u}^n$ only at a cold start. Two transports result from the sub cycle. The transport at the end of the sub cycle, $\boldsymbol{U}^{n+1} = \boldsymbol{U}^{n+M/M}$, starts the next sub cycle. The average over the sub cycle

$$\tilde{\boldsymbol{U}}^{n+1} = \frac{1}{M} \sum_{m=1}^{M} \boldsymbol{U}^{n+m/M} + \frac{\theta}{M} \left(\boldsymbol{U}^{n+1} - \boldsymbol{U}^n \right) \quad (18)$$

is used to correct the barotropic part of the intermediate velocity

$$\boldsymbol{u}^{n+1/2} = \boldsymbol{u}^* - 1/h \left(\int_{-h}^{0} \boldsymbol{u}^* dz - \tilde{\boldsymbol{U}}^{n+1} \right) \quad (19)$$

The next sub cycle must not start from the average $\tilde{\boldsymbol{U}}^{n+1}$. That would pair the averaged transport with the surface pressure at the end of the sub cycle. This mismatch grows from step to step when the Courant number of the external gravity wave is large (Banerjee et al. 2024, the split-explicit solver of FESOM2). The buoyancy equation is integrated using $\boldsymbol{u}^{n+1/2}$ and $w^{n+1/2} = -\int_{-h}^{z} \boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1/2}$ as

$$b^{n+1} - b^n = \Delta t \left(\delta b^{n+1/2} + M(b^n) \right) \quad (20)$$

with the buoyancy tendency

$$\delta b^n = -\boldsymbol{\nabla} \cdot \boldsymbol{u}^{n+1/2} b^n - \partial_z w^{n+1/2} b^n \quad (21)$$

interpolated with AB3 to time level $n + 1/2$. This scheme is one option in FESOM2.

In both schemes the advective tracer flux through the surface face is zero by default, as in pyOM2. Under the linear free surface the vertical velocity w at the surface face is not zero. A tracer that is uniform in space then changes in the surface cell at the rate $b\, w|_{z=0}/\Delta z = -b\, \boldsymbol{\nabla} \cdot \int_{-h}^{0} \boldsymbol{u}\, dz/\Delta z$. With the attribute `free_surface_tracer_flux` the flux through the surface face is $w\, b$ instead, with w at the surface face and b of the surface cell. A uniform tracer then stays uniform. This option is off by default. The external-mode treatment is selected by the attributes `implicit_free_surface` and `explicit_free_surface`: both false gives the rigid lid, the first the implicit free surface (both solved with the iterative solver above), the second the split-explicit scheme.

Vertical mixing and friction options

Vertical mixing of buoyancy and vertical friction are organised in a hierarchy of options similar to pyOM2, selected by the attribute `vmix_scheme` $\in$ {`'explicit'`, `'implicit'`} and two switches (module `implicit_vmix.py`; the previous flag `enable_implicit_vmix` maps to the implicit scheme with convective adjustment, so existing configurations are unchanged).

Explicit scheme. Fixed coefficients κ (buoyancy) and A_v (momentum) only, integrated forward in time,

$$b^{n+1} = b^n + \Delta t\, \partial_z \left(\kappa\, \partial_z b^n\right), \qquad \boldsymbol{u}^{n+1} = \boldsymbol{u}^{n+1} + \Delta t\, \partial_z \left(A_v\, \partial_z \boldsymbol{u}\right) \tag{22}$$

subject to the diffusive stability limit, which is checked at start-up (`check_vmix_stability`, called from `prepare_for_run`): $4 \max(\kappa, A_v)\, \Delta t / \Delta z^2 \leq 1/2$; the model refuses to run otherwise, and also refuses convective adjustment or the TKE closure under the explicit scheme.

Implicit scheme. Both operators are integrated with backward Euler (unconditionally stable, so arbitrarily large coefficients are allowed), as a tridiagonal solve per column with the coefficient located on the W points. Three sub-options for the coefficients: (i) constant κ and A_v; (ii) convective adjustment (`enable_conv_adjust`): background κ, but $\kappa = \kappa_{conv}$ (large) where $N^2 < 0$; (iii) the TKE closure (`enable_tke`), which overrides all constant coefficients for both buoyancy and momentum as described below. Vertical friction is applied after the surface pressure correction: since the vertical operator with zero-flux boundary conditions leaves the column-integrated momentum unchanged, the barotropic mode is not perturbed, and w is re-diagnosed afterwards.

Bottom friction. Two options act on the velocity $\boldsymbol{u}_b$ in the lowest wet cell of each U and V column. Both are off by default. Linear friction (`enable_bottom_friction`) has the rate $r = r_{bot}$ in 1/s (attribute `r_bot`). Quadratic friction has the rate $r = C_d |\boldsymbol{u}_b| / \Delta z$ with the drag coefficient C_d (attribute `r_quad_bot`). It is switched on by `enable_quadratic_bottom_friction`. Here $|\boldsymbol{u}_b|$ is the speed in the bottom cell. The other velocity component is averaged to the U or V point from the four surrounding points. If both options are on, the two rates add. The friction is an implicit step on the bottom cell,

$$\boldsymbol{u}_b^{new} = \boldsymbol{u}_b / \left(1 + \Delta t\, r\right)$$

In the rigid-lid and implicit free-surface scheme it is applied after the vertical friction. In the split-explicit scheme it is applied before the external mode is integrated. There the same bottom stress also acts on the transport inside the sub cycle. Every sub step divides $\boldsymbol{U}$ by $1 + (\Delta t / M)\, r\, \Delta z / h$.

TKE closure

The closure of Gaspar et al. (1990) is ported from pyOM2 (`tke.py`, mixin class). All closure constants are configuration attributes. They are set before `init_tke`. The defaults

are the pyOM2 `tke_module` defaults $c_k = 0.1$, $c_\epsilon = 0.7$, $\alpha = 1$, $\ell_{min} = 10^{-12}\,\mathrm{m}$ and $\kappa_M^{max} = 100\,\mathrm{m}^2/\mathrm{s}$. The floors κ_M^{min} and κ_H^{min} of viscosity and diffusivity are zero by default, as in pyOM2. With the TKE closure on, the constant coefficients A_v and κ of the hierarchy above are not used. A background viscosity or diffusivity is therefore a choice of the configuration. It is made by setting the floors. Note that the pyOM2 configurations of the acc family deviate from the module defaults and set $\alpha = 30$ (a proxy for vertical TKE transport by pressure work, which spreads the surface-injected TKE substantially deeper) and $\ell_{min} = 10^{-8}\,\mathrm{m}$; for consistency with those experiments both should be set accordingly (`alpha_tke`, `mxl_min`). The turbulent kinetic energy e is a prognostic variable on the W points,

$$\partial_t e = \partial_z\left(\alpha\,\kappa_M \partial_z e\right) + K_{diss} - P_{diss} - c_\epsilon\, e^{3/2}/\ell \tag{23}$$

with the mixing length $\ell = \sqrt{2e}/N$, limited such that it cannot grow faster than the distance to the boundaries (two sweeps, pyOM2's `tke_mxl_choice=2`), and the diffusivities

$$\kappa_M = \min(\kappa_M^{max},\, c_k\,\ell\sqrt{e}), \qquad \kappa_H = \kappa_M/\max(1, \min(10,\, 6.6\,Ri)), \qquad Ri = N^2\kappa_M/K_{diss} \tag{24}$$

There is no separate convective branch. Where $N^2 < 0$ the Richardson number is negative. The Prandtl number is then one and $\kappa_H = \kappa_M$, exactly as in pyOM2. This replaces the convective-adjustment option. The ratio K_{diss}/κ_M in Ri is the squared vertical shear of the friction operator. With `mix_eulerian` (see below) it is $\partial_z\bar{\boldsymbol{u}}\cdot\partial_z\left(\bar{\boldsymbol{u}} + \boldsymbol{u}_S\right)$. Without it, it is the squared shear of the model velocity. N^2 lives on the W points, i.e. on the top face of each cell. At the surface face there is no cell above, so N^2 is copied from the face below, as in pyOM2. For `eq_of_state` $= 5$ the density of the upper cell is evaluated at the pressure of the lower cell and compared with the density of the lower cell (adiabatic displacement, as in pyOM2).

Two sets of limits for the mixing length are available. They are selected by `mxl_choice`. The default `mxl_choice` $= 1$ is the pyOM2 form named above. Its sweeps run in the order of `tke.f90`. First a downward sweep from the surface limits $\ell_k \le \ell_{k+1} + \Delta z$. It starts from the unlimited value at the surface face. Then the value at the surface face is limited to $\ell \le \ell_{min} + z_0 + \Delta z$, with the roughness length z_0 given below. Finally an upward sweep from the bottom limits $\ell_k \le \ell_{k-1} + \Delta z$. The same length enters the diffusivity and the dissipation. With `mxl_choice` $= 2$ the limits of NEMO 4.2 are used (`nn_mxl` $= 3$ with `ln_mxl0`). All lengths live on the W points, i.e. at the top face of each tracer cell. The index n counts these points upward. Here $n = 1$ is the lowest point of the array and n_s the surface face. The starting point is the buoyancy length

$$\ell_n = \max\left(\sqrt{2e}/\sqrt{\max(N^2, 10^{-12}\,\mathrm{s}^{-2})},\ \ell_{min0}\right)$$

with $\ell_{min0} = 0.04\,\mathrm{m}$ (attribute `mxl_min0`). This floor is applied at all W points, also at dry points where $e = 0$. The surface value is then reduced to the surface length, $\ell_{n_s} = \min(\ell_{n_s}, \ell_0)$, with ℓ_0 given below. Since ℓ_0 is set before the downward sweep, it bounds the whole near-surface profile and not only the surface point. Two profiles are

formed from ℓ_n. Going downward from the surface the length is limited by the distance to the surface,

$$\ell^{up}_n = \min\left(\ell_n,\ \ell^{up}_{n+1} + \Delta z\right), \quad n = n_s - 1, \ldots, 1, \qquad \ell^{up}_{n_s} = \ell_{n_s}$$

Going upward from the bottom of the array the length is limited by the distance to the bottom,

$$\ell^{dwn}_n = \min\left(\ell_n,\ \ell^{dwn}_{n-1} + \Delta z\right), \quad n = 2, \ldots, n_s, \qquad \ell^{dwn}_1 = \ell_1$$

The two sweeps run over the whole array, dry points included. In a column whose lowest wet cell lies above the bottom of the array, the W point below the lowest wet cell is dry. There $\ell_n = \ell_{min0}$, so the lowest wet point gets $\ell^{dwn} \le \ell_{min0} + \Delta z$. In a column that is wet down to the bottom of the array the lowest point gets $\ell^{dwn}_1 = \ell_1$ without this limit. The diffusivity uses the geometric mean of the two profiles and the dissipation uses their minimum,

$$\ell_k = \sqrt{\ell^{up}\,\ell^{dwn}}, \qquad \ell_\epsilon = \min\left(\ell^{up},\ \ell^{dwn}\right)$$

ℓ_k replaces ℓ in the diffusivity κ_M, and ℓ_ϵ replaces ℓ in the dissipation $c_\epsilon e^{3/2}/\ell$. The constant c_ϵ multiplies the dissipation here. Writing the dissipation as $e^{3/2}/\ell_\epsilon$ with $\ell_\epsilon = \min(\ell^{up}, \ell^{dwn})/c_\epsilon$ is the same. The surface length ℓ_0 has two forms. Without a roughness length it follows Charnock's relation

$$\ell_0 = \kappa_{vk}\,\beta\,|\boldsymbol{\tau}|/g$$

with the kinematic surface stress $\boldsymbol{\tau}$, i.e. the stress divided by ρ_0, the von Kármán constant $\kappa_{vk} = 0.4$ (attribute `mxl_kappa`) and the Charnock constant β (attribute `mxl_beta`). The default $\beta = 2 \cdot 10^5$ is the value of NEMO. It gives $\ell_0 = 0.8\,\mathrm{m}$ at a stress of $0.1\,\mathrm{Pa}$. The value $\beta = 2 \cdot 10^4$ gives $\ell_0 = 0.08\,\mathrm{m}$ at the same stress. The modulus $|\boldsymbol{\tau}|$ is formed pointwise from `taux` and `tauy`, or taken from `tau_modulus` if the configuration provides it. With a roughness length $z_0 > 0$ (attribute `z0_surf`) the surface length is $\ell_0 = \kappa_{vk}\,z_0$ instead. The coupled configurations set $z_0 = 1.6\,H_s$ with the significant wave height H_s from the wave model. In both forms $\ell_0 \ge \ell_{min0}$. One halo exchange of ℓ_0 is made per time step. A configuration whose stress carries valid halo values can avoid it by setting `tau_halo_ok`.

The discretisation follows pyOM2: backward Euler with the dissipation linearised in the diagonal ($c_\epsilon\sqrt{e^n}/\ell$ acting on e^{n+1}), a surface W cell of half thickness $\Delta z/2$, and the surface flux boundary condition added to that cell,

$$F_e(0) = \begin{cases} \Phi_{oc} & \text{wave dissipation flux from the wave model} \\ (|\boldsymbol{\tau}|)^{3/2} & \text{classical parameterisation} \end{cases} \tag{25}$$

selected by `tke_surf_from_waves` (default on, falling back to the classical form if no wave flux is present; $\boldsymbol{\tau}$ kinematic; the first form closes the energy cycle between the wave and ocean components, the second is the pyOM2 default and serves as the control experiment). The classical form carries the factor one of pyOM2. Craig and Banner (1994) give

$\alpha_{cb}|\boldsymbol{\tau}|^{3/2}$ with $\alpha_{cb} \approx 100$. pe_simple has no attribute for this factor. A configuration that wants it overloads `_surface_forcing`, as `setups/coupled_common/exp_ocean.py` does with `EXP_ALPHA_CB`. No diffusive TKE, tracer or momentum flux passes through the surface face itself: the surface forcing enters only as the explicit flux above, which also removes a spurious shear production against the zero-velocity halo above the surface (a phantom no-slip lid). For wave-driven injection a surface roughness length z_0 can be set (attribute `z0_surf`, default 0, e.g. $1.6\,H_s$ from the wave model in the sense of Mellor & Blumberg 2004). With `mxl_choice` $= 1$ it only raises the limit of the value at the surface face. The downward sweep runs before that limit is applied, so the mixing length below the surface face does not change. With `mxl_choice` $= 2$ it sets the surface length ℓ_0 described above. If the surface cell is drained negative by buoyancy fluxes, the missing energy is added back and booked in `tke_surf_corr` as in pyOM2. There is no other floor on e, as in pyOM2. A floor $e \geq e_{min}$ is applied only if `tke_min` > 0 is set. The initial value of e in all wet cells is `tke_init` $= 10^{-12}\,\mathrm{m}^2/\mathrm{s}^2$, as in the pyOM2 configurations.

Vertical friction is computed implicitly with κ_M averaged to the U/V points as in pyOM2 ($\overline{\kappa_M}^i$). A momentum flux exists only across a face between two wet U (or V) cells, as in pyOM2. TKE is likewise exchanged only between two wet W levels. The shear production is diagnosed from the same discrete operator with the updated velocities,

$$K_{diss} = \overline{\overline{\kappa_M}^i \left(\partial_z u^{n+1}\right)^2}^i + \overline{\overline{\kappa_M}^j \left(\partial_z v^{n+1}\right)^2}^j \tag{26}$$

The backward-Euler friction step is $\boldsymbol{u}^{n+1} - \boldsymbol{u}^n = \Delta t\, \partial_z \left(\overline{\kappa_M}\, \partial_z \boldsymbol{u}^{n+1}\right)$. Multiplying it by $\boldsymbol{u}^{n+1}$ and integrating over the column gives the kinetic energy it removes,

$$\int_{-h}^{0} \frac{|\boldsymbol{u}^n|^2 - |\boldsymbol{u}^{n+1}|^2}{2\Delta t}\, dz = \int_{-h}^{0} \overline{\kappa_M} \left(\partial_z \boldsymbol{u}^{n+1}\right)^2 dz + \int_{-h}^{0} \frac{|\boldsymbol{u}^{n+1} - \boldsymbol{u}^n|^2}{2\Delta t}\, dz$$

The first term on the right is K_{diss} and is the forcing of the TKE equation. The second term is the numerical dissipation of the backward-Euler step. It is of order Δt and is not put into the TKE equation. The kinetic energy removed from the mean flow therefore reappears in the TKE forcing up to this term. $P_{diss} = \kappa_H N^2$ is the exact transfer of the implicit buoyancy mixing, evaluated with the *updated* buoyancy (the tracer solve precedes the TKE step; using the pre-mixing N^2 over-drains the TKE whenever $\kappa_H \Delta t/\Delta z^2$ is not small). At the surface face P_{diss} is the surface buoyancy flux instead, as in pyOM2,

$$P_{diss}(0) = -\frac{g}{\rho_0}\left(\frac{\partial \rho}{\partial T} F_T + \frac{\partial \rho}{\partial S} F_S\right)$$

Here F_T and F_S are the surface fluxes of temperature and salt into the ocean ($\mathrm{K\,m/s}$ and $\mathrm{psu\,m/s}$, positive into the ocean). The configuration provides them as the attributes `forc_temp_surface` and `forc_salt_surface`. Surface cooling is thus a source of TKE. Without these attributes the term is zero. For `eq_of_state` $= 0$ the temperature is the buoyancy b and $\partial\rho/\partial b = -\rho_0/g$ is used. pyOM2 uses zero there and has no surface buoyancy source of TKE for this equation of state. The surface W cell has the thickness

$\Delta z/2$. The TKE that enters it per unit area and time is therefore $F_e(0)+(\Delta z/2)\,J_s$ plus the correction `tke_surf_corr` described above. Here $J_s = -P_{diss}(0)$ is the surface buoyancy loss. In stand-alone column tests the discrete TKE budget (surface flux + shear production = dissipation + buoyancy destruction + storage) closes to machine precision.

When the model velocity is the Lagrangian mean $\boldsymbol{u}_L$ of the coupled wave–ocean formulation, the attribute `mix_eulerian` (default off, requires the Stokes velocity to be present) lets the friction operator act on the Eulerian mean velocity $\bar{\boldsymbol{u}} = \boldsymbol{u}_L - \boldsymbol{u}_S$: the down-gradient closure parameterises Eulerian fluctuations, so mixing $\boldsymbol{u}_L$ would spuriously diffuse the Stokes shear. The Stokes velocity is subtracted before the implicit solve and added back afterwards, and the shear-production booking becomes flux times *Lagrangian* shear, split into the non-negative Eulerian part $K^E_{diss} = \kappa_M(\partial_z\bar{\boldsymbol{u}})^2$ and the sign-indefinite Stokes (Langmuir) part $K^L_{diss} = \kappa_M\partial_z\bar{\boldsymbol{u}}\cdot\partial_z\boldsymbol{u}_S$, kept separately in the diagnostics with $K_{diss} = K^E_{diss} + K^L_{diss}$ feeding the TKE equation. Without `mix_eulerian` this reduces bit-identically to the plain booking above.

The calling sequence per time step is: diffusivities from the current e and N^2 at the start of the step (so buoyancy mixing and friction use the same coefficients the transfers are paired with), vertical friction after the pressure correction, and the TKE step at the end of the time step. The density is evaluated once per time step, as in pyOM2. This happens after the vertical mixing of the tracers and before the TKE step. N^2 and P_{diss} of the TKE step use this updated density. The hydrostatic pressure of the next time step is computed from the same density.

Model parameters and options

All parameters and options are attributes of the model instance, set either through the parameter dictionary passed at construction or by assigning the attribute before `prepare_for_run`; options with a default may be left unset. The tables list every attribute the model reads.

Required parameters (no default)

Attribute	Unit	Meaning
`nx`, `ny`, `nz`	—	global grid size (cells, without halo points)
`dx`, `dy`, `dz`	m	grid spacing; Δz is a uniform scalar
`dt`	s	time step
`eq_of_state`	—	equation of state: 0 (temperature carries the buoyancy, no salinity), 1 (linear $\rho(S,T)$, salinity prognostic), 5 (GSW/TEOS-10, salinity prognostic, the pressure is the depth of the cell centre below the surface in dbar), 100 (ideal-gas atmosphere)
`implicit_free_surface`	—	implicit free surface if true

Attribute	Unit	Meaning
`explicit_free_surface`	—	split-explicit free surface if true; both false: rigid lid
`periodic_in_x`, `periodic_in_y`	—	lateral boundaries periodic (true) or closed (false)
`epsilon`	—	convergence threshold of the iterative surface-pressure solver (rigid lid / implicit free surface)
`max_iterations`	—	iteration cap of that solver
`sub_cycling`	—	number of barotropic sub-cycles M (split-explicit scheme only)

The constructor additionally takes `n_pes_x`, `n_pes_y` (processor grid, default 1×1), `halo_points` (default 1) and `comm_in` (default: MPI world communicator). The constants `grav` = 9.81 and `rho_0` = 1024 are set internally (note: they overwrite any value passed in the parameter dictionary). Coriolis parameter, topography (`kbot`) and initial conditions are set through the template methods `set_coriolis`, `set_topography` and `set_initial_conditions`, overloaded in the configuration.

Forcing hooks and coupling fields

Attribute / method	Meaning
`set_forcing(tau)`	optional user method; adds forcing to the momentum tendencies `du[tau]`, `dv[tau]` (enters the AB interpolation). Surface fluxes of heat and salt are added to `dtemp[tau]` and `dsalt[tau]` in the same way. Both schemes keep these tracer increments
`set_forcing_direct()`	optional user method; direct momentum increment bypassing the AB weights, applied before the pressure solve. Used for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation, which must be a plain forward increment. It is called only in the rigid-lid and implicit free-surface scheme. The split-explicit scheme does not call it
`forc_temp_surface`	surface flux of temperature into the ocean (K m/s). It gives the surface buoyancy source $P_{diss}(0)$ of the TKE equation. It does not force the tracer. That is done in `set_forcing`
`forc_salt_surface`	the same for salt (psu m/s)
`tau_modulus`	optional modulus of the kinematic surface stress at the T points, used by `mxl_choice` = 2 instead of the pointwise modulus of `taux`, `tauy`
`taux`, `tauy`	kinematic wind stress; also the fallback surface TKE forcing $\lvert\boldsymbol{\tau}\rvert^{3/2}$

Attribute / method	Meaning
`phi_oc`	wave-breaking energy flux from the wave model; surface TKE flux when `tke_surf_from_waves` is on
`u_S`, `v_S`	Stokes velocity from the wave model; required by `mix_eulerian`

Options with defaults

Attribute	Default	Meaning
`superbee_advection`	unset	if set true, tracer advection uses the Superbee flux limiter instead of the second-order fluxes (momentum advection is always second order). Note: setting it to false raises an error; leave it unset for the second-order scheme
`Khbi`	unset	biharmonic horizontal tracer diffusion coefficient (m^4/s); active if present, forward Euler
`Ahbi`	unset	biharmonic horizontal friction coefficient (m^4/s); active if present, forward Euler. In the rigid-lid and implicit free-surface scheme it is applied to $\boldsymbol{u}$ before the AB update. In the split-explicit scheme it is a tendency in the forcing $\boldsymbol{R}^n$ of the external mode
`vmix_scheme`	`'explicit'`	vertical mixing/friction hierarchy: `'explicit'` (forward Euler, coefficient checked against the diffusive stability limit) or `'implicit'` (backward Euler, unconditionally stable). If unset, the legacy flag `enable_implicit_vmix` decides
`kappa_T`	0	background vertical diffusivity (m^2/s)
`A_v`	0	background vertical viscosity (m^2/s)

Attribute	Default	Meaning
`enable_conv_adjust`	on iff implicit	convective adjustment: $\kappa = \kappa_{conv}$ where $N^2 < 0$ (implicit scheme only)
`kappa_conv`	1	convective-adjustment diffusivity (m^2/s)
`enable_tke`	off	TKE closure; overrides the constant coefficients for both buoyancy and momentum (implicit scheme only)
`enable_bottom_friction`	off	linear bottom friction on the lowest wet cell, rate `r_bot`
`r_bot`	0	rate of the linear bottom friction (1/s)
`enable_quadratic_bottom_friction`	off	quadratic bottom friction on the lowest wet cell, rate $C_d\|\boldsymbol{u}_b\|/\Delta z$
`r_quad_bot`	0	drag coefficient C_d of the quadratic bottom friction
`fb_theta`	0.14	dissipation parameter θ of the split-explicit sub cycle
`free_surface_tracer_flux`	off	tracer flux through the surface face. Off means zero flux, as in pyOM2. On means $w\,b$ with the surface-cell value, so a uniform tracer stays uniform

TKE closure constants (read at `init_tke`)

Attribute	Default	Meaning
`c_k`	0.1	diffusivity constant, $\kappa_M = c_k \ell \sqrt{e}$
`c_eps`	0.7	dissipation constant, $c_\epsilon e^{3/2}/\ell$
`alpha_tke`	1	vertical TKE diffusion $\alpha\kappa_M$; the pyOM2 acc configurations use 30
`mxl_min`	10^{-12}	mixing-length floor (m); acc configurations use 10^{-8}
`kappaM_min`	0	viscosity floor (m^2/s), as in pyOM2. A background viscosity is set here
`kappaM_max`	100	viscosity cap (m^2/s)
`kappaH_min`	0	diffusivity floor (m^2/s). A background diffusivity is set here
`tke_min`	0	TKE floor (m^2/s^2), applied only if positive

Attribute	Default	Meaning
`tke_init`	10^{-12}	initial TKE in wet cells (m^2/s^2)
`mxl_choice`	1	limits of the mixing length. 1 is pyOM2, 2 is NEMO 4.2 with a separate length for the dissipation
`mxl_kappa`	0.4	von Karman constant of the surface length (`mxl_choice` = 2)
`mxl_beta`	$2 \cdot 10^5$	Charnock constant β of the surface length (`mxl_choice` = 2)
`mxl_min0`	0.04	minimum surface and bottom length (m) (`mxl_choice` = 2)
`tau_halo_ok`	off	set when `taux`, `tauy` carry valid halo values. Saves the halo exchange of the surface length (`mxl_choice` = 2)
`z0_surf`	0	surface roughness length (m), e.g. $1.6\,H_s$. Limits the value at the surface face (`mxl_choice` = 1) or sets the surface length (`mxl_choice` = 2)
`tke_surf_from_waves`	on	surface TKE flux from the wave dissipation Φ_{oc}; falls back to $\lvert\boldsymbol{\tau}\rvert^{3/2}$ if no wave flux is present
`mix_eulerian`	off	vertical friction acts on the Eulerian mean velocity $\boldsymbol{u}_L - \boldsymbol{u}_S$ ($\boldsymbol{u}_L$ formulation), with the K^E_{diss}/K^L_{diss} booking split

Restart state

The model has no restart routine of its own. A configuration that continues a run must save the following fields and set them again before the next step. A restart is then bit-identical to the uninterrupted run. This is tested by `pyOM3/test/pe/restart.py`, which holds the reference list.

- Always the fields `u`, `v`, `w`, `temp`, `salt`, `rho`, `p_h` and `p_s`.
- Always the three time levels of the tendencies `du`, `dv`, `dtemp` and `dsalt`.
- The time-level pointers `tau`, `taum1`, `taum2` and the AB coefficients of the driving loop. The method `loop` starts every call with a forward Euler step and the pointers $2, 1, 0$. A restart must therefore drive the time steps itself.
- With the TKE closure `tke` and `K_diss_v`. The diffusivities of a step are built from K_{diss} of the previous step through the Richardson number. Without `K_diss_v` the first step after the restart differs.

- With the split-explicit scheme `du_cor`, `dv_cor`, `U_bt` and `V_bt`.
- With `mxl_choice` = 2 `taux` and `tauy` (or `tau_modulus`). The closure uses them at the start of the step, before `set_forcing` sets the new stress.

Snapshot diagnostics

Two snapshot classes are available; either replaces the model class as the base of the configuration and overrides `prepare_for_run` and `diagnose`. Both read the attribute `snapint` (snapshot interval in seconds, required) and take the `prepare_for_run` arguments `snap_file_name` and `show_each_time_step` (default on: print every time step; off: print at output steps only). When the iterative surface-pressure solver is active, both print its per-step iteration count and accumulate the total count between outputs.

snap_nc4 writes a single netCDF file (default name `'snap.cdf'`), created with the T- and U-grid coordinate axes and the variables u, v, w, temp, p_h, ρ and p_s; salinity is included when the equation of state carries it (`eq_of_state` > 0 and $\neq 100$). At every output step each PE in turn writes its halo-free sub-block into the global arrays of the shared file (serialised with barriers). For the rigid-lid/implicit path the current time level of the surface pressure is written, for the split-explicit path the plain 2D field. This class works with the MPI domain decomposition and with single-process JAX, but not in a multihost JAX environment, since all processes must be able to open the same file.

snap_raw writes one `.npy` file per variable, per PE and per output step into a directory tree `<name>/pe=<rank>/n=<step>/` (default name `'raw_snap'`; an existing directory of that name is removed at start-up), with the coordinate axes and the processor-grid layout (`decomposition.npy`) stored once at the top level. Each process only touches its own subdirectory, and JAX sharded arrays are written through their local shard, so this class also works in a multihost JAX environment. The variable set is u, v, w, temp, salt, p_h and p_s (salt always, ρ not included). The fragmented output is meant to be reassembled afterwards using the stored decomposition, e.g. into zarr.

The realistic primitive equation model pe_realistic

pe_realistic solves the same equations as pe_simple with the same time stepping. This text describes only the differences to pe_simple. Everything that is not mentioned here is as described in the documentation of pe_simple.

The differences are the following. The grid spacings may vary in all three directions. The horizontal coordinates may be longitude and latitude on the sphere, as in pyOM2. The equation of state may be the one of TEOS-10. Isoneutral mixing is available. Horizontal momentum friction uses the discrete operators of pyOM2. Tracer advection can be taken out of the Adams–Bashforth step.

pyOM2 is the reference for all discrete operators that are new here. They were ported from pyOM2 and compared with pyOM2 in twin experiments. The tests and their results are listed in `pyOM3/pe_realistic/README_test_pe_realistic.md`.

The domain decomposition, the halo exchange and the iterative solver for the surface pressure are not part of the package. They are imported from pe_simple. The grid is built by the class `grid` in `grid.py`.

Grid

The model domain consists of $n_x \times n_y \times n_z$ cells. The horizontal coordinates x and y are either Cartesian coordinates in metres or longitude and latitude in degrees. The attribute `coord_degree` selects longitude and latitude. The vertical coordinate z is in metres.

The widths of the cells are given by the attributes `dxt`, `dyt` and `dzt`. Each of them is either a number or an array of length n_x, n_y or n_z without halo points. The numbers `dx`, `dy` and `dz` of pe_simple are accepted instead. Without either of them the model stops with an error.

A cell has its centre at a T point and its faces at U points. In x the cell i has the centre x^t_i, the width Δx^t_i and the eastern face x^u_i. The distance between the centres x^t_i and x^t_{i+1} is Δx^u_i. The same holds in y with y^t_j, Δy^t_j, y^u_j, Δy^u_j. In z the face on top of cell k is the W point z^w_k, and the distance between the centres z^t_k and z^t_{k+1} is Δz^w_k.

The grid is built from the widths as in pyOM2 (`u_centered_grid` of `numerics.f90`). The faces are summed from the widths. The centres follow from the faces by

$$x^u_i = x^u_{i-1} + \Delta x^t_i \ , \quad x^t_{i+1} = 2x^u_i - x^t_i \ , \quad \Delta x^u_i = x^t_{i+1} - x^t_i \tag{27}$$

with the start $x^t_0 = x^u_0 - \Delta x^t_0/2$ and the end $\Delta x^u_n = 2\Delta x^t_n - \Delta x^u_{n-1}$ at the last cell n. This recursion keeps the discrete gradient and divergence adjoint on a stretched grid.

On a stretched grid x^t_i is not the midpoint between the faces x^u_{i-1} and x^u_i. This follows from Eq. (27). It matters wherever a depth is taken from z^t, for instance in the equation of state below.

In x and y the widths are first extended by the halo points. In a periodic direction the halo widths are copies of their periodic images. In a closed direction they repeat the width of the edge cell. The recursion Eq. (27) then runs over the extended array. The first

interior U point is placed at `x_origin` and `y_origin` (default 0). In a periodic direction the halo coordinates are copies of the coordinates of their periodic images. They are used only for the metric factors below.

In z the recursion Eq. (27) runs over the n_z interior cells only, as in pyOM2. The coordinates are then extended linearly into the halo, and the halo widths repeat the edge widths. The array `dzt` is given from the bottom to the surface. The cell $k =$ `halo` is the lowest cell and the cell $k =$ `Nz-halo-1` is the surface cell, as in pe_simple.

The end of the recursion, $\Delta z^w_n = 2\Delta z^t_n - \Delta z^w_{n-1}$, can be zero or negative. This happens for instance when Δz^t grows by a constant ratio from cell to cell. The model then stops with an error. Δz^t must therefore vary smoothly.

The origin of z is the bottom face of the lowest cell by default. z is then the height above the bottom, as in pe_simple. With the attribute `set_zero_at_surface` the origin is the surface face instead, as in pyOM2.

With `coord_degree` the coordinates x and y are longitude λ and latitude ϕ in degrees. The widths are given in degrees as well. The coordinates are built from the widths in degrees. After that the widths are converted to metres with the factor $a\pi/180$. The radius of the earth is $a = 6370\,\mathrm{km}$ (attribute `radius`). π is the double precision value. pyOM2 uses a single precision value, which gives relative differences of $3 \cdot 10^{-8}$ in all metric quantities.

The metric factors are $\cos\phi$ at the T rows (`cost`), $\cos\phi$ at the U rows (`cosu`) and $\tan\phi/a$ at the T rows (`tantr`). In Cartesian coordinates they are 1, 1 and 0. The horizontal areas of the cells are

$$A^t = \cos\phi^t\,\Delta x^t \Delta y^t \;, \quad A^u = \cos\phi^t\,\Delta x^u \Delta y^t \;, \quad A^v = \cos\phi^u\,\Delta x^t \Delta y^u \tag{28}$$

for the T, U and V cells, with Δx and Δy in metres.

The attributes `dx`, `dy` and `dz` are set to None after the grid is built. Code from pe_simple that still uses them therefore stops with an error.

The grid arrays are one-dimensional. With MPI every process holds its part including the halo points. With JAX every shard holds its own halo points, as all fields do in pe_simple. With JAX n_x and n_y must be divisible by the number of shards in x and y.

Discrete operators

All operators of pe_simple are written in flux form. In pe_realistic every flux is multiplied by the width of the face it crosses. Every flux divergence is divided by the volume of the cell. On a uniform Cartesian grid every operator reduces to the one of pe_simple. This is tested to round-off.

The vertical velocity at the top face of cell k follows from continuity,

$$w_k = -\sum_{k' \le k} \Delta z^t_{k'} \left(\frac{u_i - u_{i-1}}{\cos\phi^t_j\,\Delta x^t_i} + \frac{\cos\phi^u_j\,v_j - \cos\phi^u_{j-1}\,v_{j-1}}{\cos\phi^t_j\,\Delta y^t_j} \right) \tag{29}$$

with u and v of cell k' and the sum over the wet cells of the column.

The tracer fluxes are those of pe_simple. The meridional flux is multiplied by $\cos\phi^u$. The flux divergence is divided by $\cos\phi^t\Delta x^t$, $\cos\phi^t\Delta y^t$ and Δz^t. The Superbee scheme uses the Courant numbers $|u|\Delta t/(\cos\phi^t\Delta x^t)$, $|\cos\phi^u v|\Delta t/(\cos\phi^t\Delta y^t)$ and $|w|\Delta t/\Delta z^t$.

The momentum advection is that of pe_simple with volume transports. The transports through the faces are $u\Delta z^t\Delta y^t$, $v\Delta z^t\cos\phi^u\Delta x^t$ and wA^t. The divergence is divided by the volume $A^u\Delta z^t$ of the U cell or $A^v\Delta z^t$ of the V cell.

The Coriolis force averages fv to the U point and fu to the V point with the widths of the cells as weights, as in pyOM2,

$$(\partial_t u)_{cor} = \frac{1}{4\Delta x^u_i}\left(f_i\,\Delta x^t_i\,(v_{i,j}+v_{i,j-1})+f_{i+1}\,\Delta x^t_{i+1}\,(v_{i+1,j}+v_{i+1,j-1})\right) \tag{30}$$

$$\begin{aligned}(\partial_t v)_{cor} = &-\frac{1}{4\cos\phi^u_j\,\Delta y^u_j}\left(f_j\cos\phi^t_j\Delta y^t_j\,(u_{i,j}+u_{i-1,j})\right.\\ &\left.+f_{j+1}\cos\phi^t_{j+1}\Delta y^t_{j+1}\,(u_{i,j+1}+u_{i-1,j+1})\right)\end{aligned} \tag{31}$$

with the Coriolis parameter f at the T points. On a uniform grid this is the average of pe_simple.

On the sphere the momentum equations contain the metric terms $+uv\tan\phi/a$ and $-u^2\tan\phi/a$. They are discretised with the same weights as the Coriolis force,

$$(\partial_t u)_{met} = \frac{1}{8\Delta x^u_i}\left(\left(\frac{\tan\phi}{a}\,\Delta x^t(u_i+u_{i-1})(v_j+v_{j-1})\right)_i+(\cdots)_{i+1}\right) \tag{32}$$

$$(\partial_t v)_{met} = -\frac{1}{8\cos\phi^u_j\,\Delta y^u_j}\left(\left(\frac{\tan\phi}{a}\cos\phi^t\Delta y^t(u_i+u_{i-1})^2\right)_j+(\cdots)_{j+1}\right) \tag{33}$$

where $(\cdots)$ repeats the bracket before it at the next T point. In Cartesian coordinates both terms vanish.

The pressure gradients are $-(p_{i+1}-p_i)/(\cos\phi^t\Delta x^u_i)$ and $-(p_{j+1}-p_j)/\Delta y^u_j$. This holds for the hydrostatic pressure p_h and for the surface pressure p_s.

The hydrostatic pressure is integrated downward from the surface cell with the distances Δz^w,

$$p_h(n_z)=\frac{g}{2\rho_0}\rho(n_z)\,\Delta z^w(n_z)\ ,\quad p_h(k)=p_h(k+1)+\frac{g}{2\rho_0}\left(\rho(k+1)+\rho(k)\right)\Delta z^w(k) \tag{34}$$

where n_z denotes the surface cell here.

The surface pressure of the rigid lid and of the implicit free surface follows from the vertically integrated continuity equation, as in pe_simple. The depth of the water column at the U and V points is $h^u=\sum\Delta z^t$ and $h^v=\sum\Delta z^t$ over the wet U and V cells. The discrete equation for p_s is

$$\begin{aligned}&\frac{1}{\cos\phi^t\Delta x^t_i}\left(\frac{h^u_i(p_{i+1}-p_i)}{\cos\phi^t\Delta x^u_i}-\frac{h^u_{i-1}(p_i-p_{i-1})}{\cos\phi^t\Delta x^u_{i-1}}\right)\\ +\ &\frac{1}{\cos\phi^t_j\Delta y^t_j}\left(\frac{\cos\phi^u_j h^v_j(p_{j+1}-p_j)}{\Delta y^u_j}-\frac{\cos\phi^u_{j-1}h^v_{j-1}(p_j-p_{j-1})}{\Delta y^u_{j-1}}\right)=\frac{1}{\Delta t}\,\boldsymbol{\nabla}\cdot\int\boldsymbol{u}^*dz\end{aligned}$$

with the divergence on the right discretised as for w above. The implicit free surface adds $-p_s/(g\Delta t^2)$ on the left as in pe_simple. The equation is solved with the conjugate-gradient solver of pe_simple.

The rigid lid does not converge on a domain that is periodic in x and contains land. pyOM2 uses a streamfunction with island integrals there, which pe_realistic does not have. Such domains need the implicit free surface.

The vertical mixing of tracers and momentum uses the widths Δz^t and the distances Δz^w,

$$(\partial_t T)_{mix} = \frac{1}{\Delta z^t_k}\left(\kappa_k \frac{T_{k+1}-T_k}{\Delta z^w_k} - \kappa_{k-1}\frac{T_k - T_{k-1}}{\Delta z^w_{k-1}}\right)$$

with the coefficient κ_k at the top face of cell k. The stability limit of the explicit scheme becomes $4\max(\kappa, A_v)\Delta t/\min(\Delta z^t \Delta z^w) \le 1/2$.

Horizontal friction

Horizontal momentum friction uses the operators of pyOM2 (`harmonic_friction.f90` and `biharmonic_friction.f90`). They place the fluxes and the metric factors on the rows of the U and V points. pe_simple uses an operator at the T points for u and v instead. On a grid with varying Δy or on the sphere that operator is not exact for u and v.

For u the harmonic operator with coefficient A is

$$\begin{aligned} F_u &= \frac{f^x_i - f^x_{i-1}}{\cos\phi^t_j\,\Delta x^u_i} + \frac{f^y_j - f^y_{j-1}}{\cos\phi^t_j\,\Delta y^t_j} \\ f^x_i &= A\,(\cos\phi^t_j)^p\,\frac{u_{i+1}-u_i}{\cos\phi^t_j\,\Delta x^t_{i+1}}\ , \quad f^y_j = A\,(\cos\phi^u_j)^p\cos\phi^u_j\,\frac{u_{j+1}-u_j}{\Delta y^u_j} \end{aligned}$$

A flux exists only between two wet U cells.

For v the harmonic operator is

$$\begin{aligned} F_v &= \frac{f^x_i - f^x_{i-1}}{\cos\phi^u_j\,\Delta x^t_i} + \frac{f^y_j - f^y_{j-1}}{\cos\phi^u_j\,\Delta y^u_j} \\ f^x_i &= A\,(\cos\phi^u_j)^p\,\frac{v_{i+1}-v_i}{\cos\phi^u_j\,\Delta x^u_i}\ , \quad f^y_j = A\,(\cos\phi^t_{j+1})^p\cos\phi^t_{j+1}\,\frac{v_{j+1}-v_j}{\Delta y^t_{j+1}} \end{aligned}$$

A flux exists only between two wet V cells.

The harmonic friction is $\boldsymbol{u} \leftarrow \boldsymbol{u} + \Delta t\,(F_u, F_v)$ with $A =$ `Ah`. The exponent p is `hor_friction_cosPower` (default 0). It is applied only in the rigid-lid and implicit free-surface scheme.

The biharmonic friction applies the harmonic operator twice, $\boldsymbol{u} \leftarrow \boldsymbol{u} - \Delta t\,F(F(\boldsymbol{u}))$. Each pass uses $A = \sqrt{A_{hbi}}$ with $A_{hbi} =$ `Ahbi`. The exponent p is `biha_friction_cosPower` (default 0). Since $(\cos\phi)^p$ enters each pass, the viscosity scales with $(\cos\phi)^{2p}$. In the split-explicit scheme the biharmonic friction enters the forcing of the external mode as the tendency $-F(F(\boldsymbol{u}))$ of the same operator.

The biharmonic mixing of tracers (`Khbi`) uses the operator at the T points with the metric factors of the tracer fluxes above.

The bottom friction is that of pe_simple. The thickness of the bottom cell is Δz^t of that cell instead of Δz.

Split-explicit external mode

The split-explicit scheme is that of pe_simple. The Coriolis force on the transports uses the weights of the Coriolis force above. The pressure gradient and the divergence of the transports carry the metric factors above.

On the sphere the external mode also contains the metric terms. They are the vertical integrals of $+uv \tan\phi/a$ and $-u^2 \tan\phi/a$ with $u = U/h^u$ and $v = V/h^v$. They are discretised as the metric terms above and multiplied by h^u and h^v. The metric terms of the baroclinic part are already part of the Coriolis tendency of the three-dimensional velocity.

Equation of state

The attribute `eq_of_state` selects the equation of state. The value 0 means that temperature is buoyancy. The value 1 is the linear equation of state. The value 5 is the 48-term equation of state of TEOS-10 (GSW), as in pyOM2. The value 100 is the ideal gas of the atmosphere. The density is the anomaly to $\rho_0 = 1024\,\mathrm{kg/m^3}$, as in pyOM2.

The pressure in the equation of state of TEOS-10 is the depth of the cell below the surface face in dbar, as in pyOM2. It is z^w of the surface face minus z^t of the cell. It is not $z^t + \Delta z^t/2$, since z^t is not the midpoint of the cell on a stretched grid. The value 100 needs the default origin of z at the bottom.

N^2 for the value 5 uses the adiabatic displacement as in pe_simple. The density is evaluated once per time step as in pe_simple.

TKE closure

The TKE closure is that of pe_simple with the following changes. The mixing length sweeps use the widths Δz^t instead of Δz. The surface value of `mxl_choice` $= 1$ is limited by $\ell_{min} + z_0 + \Delta z^t$ of the surface cell. The vertical shear and N^2 use the distances Δz^w. The surface W cell has the thickness $\Delta z^w/2$ of the surface face.

The shear production K_{diss} is computed at the U and V points and averaged to the T points. The average is weighted with the areas, as in pyOM2,

$$K^t_i = \frac{A^u_i K^u_i + A^u_{i-1} K^u_{i-1}}{2 A^t_i} \tag{35}$$

and correspondingly in y with A^v. An average without the areas would change K_{diss} by 6% at 60° latitude.

Isoneutral mixing

Isoneutral mixing is switched on by `enable_neutral_diffusion`. It is a port of `isoneutral.f90` and `isoneutral_diffusion.f90` of pyOM2. It uses the triads of Griffies et al., as pyOM2 does.

The isoneutral slopes follow from the gradients of temperature and salinity and the derivatives of density. For `eq_of_state` $= 0$ the derivative with respect to temperature is $-\rho_0/g$. pyOM2 uses zero there. For the value 1 the derivatives are the constants of the linear equation of state. For the value 5 they are the derivatives of the TEOS-10 equation of state. The value 100 is not possible.

Steep slopes are tapered with

$$\tau(s) = \frac{1}{2}\left(1 + \tanh\frac{s_c - |s|}{\Delta s}\right) \tag{36}$$

with $s_c =$ `iso_slopec` (default 10^{-3}) and $\Delta s =$ `iso_dslope` (default $8 \cdot 10^{-4}$).

The diffusivity is the three-dimensional field `K_iso`. It is set to the constant `K_iso_0` at the start unless the configuration has set the field. The horizontal diffusivity in the tensor is $\max(K_{steep}, K_{iso}\,\tau)$ with $K_{steep} =$ `K_iso_steep`. The field can be changed during the run, for instance by another parameterisation.

With `enable_skew_diffusion` the skew flux of Gent and McWilliams is added, as in pyOM2. Its diffusivity is the three-dimensional field `K_gm`. It is set to the constant `K_gm_0` at the start unless the configuration has set the field. On the lateral faces the triads then use $K_{iso} - K_{gm}$. On the top faces they use $K_{iso} + K_{gm}$.

The horizontal and mixed parts of the tensor are applied with a forward Euler step. The vertical part K_{33} is applied with the implicit vertical diffusion of the model. Isoneutral mixing acts on temperature and, for an equation of state with salinity, on salinity.

At the start the model checks the stability of the explicit part, as pyOM2 does. The quantity

$$\delta = \min\left(\frac{\cos\phi^t \Delta x^t \Delta z^t}{4K_{iso,0}\Delta t},\ \frac{\Delta y^t \Delta z^t}{4K_{iso,0}\Delta t}\right)$$

is the steepest slope that is stable. The model stops with an error if δ is smaller than s_c.

The following parts of pyOM2 are not ported. These are the dissipation diagnostics of the isoneutral and skew fluxes, the treatment in the mixed layer and the friction of the TEM formulation.

Tracer advection outside the Adams–Bashforth step

With thin cells on a fine horizontal grid the vertical Courant number $|w|\Delta t/\Delta z^t$ can limit the time step. The Superbee scheme is stable up to a Courant number of 1 in a forward step. Inside the Adams–Bashforth step with the weights 1.6 and -0.6 it becomes unstable above about 0.45. The attribute `vadv_sub_cycling` $= M$ therefore takes the tracer advection

out of the Adams–Bashforth step. The default $M = 0$ keeps the advection in the Adams–Bashforth step.

With $M > 0$ the advection uses the velocities u, v and w at the end of the time step. It consists of a step in x, a step in y and M steps in z with the time step $\Delta t/M$. Each of them is a forward step of the Superbee scheme, as in MITgcm. The vertical Courant number may then be as large as M. The Adams–Bashforth step adds only the forcing of the tracers. The option needs `superbee_advection`.

All of the advection has to leave the Adams–Bashforth step, not the vertical part alone. Horizontal and vertical advection cancel in a flow along isopycnals. A step in which one of them is forward and the other is part of the Adams–Bashforth step is unstable in such a flow. For the same reason the vertical advection of momentum stays in the Adams–Bashforth step.

The velocities at the end of the time step are needed. With the velocities at the start of the time step the forward tracer step and the Adams–Bashforth step of momentum amplify internal waves.

A step in one direction changes a uniform tracer, since the flow in that direction alone is not free of divergence. Each step therefore carries the volume ϱ that the cell would have after the steps done so far (Easter 1993, Lin and Rood 1996). It starts with $\varrho = 1$ and changes in each step in x as

$$\varrho \leftarrow \varrho - \frac{\Delta t}{\cos\phi^t \Delta x^t} \left(u_i - u_{i-1}\right)$$

and correspondingly in y and z. The fluxes are formed with the tracer value $T = q/\varrho$ and change the tracer content q. The content is conserved to round-off, and a uniform tracer stays uniform.

The flux through the surface face is handled in the steps in z. It is zero, or $w\,T$ of the surface cell with `free_surface_tracer_flux`. Its mean over the M steps is kept in `vadv_fs_flux` for a configuration that books the tracer budget.

The option adds no new state to the restart. The tendencies `dtemp` and `dsalt` then contain the forcing only. The first step after switching the option on or off should therefore not use the old tendencies.

Time step

The order of the calls in a time step of the split-explicit scheme is

1. diffusivities of the TKE closure (`set_tke_diffusivities`)
2. momentum tendencies, Coriolis force and hydrostatic pressure (`momentum_tendency_expl`)
3. forcing of the configuration (`set_forcing`)
4. bottom friction

5. external mode with sub cycling (`solve_expl_surface`)
6. vertical friction
7. vertical velocity
8. tracer advection and biharmonic tracer mixing, or the advection outside the Adams–Bashforth step
9. Adams–Bashforth update of the tracers
10. isoneutral mixing
11. vertical mixing of the tracers
12. density
13. TKE step (`integrate_tke`)

The order of the calls in a time step of the rigid lid and of the implicit free surface is

1. diffusivities of the TKE closure
2. tracer advection and biharmonic tracer mixing
3. momentum tendencies and hydrostatic pressure (`momentum_tendency_impl`)
4. forcing of the configuration (`set_forcing`)
5. biharmonic and harmonic momentum friction
6. Adams–Bashforth update of the velocities
7. direct forcing of the configuration (`set_forcing_direct`)
8. surface pressure and its gradient (`solve_for_pressure`)
9. vertical friction
10. bottom friction
11. vertical velocity
12. Adams–Bashforth update of the tracers, or the advection outside the Adams–Bashforth step
13. isoneutral mixing
14. vertical mixing of the tracers
15. density

16. TKE step

In both schemes the steps are those of pe_simple, with the isoneutral mixing, the harmonic friction and the advection outside the Adams–Bashforth step added.

Model parameters and options

The parameters and options of pe_simple are also those of pe_realistic. The tables list only the differences.

Required parameters

Attribute	Unit	Meaning
`dxt`, `dyt`	m or °	widths of the cells in x and y, a number or an array without halo. Degrees with `coord_degree`. Replace `dx` and `dy` of pe_simple, which are still accepted
`dzt`	m	widths of the cells in z, a number or an array without halo, from the bottom to the surface. Replaces `dz` of pe_simple, which is still accepted
`eq_of_state`	—	equation of state, 0, 1, 5 (TEOS-10) or 100

The configuration must set the Coriolis parameter with `set_coriolis`. The field `coriolis_t` is zero otherwise.

Options with defaults

Attribute	Default	Meaning
`coord_degree`	off	longitude and latitude in degrees instead of Cartesian coordinates
`x_origin`, `y_origin`	0	position of the first interior U point
`radius`	6370 km	radius of the earth (m)
`set_zero_at_surface`	off	origin of z at the surface face instead of the bottom face
`Ah`	unset	harmonic momentum friction (m^2/s), active if present. Rigid lid and implicit free surface only
`hor_friction_cosPower`	0	exponent p of $(\cos\phi)^p$ in the harmonic friction
`biha_friction_cosPower`	0	exponent p of $(\cos\phi)^p$ in each pass of the biharmonic friction

Attribute	Default	Meaning
`enable_neutral_diffusion`	off	isoneutral mixing
`enable_skew_diffusion`	off	skew flux of Gent and McWilliams
`K_iso_0`	0	isoneutral diffusivity (m^2/s), initial value of `K_iso`
`K_gm_0`	0	skew diffusivity (m^2/s), initial value of `K_gm`
`K_iso_steep`	0	lower limit of the tapered horizontal diffusivity (m^2/s)
`iso_slopec`	10^{-3}	slope at which the taper is one half
`iso_dslope`	$8 \cdot 10^{-4}$	width of the taper
`K_iso`, `K_gm`	unset	three-dimensional diffusivities, set from `K_iso_0` and `K_gm_0` if not given
`vadv_sub_cycling`	0	number M of vertical steps of the tracer advection outside the Adams–Bashforth step. 0 keeps the advection in the Adams–Bashforth step

Snapshot diagnostics

The snapshot classes `snap_nc4` and `snap_raw` are those of pe_simple. They write the coordinates of the grid above. These are x^t, x^u, y^t, y^u, z^t and z^w, in degrees with `coord_degree`. z^w is written under the name `zu`.

Limits

- The grid must not include the poles. The zonal width $\cos\phi\,\Delta x$ goes to zero there, and $\tan\phi$ grows without bound.
- The rigid lid does not converge on a domain that is periodic in x and contains land.
- TEM friction, the EKE model and IDEMIX of pyOM2 are not part of pe_realistic.
- With varying Δx^t in a periodic direction pe_realistic and pyOM2 build different x^t and Δx^u in the halo.
- Δz^t must vary smoothly, see the grid above.

Documentation for the simple non-hydrostatic model nh_simple

nh_simple is a finite-difference sibling of the pseudo-spectral model ps3D: it integrates the same scaled non-hydrostatic Boussinesq equations on the same kind of domain (horizontally always periodic, vertically periodic or bounded by rigid lids), but discretises them on an Arakawa C grid in physical space and uses the Fourier transform only for the pressure inversion. The LES closures are ported from ps3D. Within pyOM3, nh_simple is one of the ocean components that can be coupled to the spectral surface wave model wim_simple; the hooks of the Lagrangian-mean formulation of that coupling are noted where they appear.

Continuous equations

The scaled non-hydrostatic Boussinesq equations (see the scaling section of the ps3D documentation for the derivation) are

$$\partial_t \boldsymbol{u} + f \underset{\neg}{\boldsymbol{u}} + \boldsymbol{\nabla} p = -Ro\,(\boldsymbol{u}\cdot\boldsymbol{\nabla}\boldsymbol{u} + w\partial_z \boldsymbol{u}) \quad , \quad \partial_t b + N^2 w = -Ro\,(\boldsymbol{u}\cdot\boldsymbol{\nabla} b + w\partial_z b) \tag{37}$$

$$\delta^2 \partial_t w + \partial_z p - b = -Ro\,\delta^2\,(\boldsymbol{u}\cdot\boldsymbol{\nabla} w + w\partial_z w) \tag{38}$$

with the diagnostic relation $\boldsymbol{\nabla}\cdot\boldsymbol{u}+\partial_z w = 0$. Vectors are horizontal, $\underset{\neg}{\boldsymbol{u}}$ denotes anticlockwise rotation by 90°, Ro is the Rossby number and $\delta = H/L$ the aspect ratio; f and the background stability frequency N are constants (f-plane, uniform stratification). The buoyancy b is the deviation from the background stratification $N^2 z/Ro$; the term $N^2 w$ is the advection of this background. Kinetic and potential energy are $\boldsymbol{u}^2/2 + \delta^2 w^2/2$ and $b^2/2N^2$. Setting $Ro = 1$ and $\delta^2 = 1$ recovers the plain dimensional equations, which is the configuration required by the LES closures below.

Grid and spatial discretisation

All fields live on an Arakawa C grid with uniform spacings $\Delta x, \Delta y, \Delta z$: b and p at cell centres, u, v, w on the respective cell faces. The momentum advection is the second-order energy-conserving flux form, e.g. for u

$$(\partial_t u)_{adv} = -Ro\left(\delta_x^-\left(\overline{u}^{x+}\overline{u}^{x+}\right) + \delta_y^-\left(\overline{u}^{y+}\overline{v}^{x+}\right) + \delta_z^-\left(\overline{u}^{z+}\overline{w}^{x+}\right)\right) \tag{39}$$

in the finite-difference and averaging operator notation of the ps3D documentation ($\delta^{\pm}$ forward/backward differences, $\overline{\cdot}^{\pm}$ two-point averages), and analogously for v and w (the w tendency carries the factor δ^2). The Coriolis force uses the standard four-point average of the C grid.

For the buoyancy advection three flux-form schemes are available, selected by attribute: the second-order centred scheme (default, energy conserving), the second-order scheme with Superbee flux limiter (`buoyancy_superbee_advection`, from MITgcm; monotone, and its implicit scalar mixing can replace the explicit SGS buoyancy diffusion of the LES

closures, see below), and the third-order upwind scheme (`buoyancy_upwind3_advection`, from MITgcm). If both switches are set, the Superbee scheme is used. Momentum advection is always the energy-conserving second-order form.

Pressure solver

The velocity is first advanced with all terms except the pressure gradient to an intermediate value $\boldsymbol{u}^*, w^*$; the pressure then enforces the divergence constraint,

$$\left(\delta_x^- \delta_x^+ + \delta_y^- \delta_y^+ + \delta_z^- \delta_z^+ / \delta^2\right) p = \frac{1}{\Delta t} \left(\delta_x^- u^* + \delta_y^- v^* + \delta_z^- w^*\right) \tag{40}$$

followed by the projection $\boldsymbol{u} = \boldsymbol{u}^* - \Delta t \, \boldsymbol{\nabla}^+ p$ and $w = w^* - (\Delta t / \delta^2) \, \delta_z^+ p$. As shown in the ps3D documentation, the Fourier transform of the discrete one-dimensional operator $\delta^- \delta^+$ is $-\hat{k}^+ \hat{k}^- = -2\,(1 - \cos k\Delta)\,/\Delta^2$, which approaches $-k^2$ for $k\Delta \to 0$; the discrete Poisson equation is therefore diagonal in spectral space with the eigenvalue $\hat{k}^2 = \hat{k}_x^+ \hat{k}_x^- + \hat{k}_y^+ \hat{k}_y^- + \hat{k}_z^+ \hat{k}_z^- / \delta^2$, so the discrete divergence of the projected velocity vanishes to round-off.

Two cases are implemented, selected by `periodic_in_z`. For the triple-periodic domain a 3D FFT (parallel, through the p3dfft library) inverts the full eigenvalue; the mean mode $\hat{k}^2 = 0$ is set to zero. For rigid lids at top and bottom, a 2D FFT in x, y is combined with a tridiagonal solve in z (Thomas algorithm) with Neumann boundary conditions at the lids; the horizontal-mean column ($\hat{k}_x = \hat{k}_y = 0$), for which the tridiagonal system is singular, is integrated directly by the second-order recurrence $p_{k+1} = 2p_k - p_{k-1} + \delta^2 \Delta z^2 f_k$ with $p_0 = 0$.

Boundary conditions

The domain is always periodic in x and y (halo wrap or MPI exchange). In z it is either periodic (`periodic_in_z` true) or bounded by rigid free-slip lids: all vertical fluxes — advective, frictional, diffusive and the LES fluxes — are set to zero on the top and bottom faces, and w vanishes there.

Friction and mixing

Harmonic friction with constant coefficients acts on the momentum when the attributes are present: with both `Ah` and `Av` the full operator $A_h \boldsymbol{\nabla}_h^2 + A_v \partial_{zz}$, with only one of them the corresponding horizontal-only or vertical-only part. Harmonic buoyancy mixing requires both `Kh` and `Kv`. Biharmonic friction and mixing ($-A_{hbi} \boldsymbol{\nabla}_h^4$ etc.) are activated by the pairs `Ahbi`, `Avbi` for momentum and `Khbi`, `Kvbi` for buoyancy. With only `Ahbi` or only `Khbi` the horizontal biharmonic operator $-A_{hbi} \boldsymbol{\nabla}_h^2 \boldsymbol{\nabla}_h^2$ acts alone. All these operators are applied with a forward Euler step outside the Adams–Bashforth interpolation of the adiabatic tendencies, so their coefficients are subject to the usual explicit diffusive stability limits.

Time stepping

The adiabatic tendencies (advection, Coriolis, buoyancy force) are interpolated with the multi-step Adams–Bashforth scheme,

$$(c_A, c_B, c_C) = (1, 0, 0) \;\rightarrow\; \left(\frac{3}{2}, -\frac{1}{2}, 0\right) \;\rightarrow\; \left(\frac{23}{12}, -\frac{16}{12}, \frac{5}{12}\right) \tag{41}$$

i.e. a forward Euler start, one AB2 step and third-order Adams–Bashforth from the third step on, with the tendencies held in a rotating three-level buffer. Friction, mixing and the LES closures update the fields directly with forward Euler before the Adams–Bashforth increment; the pressure projection concludes the step. A user-defined forcing can be added to the tendencies at the current time level through the optional method `set_forcing(tau)` (it then enters the Adams–Bashforth interpolation), or applied as a direct increment through `set_forcing_direct()` after the Adams–Bashforth update and before the pressure solve. The latter bypasses the AB weights and is the hook for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation of the wave coupling, which must be a plain forward increment and must precede the projection so that it acts on its divergence.

LES closures

The LES closures of ps3D (`les_closures.f90`) are ported as the mixin class `les` (module `les.py`): the Smagorinsky closure, the filtered-structure-function closure of Ducros et al. (1996), and a Deardorff-type prognostic TKE closure, plus an optional Peclet bound on the viscosities. The closures are formulated dimensionally and refuse to run unless $Ro = 1$ and $\delta^2 = 1$. One call of `les_closure()` per time step (after the harmonic friction, before the Adams–Bashforth increment, e.g. hooked through `set_forcing`) evaluates the enabled closures and applies the anisotropic viscous and diffusive flux divergences to u, v, w and b with a forward Euler step. The closures do not combine. Each enabled closure overwrites all viscosities and diffusivities. The closures are evaluated in the order Ducros, Smagorinsky, TKE. Only the last enabled closure is therefore effective. The strain-rate tensor is computed on its natural C-grid positions only for the Smagorinsky closure. The Ducros closure uses the filtered velocities instead. The TKE closure takes its shear production from the discrete mixing fluxes.

Smagorinsky (`enable_smagorinsky`): $\nu = (c_s\Delta)^2\sqrt{2S_{ij}S_{ij}}$ with $\Delta = (\Delta x \Delta y \Delta z)^{1/3}$ and $c_s =$ `smag_coeff`; the diffusivity is ν/Pr with the turbulent Prandtl number `prandtl`. The viscosity is evaluated separately at the four staggered positions where its fluxes are needed (cell centre, horizontal corner, above u, above v).

Ducros et al. (1996) (`enable_ducros_96`): the viscosity is built from the second-order structure function of the high-pass filtered velocity (two iterated horizontal Laplace filters, three with `enable_three_level_filter`). The Smagorinsky and Ducros viscosities are bounded below by `min_smag`. The TKE closure does not use this floor.

Prognostic TKE (`enable_tke_closure`): a Deardorff-type closure with prognostic subgrid TKE on the w grid, a stratification-limited length scale $\ell = \min(\Delta,\, 0.76\sqrt{e/N^2})$ where

$N^2 > 0$ and $\ell = \Delta$ where $N^2 \le 0$, dissipation $(0.19 + 0.51\ell/\Delta)\, e^{3/2}/\ell$, TKE diffusion with the diffusivity $c_{eke}\, \ell\sqrt{e}$ with $c_{eke} =$ `c_eke`, and shear and buoyancy production diagnosed from the actual discrete mixing fluxes of the previous step. Viscosity and diffusivity are $\nu = c_m \ell\sqrt{e}$ and $\kappa = (1 + 2\ell/\Delta)\, \nu$ with $c_m =$ `c_m`; N^2 always includes the background N_0^2. The TKE diffusivity is capped by $\min(\Delta x, \Delta y, \Delta z)^2/(4\Delta t)$ for the stability of the explicit step.

Two further sources can be added to the TKE equation. The first is an optional external volume source in $\mathrm{m}^2/\mathrm{s}^3$. The configuration sets it as the attribute `tke_source` (e.g. for wave breaking). The second is the Stokes production of subgrid TKE, selected by `les_stokes_prod` (default `None`, no Stokes production). It needs the Stokes velocity $\boldsymbol{u}_S$ (attributes `u_S`, `v_S`) and the vertical subgrid momentum flux of the mixing, $\langle \boldsymbol{u}' w' \rangle = -\nu\, \partial_z \boldsymbol{u}$, which vanishes on the lids. The option `'A'` takes $-\langle \boldsymbol{u}' w' \rangle \cdot \partial_z \boldsymbol{u}_S$ after McWilliams et al. (1997). The option `'B'` takes $\boldsymbol{u}_S \cdot \partial_z \langle \boldsymbol{u}' w' \rangle$ after Leibovich (1980). Summation by parts shows that both forms have the same column integral. Like the other production terms, the Stokes production is diagnosed in one step and used in the next.

Stability caps. All closures cap their viscosities and diffusivities against the explicit diffusive limit of the forward Euler update. The caps are direction-aware: each staggered viscosity is capped with the most restrictive grid spacing among the directions its fluxes act in (e.g. the cell-centre viscosity by $\min(\Delta x, \Delta y, \Delta z)^2/(4\Delta t)$). This deviates deliberately from the original ps3D caps, which use geometric means of the spacings and are valid only for near-isotropic grids: for $\Delta x \gg \Delta z$ they admit vertically unstable viscosities. The buoyancy diffusivities are capped by $\kappa_h \le \min(\Delta x, \Delta y)^2/(8\Delta t)$ in the horizontal and by $\kappa_v \le \Delta z^2/(4\Delta t)$ in the vertical. The viscosity caps bound each direction separately by $\nu\Delta t/\Delta^2 \le 1/4$, with Δ the spacing in that direction. The forward Euler step of the three-dimensional diffusion operator is stable only for $\nu\Delta t\, (\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2}) \le 1/2$. With all three directions at their caps the sum is $3/4$. The viscosity caps therefore do not guarantee stability in this case. The same holds for the TKE diffusivity. The diffusivity caps of the buoyancy add up to $1/2$ and satisfy the limit.

Boundary treatment. With rigid lids the vertical shear seen by the Smagorinsky closure is zeroed on the lids (`les_freeslip_strain`, default on), consistent with the free-slip zero-flux condition of the mixing; otherwise the zero halo above the lid produces a spurious shear that inflates the viscosity in the top and bottom cells. The Ducros and TKE closures do not use the strain-rate tensor. The switch has no effect on them.

Scalar mixing. The explicit SGS buoyancy diffusion can be switched off (`les_mix_b`) when a flux-limited buoyancy advection scheme (Superbee) provides the scalar SGS mixing implicitly. The buoyancy production of the TKE closure is the upward subgrid buoyancy flux $\langle w' b' \rangle$ on the w grid. With the explicit diffusion it is $\langle w' b' \rangle = -\kappa\, \partial_z b$, with κ the vertical diffusivity. With `les_mix_b` off it is the implicit vertical subgrid flux of the Superbee limiter. The vertical buoyancy flux of the Superbee scheme is the centred flux $w\overline{b}^{z+}$ plus a correction from the limiter. This correction is the implicit subgrid flux. It vanishes on the lids. The buoyancy production is zero only when `les_mix_b` is off and the Superbee scheme is not selected, for instance with the third-order upwind scheme.

Peclet bound (`enable_peclet_bound`): where the grid Peclet number of the resolved flow exceeds `Peclet_limit`, the cell-centre and horizontal-corner viscosities are raised proportionally. The bound is not purely horizontal. The cell-centre viscosity is also the vertical viscosity of w. The bound is applied after the stability caps of the closures and is not capped itself. The raised viscosities can therefore exceed the explicit stability limit. The diffusivities are not changed. The bound uses the prognostic (advecting) velocity, i.e. $\boldsymbol{u}_L$ in the Lagrangian-mean formulation.

Coupled wave–ocean setups. When the prognostic velocity is the Lagrangian mean $\boldsymbol{u}_L$, the attribute `les_mix_eulerian` makes the strains and mixing fluxes act on the Eulerian velocity $\boldsymbol{u} = \boldsymbol{u}_L - \boldsymbol{u}_S$ (attributes `u_S`, `v_S` and optionally `w_S` supplied by the coupler), with the tendency added to the prognostic $\boldsymbol{u}_L$ — the down-gradient closure parameterises Eulerian fluctuations, so mixing $\boldsymbol{u}_L$ would spuriously diffuse the Stokes shear. This is the LES counterpart of the `mix_eulerian` option of the TKE closure in pe_simple.

Domain decomposition

The parallelisation follows the pencil decomposition of the p3dfft library used by the pressure solver: the x direction is kept local on every processor, and the constructor arguments `n_pes_x` and `n_pes_y` define the processor grid over the y and z directions (internally the blocks are addressed as y/z blocks). Model arrays are Fortran-ordered (N_x, N_y, N_z) with halo points on all sides; the pressure solver requires this ordering. Halo exchanges wrap periodically in x and y; in z they wrap only for the triple-periodic case.

Snapshot diagnostics

Two snapshot classes live in the module `snapshot.py`; either replaces the model class as the base of the configuration and overrides `setup` and `diagnose`. Both read the attribute `snapint` (snapshot interval in seconds, required), take the setup arguments `snap_file_name` and `show_each_time_step` (default on: print the model time every step; off: only at output steps), and write at $n = 0$ and whenever $n \bmod (\text{snapint}/\Delta t) = 0$. Both record the variables u, v, w, b, p on their staggered coordinate axes (`xt`/`yt`/`zt` for cell centres and `xu`/`yu`/`zu` for the faces, shifted by half a spacing); the vertical axis is negative downwards with the surface at $z = 0$ and the cell centres at $z = -\Delta z/2, \ldots$. Halo points are stripped before writing.

snap_nc4 writes a single netCDF file (default `'snap.cdf'`). At start-up it detects whether the netCDF4/HDF5 build supports parallel I/O (`__has_parallel4_support__` or pnetcdf) and reports the result. In parallel mode the file is created and opened collectively by all PEs, and every PE writes its subdomain into the global arrays in a single collective pass per snapshot. Without parallel support the original serialised path is used: PE0 creates the file and extends the time dimension, then each PE in turn opens the file and writes its block, synchronised with barriers — correct on any build, but increasingly slow with many PEs. The internal Fortran ordering is transposed on the fly, so the file layout (time, z, y, x) is the same in both modes.

snap_zarr_gather writes a single zarr store (default `'snap.zarr'`) with field arrays of shape (time, n_x, n_y, n_z), always x-first regardless of the internal ordering, one time slice per chunk. The design goal is a Lustre-friendly file count: with the default `z_chunks` = 1 each snapshot adds one chunk file per field — of the order of six files per snapshot regardless of the number of PEs — instead of the one-file-per-PE-per-field fragmentation of a per-block chunking, which can overwhelm the metadata server of a parallel file system at large PE counts. Every PE participates in the collective gather of each field (`gather_array_mpi`), but only PE0 touches the store, writing a full horizontal slab of $\lceil n_z/$`z_chunks`$\rceil$ levels per chunk; increase `z_chunks` only if a single field slab is too large to hold and write in one piece. Fields are gathered and written one at a time to bound the PE0 memory to one global field. The time axis and the field arrays are resized (appended) per snapshot, so the store remains valid after an interrupted run; an existing store of the same name is removed at start-up. The on-disk format is pinned to zarr v2 even when a zarr-python v3 library writes it, so the output stays readable by v2 environments; the cosmetic consolidated-metadata warning of v3 is silenced. Compared to the netCDF path this trades the gather (one global field on PE0) for independence from parallel-I/O library support.

Model parameters and options

All parameters and options are attributes of the model instance, set either through the parameter dictionary passed at construction or by assigning the attribute before the time loop; options with a default may be left unset.

Required parameters (no default)

Attribute	Unit	Meaning
`nx`, `ny`, `nz`	—	global grid size (cells, without halo points)
`dx`, `dy`, `dz`	m	grid spacing (uniform)
`dt`	s	time step
`f0`	1/s	Coriolis parameter (f-plane)
`N0`	1/s	background stability frequency
`Ro`	—	Rossby number of the scaled equations (1 for the dimensional form; the LES closures require 1)
`dsqr`	—	squared aspect ratio δ^2 (1 for the fully non-hydrostatic dimensional form; the LES closures require 1)
`periodic_in_z`	—	triple-periodic domain if true; rigid free-slip lids at top and bottom if false. x and y are always periodic

The constructor additionally takes `n_pes_x`, `n_pes_y` (processor grid over the y and z pencils, default 1×1; their product must equal the communicator size), `halo_points` (default 1) and `comm_in` (default: MPI world communicator).

Forcing hooks and coupling fields

Attribute / method	Meaning
`set_forcing(tau)`	optional user method; adds forcing to the tendencies at time level `tau` (enters the Adams–Bashforth interpolation). Also the natural hook to call `les_closure()`
`set_forcing_direct()`	optional user method; direct increment after the Adams–Bashforth update and before the pressure solve, bypassing the AB weights. Used for the Stokes tendency $\Delta \boldsymbol{u}_S$ of the Lagrangian-mean ($\boldsymbol{u}_L$) formulation
`u_S`, `v_S`, `w_S`	Stokes velocity from the wave model (w_S optional); required by `les_mix_eulerian` and by `les_stokes_prod`
`tke_source`	optional external volume source of subgrid TKE ($\mathrm{m}^2/\mathrm{s}^3$), set by the configuration and added to the forcing of the TKE equation (TKE closure only)

Options with defaults

Attribute	Default	Meaning
`buoyancy_superbee_advection`	off	buoyancy advection with the Superbee flux limiter instead of the centred second-order scheme
`buoyancy_upwind3_advection`	off	third-order upwind buoyancy advection
`Ah`, `Av`	unset	harmonic viscosities (m^2/s); either one alone activates the horizontal-only / vertical-only operator
`Kh`, `Kv`	unset	harmonic buoyancy diffusivities (m^2/s); both must be present
`Ahbi`, `Avbi`	unset	biharmonic viscosities (m^4/s); `Ahbi` alone activates the horizontal-only operator
`Khbi`, `Kvbi`	unset	biharmonic buoyancy diffusivities (m^4/s); `Khbi` alone activates the horizontal-only operator

LES options (read at `init_les`)

Attribute	Default	Meaning
`enable_smagorinsky`	off	Smagorinsky closure
`enable_ducros_96`	off	Ducros et al. (1996) structure-function closure
`enable_tke_closure`	off	Deardorff-type prognostic TKE closure
`enable_three_level_filter`	off	three instead of two iterated Laplace filters (Ducros closure)
`enable_peclet_bound`	off	raise the cell-centre and horizontal-corner viscosities where the grid Peclet number exceeds `Peclet_limit` (applied after the stability caps, not capped)
`les_mix_eulerian`	off	strains and mixing act on the Eulerian velocity $\boldsymbol{u}_L - \boldsymbol{u}_S$ ($\boldsymbol{u}_L$ formulation)
`les_freeslip_strain`	on	zero the vertical shear seen by the Smagorinsky closure at the rigid lids (no effect on the other closures)
`les_mix_b`	on	explicit SGS buoyancy diffusion; switch off when a flux-limited advection scheme provides the scalar SGS mixing
`les_stokes_prod`	`None`	Stokes production of subgrid TKE (TKE closure only). `'A'` uses $-\langle \boldsymbol{u}' w' \rangle \cdot \partial_z \boldsymbol{u}_S$ (McWilliams et al. 1997), `'B'` uses $\boldsymbol{u}_S \cdot \partial_z \langle \boldsymbol{u}' w' \rangle$ (Leibovich 1980). Requires `u_S`, `v_S`
`smag_coeff`	0.148	Smagorinsky coefficient
`prandtl`	0.47	turbulent Prandtl number
`min_smag`	0	viscosity floor (m^2/s) of the Smagorinsky and Ducros closures, not used by the TKE closure
`Peclet_limit`	50	grid Peclet number bound
`c_m`	0.1	TKE-closure viscosity constant
`c_eke`	2	TKE diffusion constant
`tke_min`	10^{-8}	TKE floor (m^2/s^2)
`les_epsln`	10^{-20}	numerical floor of the closure

Snapshot diagnostics

Both snapshot classes (module `snapshot.py`) read the attribute `snapint` (snapshot interval in seconds, required) and take the setup arguments `snap_file_name` (defaults `'snap.cdf'` / `'snap.zarr'`) and `show_each_time_step` (default on); `snap_zarr_gather` additionally

takes `z_chunks` (default 1: one chunk file per field and snapshot; larger values split the vertical into $\lceil n_z/$`z_chunks`$\rceil$-level chunks).

The realistic non-hydrostatic model nh_realistic

nh_realistic is pe_realistic with a prognostic vertical velocity and a non-hydrostatic pressure. This text describes only the differences to pe_realistic. Everything that is not mentioned here is as described in the documentation of pe_realistic and pe_simple.

The differences are the following. The vertical velocity w has its own momentum equation. A three-dimensional pressure p_{nh} makes the velocity free of divergence in every cell. The horizontal coordinates must be Cartesian.

The package contains only what is new. The grid, the masks, the tracers, the equation of state, the vertical mixing, the TKE closure, the isoneutral mixing, the surface pressure and the split-explicit external mode are imported from pe_realistic. The file `momentum.py` holds the equation for w. The file `vmix.py` holds the vertical friction of w. The files `pressure.py` and `solver.py` hold the equation for p_{nh} and its solver. The file `model.py` holds the time step.

The formulation follows pyOM2 with `enable_hydrostatic` off and the non-hydrostatic model of Oceananigans. The solver for p_{nh} follows Oceananigans. The tests and their results are listed in the section on the nhr family of `pyOM3/test/README_test_suite.md`.

Grid

The grid is that of pe_realistic with Cartesian coordinates. The widths `dxt`, `dyt` and `dzt` are in metres and may vary in all three directions. The model stops with an error if `coord_degree` is set. On the sphere the equation for w would need the metric term $-(u^2+v^2)/a$ and the Coriolis terms with $\cos\phi$. nh_realistic has neither of them.

The metric factors `cost` and `cosu` of pe_realistic are therefore 1, and `tantr` is 0. The kernels imported from pe_realistic still contain them. The new kernels of nh_realistic do not. The horizontal area of a T cell is $A^t = \Delta x^t \Delta y^t$, and its volume is $V^t = A^t \Delta z^t$.

The vertical velocity w_k sits at the top face of the T cell k. This face is the centre of the W cell k. The W cell k reaches from the centre z^t_k to the centre z^t_{k+1}. Its height is Δz^w_k, and its volume is $A^t \Delta z^w_k$.

The mask `maskW` of pe_realistic is open at the surface face, since the free surface needs a vertical velocity there. nh_realistic adds the mask `maskWc`, which is `maskW` with the surface face closed. The equation for w, its friction and the equation for p_{nh} use `maskWc`. The surface face thus belongs to the free surface and not to the equation for w.

Pressure

All pressures are divided by ρ_0, as in pe_realistic. The pressure is the sum

$$p = p_h + p_s + p_{nh} \tag{42}$$

of the hydrostatic pressure p_h, the surface pressure p_s and the non-hydrostatic pressure p_{nh}. p_h and p_s are computed as in pe_realistic.

The momentum equations are

$$\partial_t \boldsymbol{u} + \ldots = -\boldsymbol{\nabla}_h \left(p_h + p_s + p_{nh}\right) \ , \quad \partial_t w + \boldsymbol{u} \cdot \boldsymbol{\nabla} w = -\partial_z p_{nh} + F_w \tag{43}$$

where $\boldsymbol{u} = (u, v)$ is the horizontal velocity, the dots stand for all other terms of pe_realistic, and F_w is the friction of w.

The equation for w carries neither the buoyancy nor p_h nor p_s. The full equation would be $\partial_t w + \boldsymbol{u} \cdot \boldsymbol{\nabla} w = -\partial_z p - g\rho/\rho_0 + F_w$. p_s does not depend on z. p_h is integrated downward from the surface with

$$p_h(k) = p_h(k+1) + \frac{g}{2\rho_0} \left(\rho(k+1) + \rho(k)\right) \Delta z_k^w \tag{44}$$

which is the integration of pe_realistic. Eq. (44) gives at the W point k the vertical gradient

$$\frac{p_h(k+1) - p_h(k)}{\Delta z_k^w} = -\frac{g}{2\rho_0} \left(\rho(k+1) + \rho(k)\right) \tag{45}$$

The right hand side is the buoyancy force averaged to the W point. The vertical gradient of p_h and the buoyancy force therefore cancel exactly, also on a stretched grid. Only p_{nh} is left in the equation for w.

With $p_{nh} = 0$ and w taken from continuity the model is pe_realistic. The attribute `enable_nonhydrostatic` (default on) selects the non-hydrostatic model. With it off the time step is that of pe_realistic to the last bit. This is tested.

Advection and friction of w

The advection of w is the energy-conserving second-order form of the momentum advection of pe_realistic. The transports through the faces of a T cell are

$$U_i = \Delta y^t \Delta z^t u_i \ , \quad V_j = \Delta x^t \Delta z^t v_j \ , \quad W_k = A^t w_k \tag{46}$$

with the masks of the faces. The transport through a face of the W cell k is the mean of the transports through the same face of the T cells k and $k+1$. The fluxes of w are

$$f_i^x = -\frac{1}{4} \left(U_{i,k} + U_{i,k+1}\right) \left(w_i + w_{i+1}\right) \ , \quad f_k^z = -\frac{1}{4} \left(W_k + W_{k+1}\right) \left(w_k + w_{k+1}\right) \tag{47}$$

and f^y correspondingly with V. f_k^z sits at the T point $k+1$. The advective tendency of w_k is the divergence of these fluxes divided by the volume $A^t \Delta z_k^w$ of the W cell.

The divergence of the transports through the W cell k is the mean of the divergences of the T cells k and $k+1$. It vanishes when the velocity is free of divergence. The advection then conserves w^2. On a uniform grid it is the advection of w of nh_simple.

The equation for w has no Coriolis term, as in pyOM2.

The horizontal friction of w uses the coefficients `Ahbi` and `Ah` of the horizontal friction of u and v. It uses the operator at the T points of pe_realistic, since w sits at the T points in the horizontal. For w this operator is exact. A flux exists only between two wet W cells.

The vertical friction of w has the flux at the T point k between the W cells $k-1$ and k,

$$F_k = A_k \frac{w_k - w_{k-1}}{\Delta z_k^t} \tag{48}$$

The tendency of w_k is $(F_{k+1} - F_k)/\Delta z_k^w$. The roles of Δz^t and Δz^w are thus swapped with respect to the vertical friction of u and v. A flux exists only between two wet W cells.

The coefficient A_k follows the vertical friction of u and v. It is the constant `A_v` with `vmix_scheme` explicit or implicit. With the TKE closure it is the viscosity of the closure averaged from the two W points to the T point. The step is a forward step for `vmix_scheme` explicit and an implicit step otherwise. The implicit step is the tridiagonal solve of pe_realistic on the W grid. The energy that the friction of w removes is not given to the TKE closure.

Projection

The velocities are first advanced with all terms except the gradient of p_{nh}. This gives u^*, v^* and w^*. The surface pressure p_s is already part of u^* and v^*. The vertical friction is also part of them.

The vertical velocity at the surface face is taken from continuity as in pe_realistic,

$$w_{surf} = -\frac{1}{A^t} \sum_k \left(U_i - U_{i-1} + V_j - V_{j-1} \right) \tag{49}$$

with u^* and v^* in the transports and the sum over the wet cells of the column. With the free surface w_{surf} is the change of the surface elevation in time. With the rigid lid it is zero to the tolerance of the surface pressure solver.

The divergence of the T cell is

$$D = \frac{1}{V^t} \left(U_i - U_{i-1} + V_j - V_{j-1} + W_k - W_{k-1} \right) \tag{50}$$

The velocity is corrected by

$$u_i = u_i^* - \Delta t \frac{p_{i+1} - p_i}{\Delta x_i^u} \ , \quad v_j = v_j^* - \Delta t \frac{p_{j+1} - p_j}{\Delta y_j^u} \ , \quad w_k = w_k^* - \Delta t \frac{p_{k+1} - p_k}{\Delta z_k^w} \tag{51}$$

with $p = p_{nh}$, on the wet faces and the interior faces in z. The surface face is closed in Eq. (51). w_{surf} therefore stays as it is.

The pressure p_{nh} makes D of the corrected velocity vanish in every wet cell. Inserting Eq. (51) into $D = 0$ gives

$$\begin{aligned} a_i^x \left(p_{i+1} - p_i \right) - a_{i-1}^x \left(p_i - p_{i-1} \right) + a_j^y \left(p_{j+1} - p_j \right) - a_{j-1}^y \left(p_j - p_{j-1} \right) & \\ + a_k^z \left(p_{k+1} - p_k \right) - a_{k-1}^z \left(p_k - p_{k-1} \right) &= \frac{V^t D^*}{\Delta t} \end{aligned} \tag{52}$$

with D^* the divergence of u^*, v^*, w^* and w_{surf}. The coefficients are

$$a_i^x = \frac{\Delta y^t \Delta z^t}{\Delta x_i^u} \ , \quad a_j^y = \frac{\Delta x^t \Delta z^t}{\Delta y_j^u} \ , \quad a_k^z = \frac{A^t}{\Delta z_k^w} \tag{53}$$

on the wet faces and zero on the closed faces. Eq. (52) is the volume times the divergence of the gradient of p_{nh}.

The coefficient of a face is the same in the equations of the two cells next to it. The matrix of Eq. (52) is therefore symmetric, also on a stretched grid with topography. This is tested. Without the volume factor the matrix would not be symmetric on a stretched grid.

All faces at the boundary of the wet region are closed in Eq. (52). A solution exists only if the right hand side summed over the wet cells vanishes. Summed over a column, the transports through the inner faces cancel. What is left is the sum of the horizontal transports of the column and $A^t w_{surf}$. With Eq. (49) this sum is zero. The right hand side thus sums to zero in every column, and Eq. (52) has a solution.

Eq. (52) fixes p_{nh} only up to a constant. After every iteration the mean of p_{nh} over the wet cells is set to zero, as in Oceananigans. The same is done with the residual.

Solver

Eq. (52) is solved with the method of conjugate gradients with a preconditioner. The solver is written for JAX and MPI in the same way as the solver for p_s of pe_simple. The first guess is p_{nh} of the previous time step.

The preconditioner solves the vertical part of Eq. (52) exactly in every column. It drops the horizontal coefficients a^x and a^y. The vertical part is the tridiagonal system

$$-a_{k-1}^z z_{k-1} + (1+\epsilon)\left(a_k^z + a_{k-1}^z\right) z_k - a_k^z z_{k+1} = r_k \tag{54}$$

for the preconditioned residual z with the residual r. It is solved with the Thomas algorithm. The factor $1+\epsilon$ with $\epsilon = 0.01$ makes the system of each column regular, as in Oceananigans. In a dry cell and in a wet cell without a wet cell above or below, the preconditioner is the identity. The preconditioner is that of Marshall et al. (1997).

The preconditioner suits grids with $\Delta z \ll \Delta x$. There the vertical coefficients a^z are much larger than the horizontal ones. In the test configuration they are larger by a factor of $3 \cdot 10^4$ to $2 \cdot 10^7$. The preconditioner reduces the number of iterations there from 1000 to 3500 to 50 to 150 per time step. For $\Delta z \approx \Delta x$ it helps less.

The residual r is the right hand side minus the left hand side of Eq. (52). The divergence of the velocity corrected with the current p_{nh} is $\Delta t\, r/V^t$. The iteration stops when $\Delta t \max|r|/V^t$ is smaller than `epsilon_nh` (default $10^{-12}\,\mathrm{s}^{-1}$). `epsilon_nh` is thus the largest divergence that the velocity keeps. The iteration also stops after `max_iterations_nh` iterations (default `max_iterations`).

The stopping condition is met by the vertical structure of p_{nh}. The horizontal structure of p_{nh} is less well determined when $\Delta z \ll \Delta x$. It still changes with the round-off of the

sums in the solver. Runs with different domain decompositions therefore differ by about 10^{-8} relative with the default `epsilon_nh`. With `epsilon_nh` $= 10^{-16}$ they differ by about 10^{-10}.

Tracers

The tracers are advected with u, v and w of the end of the previous time step, as in pe_realistic. In pe_realistic w follows from continuity and the velocity is free of divergence to round-off. In nh_realistic the velocity is free of divergence to `epsilon_nh`. A uniform tracer therefore stays uniform only to `epsilon_nh` Δt per time step. The content of a tracer is still conserved to round-off, since the advection is in flux form.

Time step

The order of the calls in a time step of the split-explicit scheme is

1. diffusivities of the TKE closure (`set_tke_diffusivities`)
2. momentum tendencies, Coriolis force and hydrostatic pressure (`momentum_tendency_expl`)
3. advective tendency of w (`w_tendency`)
4. forcing of the configuration (`set_forcing`)
5. horizontal friction of w and Adams–Bashforth update of w
6. bottom friction
7. external mode with sub cycling (`solve_expl_surface`)
8. vertical friction of u, v and of w
9. surface vertical velocity and the solve for p_{nh} (`nh_projection`)
10. tracer advection and biharmonic tracer mixing, or the advection outside the Adams–Bashforth step
11. Adams–Bashforth update of the tracers
12. isoneutral mixing
13. vertical mixing of the tracers
14. density
15. TKE step (`integrate_tke`)

The order of the calls in a time step of the rigid lid and of the implicit free surface is

1. diffusivities of the TKE closure
2. tracer advection and biharmonic tracer mixing
3. momentum tendencies and hydrostatic pressure (`momentum_tendency_impl`)
4. advective tendency of w (`w_tendency`)
5. forcing of the configuration (`set_forcing`)
6. biharmonic and harmonic friction of u, v and of w
7. Adams–Bashforth update of u, v and of w
8. direct forcing of the configuration (`set_forcing_direct`)
9. surface pressure and its gradient (`solve_for_pressure`)
10. vertical friction of u, v and of w
11. bottom friction
12. surface vertical velocity and the solve for p_{nh} (`nh_projection`)
13. Adams–Bashforth update of the tracers, or the advection outside the Adams–Bashforth step
14. isoneutral mixing
15. vertical mixing of the tracers
16. density
17. TKE step

In both schemes the steps are those of pe_realistic. The steps for w are added. The solve for p_{nh} replaces the vertical velocity from continuity.

The forcing of the configuration may add to the tendency `dw` at the level `tau`, as it does for `du` and `dv`. The tendency then enters the Adams–Bashforth step.

The solve for p_{nh} comes after the vertical friction and the bottom friction. The velocity at the end of the time step is therefore free of divergence. The surface pressure is solved before. It removes the depth-averaged divergence, which is the mode with the slowest convergence in Eq. (52).

The restart needs the fields of pe_realistic and in addition `dw` and `p_nh`. Without `p_nh` the solver starts from another first guess. The run is then no longer identical to the last bit.

Model parameters and options

The parameters and options of pe_realistic are also those of nh_realistic. The tables list only the differences.

Required parameters

Attribute	Unit	Meaning
`dxt`, `dyt`	m	widths of the cells in x and y, as in pe_realistic but in metres only

`coord_degree` must be off.

Options with defaults

Attribute	Default	Meaning
`enable_nonhydrostatic`	on	prognostic w and the solve for p_{nh}. Off gives pe_realistic
`epsilon_nh`	10^{-12}	largest divergence (s^{-1}) that the solve for p_{nh} leaves
`max_iterations_nh`	`max_iterations`	largest number of iterations of the solve for p_{nh}
`Ah`, `Ahbi`	unset	as in pe_realistic, also acting on w
`A_v`	0	as in pe_realistic, also acting on w

The number of iterations of the last solve for p_{nh} is `congr_itts_nh`. The divergence it leaves is `estimated_error_nh`.

Snapshot diagnostics

The snapshot classes `snap_nc4` and `snap_raw` are those of pe_realistic. `snap_nc4` also writes p_{nh} under the name `p_nh`. w is written on the W points as in pe_realistic. It is now the prognostic w.

Limits

- The horizontal coordinates must be Cartesian, see the grid above.
- The equation for w has no Coriolis term.
- The energy that the friction of w removes is not given to the TKE closure.

- With the free surface w_{surf} does not enter the advection of w, since the surface face is closed in the equation for w. The advection of w then does not conserve w^2 exactly in the uppermost W cell.
- The preconditioner suits $\Delta z \ll \Delta x$. For $\Delta z \approx \Delta x$ the number of iterations grows. A preconditioner with the FFT of a uniform grid, as in Oceananigans, is not available.
- Runs with different domain decompositions are not identical to the last bit, see the solver above.
- The LES closures of nh_simple are not part of nh_realistic.

The spectral wave model wim_simple (Wave Interaction Model, WIM)

wim_simple is a spectral surface wave model, coupled to an ocean model in an energetically and dynamically consistent way. Basis is the wave action spectral density equation for deep water surface gravity waves

$$\partial_t A + \dot{\boldsymbol{x}} \cdot \boldsymbol{\nabla}_{\boldsymbol{x}} A + \dot{\boldsymbol{k}} \cdot \boldsymbol{\nabla}_{\boldsymbol{k}} A \;\; = \;\; S = S_{in} + S_{out} + S_{nl} \tag{55}$$

with wave action density $A(\boldsymbol{k}, \boldsymbol{x}, t)$, the horizontal wavenumber vector $\boldsymbol{k} = (k_x, k_y)$, the dispersion relation $\Omega = \sigma + \boldsymbol{U} \cdot \boldsymbol{k}$ with intrinsic frequency $\sigma(\boldsymbol{k}) = \sqrt{g|\boldsymbol{k}|}$ and Doppler shift velocity $\boldsymbol{U}$, transport velocities $\dot{\boldsymbol{x}} = \boldsymbol{\nabla}_{\boldsymbol{k}}\Omega$ and $\dot{\boldsymbol{k}} = -\boldsymbol{\nabla}_{\boldsymbol{x}}\Omega$, action input by wave growth S_{in}, action dissipation by wave breaking S_{out}, and non-linear action transfer S_{nl} and operators $\boldsymbol{\nabla}_{\boldsymbol{x}} = (\partial_x, \partial_y)$ and $\boldsymbol{\nabla}_{\boldsymbol{k}} = (\partial_{k_x}, \partial_{k_y})$. Only deep water waves are considered, with $\boldsymbol{\nabla}_{\boldsymbol{x}}\sigma = 0$ and $\partial_t \sigma = 0$, but the concept extends readily to waves at finite depth. Since the scattering integral S_{nl} conserves energy and momentum of the waves, it holds that $\int \sigma S_{nl} d\boldsymbol{k} = 0$ and $\int \boldsymbol{k} S_{nl} d\boldsymbol{k} = 0$.

The surface wave model predicts the Stokes velocity $\boldsymbol{u}_S$ and the wave stress $\boldsymbol{\tau}_o$ to force an ocean model. Considering the Lagrangian ocean velocity $\boldsymbol{u}_L = \boldsymbol{u} + \boldsymbol{u}_S$ and its kinetic energy, the consistent energy cycle becomes

$$\partial_t \int_{-\infty}^{0} |\boldsymbol{u}_L|^2/2 dz + \boldsymbol{\nabla}_{\boldsymbol{x}} \cdot \int_{-\infty}^{0} \boldsymbol{u}_h^L \pi dz \;\; = \;\; C \tag{56}$$

$$\partial_t \int E d\boldsymbol{k} + \boldsymbol{\nabla}_{\boldsymbol{x}} \cdot \int \left(\dot{\boldsymbol{x}} E + \dot{\boldsymbol{x}} A (\boldsymbol{U} \cdot \boldsymbol{k})\right) d\boldsymbol{k} \;\; = \;\; \int (\Omega S_{in} + \boldsymbol{U} \cdot \boldsymbol{k}\, S_{nl}) d\boldsymbol{k} + \int \sigma S_{out} d\boldsymbol{k} - C$$

with the wave energy spectral density $E = \sigma A$ and the exchange term

$$C \;\; = \;\; \int_{-\infty}^{0} \boldsymbol{u}_h^L \cdot \partial_t \boldsymbol{u}_S dz + \int_{-\infty}^{0} \boldsymbol{u}_h^L \cdot \partial_z \boldsymbol{\tau} dz = \int \boldsymbol{U} \cdot \boldsymbol{k} (\partial_t A - S_{out}) d\boldsymbol{k} \tag{57}$$

and the setting, following Vanneste and Young for the Doppler shift velocity $\boldsymbol{U}$,

$$\boldsymbol{U}(\boldsymbol{x}, \boldsymbol{k}, t) = \int_{-\infty}^{0} \boldsymbol{u}_h^L Q dz \;\; , \;\; \boldsymbol{u}_h^S(\boldsymbol{x}, z, t) = \int Q A \boldsymbol{k} d\boldsymbol{k} \;\; , \;\; Q(k, z) = 2k e^{2kz} \tag{58}$$

$$\partial_z \boldsymbol{\tau}(\boldsymbol{x}, z, t) = -\int Q \boldsymbol{k} S_{out} d\boldsymbol{k} \;\; , \;\; \boldsymbol{\tau}_o(\boldsymbol{x}, t) = \int_{-\infty}^{0} \partial_z \boldsymbol{\tau} dz = -\int \boldsymbol{k} S_{out} d\boldsymbol{k} \tag{59}$$

with $\int_{-\infty}^{0} Q\, dz = 1$ and the ocean momentum forcing $\boldsymbol{\tau}_o = -\int \boldsymbol{k} S_{out} d\boldsymbol{k}$, i.e. the stress on the ocean is the dissipation of wave momentum. Note that $\boldsymbol{U}$ contains both the Eulerian mean and the Stokes horizontal velocity components. Waves and ocean exchange energy by the effect of the Doppler shift and the ocean stress, both given by C; other source terms of the wave energy are the wave growth by the wind, $\int \Omega S_{in} d\boldsymbol{k}$, and the dissipation by

wave breaking, $\int \sigma S_{out} d\boldsymbol{k}$, which may be directed into a turbulent ocean energy reservoir (see below). The cycle closes exactly up to the term $\int \boldsymbol{U} \cdot \boldsymbol{k}\, S_{nl} d\boldsymbol{k}$, which represents the work done on the sheared mean flow by the vertical redistribution of Stokes momentum through the nonlinear transfer. Only the baroclinic part of $\boldsymbol{U}$ contributes to this term, since $\int \boldsymbol{k} S_{nl} d\boldsymbol{k} = 0$. The discrete interaction approximation used for S_{nl} evaluates resonant quadruplets based on the intrinsic frequency and thus enforces $\int \sigma S_{nl} d\boldsymbol{k} = 0$, such that with a background current this term appears as a spurious source; since it is small compared to the other sources, it is ignored.

Relation to WAM

The concept and discretisation of wim_simple follows closely the example of WAM found in WAM (cycle 7) at `https://github.com/myWAveModel/WAM`, except for simplified action sources. Instead of the canonical wavenumber coordinates $\boldsymbol{k}$ used above, frequency $\nu = \sigma/(2\pi)$ and wavenumber angle θ with $k_x = k \sin\theta$, $k_y = k\cos\theta$ and $k = |\boldsymbol{k}|$ are used as phase dimensions, covered by an equidistant grid in θ and a logarithmic grid in ν as in WAM. Different to WAM, however, ν is here the *intrinsic* frequency, while in WAM the extrinsic frequency is used; the intrinsic frequency is better suited to represent wave–mean flow interaction, following the example of WAVEWATCH III. Instead of the spectral energy density $E(\boldsymbol{k}) = \sigma A(\boldsymbol{k})$, the model carries the WAM-like spectral density $E(\theta, \nu)$, related by $\int_{-\infty}^{0} |\boldsymbol{u}_L|^2/2 dz = \int E(\boldsymbol{k}) d\boldsymbol{k} = g \int E(\theta,\nu) d\theta d\nu$. Units of $E(\boldsymbol{k})d\boldsymbol{k}$ are m^3/s^2, thus $E(\theta,\nu)d\theta d\nu$ has units m^2 (the surface elevation variance) and $E(\theta,\nu)$ has units $m^2 s$, while $E(\boldsymbol{k})$ has units m^5/s^2; the factor g shows up accordingly in the definitions of $\boldsymbol{u}_S$ and $\boldsymbol{\tau}$ below.

The model equation formulated for $A(\theta,\nu)$ reads

$$\partial_t A + \boldsymbol{\nabla}_{\boldsymbol{x}} \cdot (\dot{\boldsymbol{x}} A) + \partial_\theta (\dot\theta A) + \partial_\nu (\dot\nu A) \;\; = \;\; S_{in} + S_{out} + S_{nl} \tag{60}$$

We need expressions for group velocity $\dot{\boldsymbol{x}}$ and the drift velocities $\dot\theta$ and $\dot\nu$.

Group velocity

Since $\boldsymbol{U} = \boldsymbol{U}(k)$, the group velocity is given by

$$\dot{\boldsymbol{x}} = (c/2)\boldsymbol{s} + \boldsymbol{U} + \boldsymbol{k} \cdot \boldsymbol{\nabla}_{\boldsymbol{k}} \boldsymbol{U} \tag{61}$$

with the phase velocity $c = \sigma/k = \sqrt{g}/\sqrt{k} = g/\sigma$ and the wave direction vector $\boldsymbol{s} = (\sin\theta, \cos\theta)$. Since the wavenumber gradient of the Doppler velocity $\boldsymbol{U}$ is purely radial (it points along $\boldsymbol{k}$),

$$\boldsymbol{k} \cdot \boldsymbol{\nabla}_{\boldsymbol{k}} \boldsymbol{U} = \begin{pmatrix} (k_x \partial_k U + k_y \partial_k V)\, k_x/k \\ (k_x \partial_k U + k_y \partial_k V)\, k_y/k \end{pmatrix} = (\boldsymbol{s} \cdot \partial_k \boldsymbol{U})\; \boldsymbol{k} = k\,(\sin\theta\, \partial_k U + \cos\theta\, \partial_k V)\; \boldsymbol{s}, \tag{62}$$

with $\partial_k \boldsymbol{U} = (\partial_k U, \partial_k V)$. The full group velocity reads

$$\dot{\boldsymbol{x}} = (c/2)\, \boldsymbol{s} + \boldsymbol{U} + k\,(\sin\theta\, \partial_k U + \cos\theta\, \partial_k V)\; \boldsymbol{s}. \tag{63}$$

With $k = (2\pi\nu)^2/g$ one has $k\,\partial_k U_i = k\partial_\nu U_i \partial\nu/\partial k = \frac{\nu}{2}\,\partial_\nu U_i$ so the radial correction can be written with a single scalar factor

$$R = \frac{\nu}{2}\,(\boldsymbol{s}\cdot\partial_\nu \boldsymbol{U})\,, \qquad \dot{\boldsymbol{x}} = (c/2)\boldsymbol{s} + \boldsymbol{U} + R\boldsymbol{s} \tag{64}$$

i.e. the same directional projection $R\,\boldsymbol{s}$ enters both components. This reduces to the pure group velocity $\dot{\boldsymbol{x}} = (c/2)\boldsymbol{s} + \boldsymbol{U}$ whenever the current has no frequency dependence ($\partial_\nu \boldsymbol{U} = 0$), e.g. for a barotropic current.

Other drift velocities

Since $\boldsymbol{U} = \boldsymbol{U}(k)$, the material derivative of $\boldsymbol{U}$ along a ray contains, besides $\partial_t \boldsymbol{U} + \dot{\boldsymbol{x}}\cdot\boldsymbol{\nabla}_{\boldsymbol{x}}\boldsymbol{U}$, also the contribution $\dot{\boldsymbol{k}}\cdot\boldsymbol{\nabla}_{\boldsymbol{k}}\boldsymbol{U}$, so $\dot\sigma$ becomes with $\dot{\boldsymbol{k}} = -(\boldsymbol{\nabla}\boldsymbol{U})\cdot\boldsymbol{k}$

$$\dot\sigma \;=\; \partial_t\Omega - \dot{\boldsymbol{U}}\cdot\boldsymbol{k} - \boldsymbol{U}\cdot\dot{\boldsymbol{k}} = (\dot{\boldsymbol{x}} - \boldsymbol{U})\cdot\dot{\boldsymbol{k}} - (\dot{\boldsymbol{k}}\cdot\boldsymbol{\nabla}_{\boldsymbol{k}}\boldsymbol{U})\cdot\boldsymbol{k} \tag{65}$$

With $\boldsymbol{\nabla}_{\boldsymbol{k}} U_i = (\partial_k U_i)\,\boldsymbol{k}/k$ and $k\,\partial_k U_i = (\nu/2)\partial_\nu U_i$ one finds $(\dot{\boldsymbol{k}}\cdot\boldsymbol{\nabla}_{\boldsymbol{k}}\boldsymbol{U})\cdot\boldsymbol{k} = (\dot{\boldsymbol{k}}\cdot\boldsymbol{s})\,k\,(\boldsymbol{s}\cdot\partial_k\boldsymbol{U}) = R\,(\boldsymbol{s}\cdot\dot{\boldsymbol{k}})$ with R from Eq. (64). Since also $\dot{\boldsymbol{x}} - \boldsymbol{U} = (c/2 + R)\,\boldsymbol{s}$ by Eq. (64), we obtain

$$\dot\sigma = 2\pi\dot\nu = ((c/2 + R)\,\boldsymbol{s})\cdot\dot{\boldsymbol{k}} - R\,(\boldsymbol{s}\cdot\dot{\boldsymbol{k}}) = (c/2)\,\boldsymbol{s}\cdot\dot{\boldsymbol{k}} \tag{66}$$

Finally,

$$\dot\theta = (\cos\theta\dot{k}_x - \sin\theta\dot{k}_y)k^{-1} \tag{67}$$

results using $\tan\theta = k_x/k_y$ and $\dot{k}_x = -k_x\partial_x U - k_y\partial_x V$ and $\dot{k}_y = -k_x\partial_y U - k_y\partial_y V$.

High frequency cut-off

Since the growth rates γ_{in} and γ_{out} increase strongly with frequency, which would require a very small time step at large ν, the wave model is only calculated up to a cut-off frequency ν_c, and for the higher frequencies $\nu > \nu_c$ a Phillips spectrum with a ν^{-5} tail of the wave spectrum is applied, the so-called high-frequency tail. It has the form $E_{hi}(\nu) = E(\nu_c,\theta)\nu_c^5\nu^{-5}$. The cut-off frequency ν_c is calculated as in WAMDI (1988)

$$\nu_c = \max(2.5\bar\nu, 3\nu_{pm}) \tag{68}$$

with the mean frequency $2\pi\bar\nu = \int E d\theta d\nu / \int (E/\sigma) d\theta d\nu$, the "wind sea" frequency $2\pi\nu_{pm} = g/(28u_\star)$, and the friction velocity $u_\star$. For $\nu > \nu_c$ it is assumed that Eq. (60) becomes local and steady, such that the source terms balance locally in space and time and in ν and θ, i.e. $S|_{\nu>\nu_c} = 0$. The per-column prognostic cut-off and the imposed ν^{-5} tail are activated by the attribute `mask_energy` (default off); the resulting prognostic/diagnostic bin mask is also the basis of the equilibrium tail-flux booking of `tail_flux = 'input'` described below, which therefore requires `mask_energy` to be active.

Source terms

The source terms S_{in} and S_{out} are parameterized by growth rates, $S_{in} = \gamma_{in} A$ and $S_{out} = \gamma_{out} A$, for which the simplest forms following WAMDI (1988) are used

$$\gamma_{out} = -\beta_o g^{-4} \bar{E}^2 \bar{\sigma}^7 \ \sigma^2 \quad , \quad \gamma_{in} = \sigma \frac{\rho_a}{\rho_o} \times \ 0.25 \max\left(0, 28(u^*/c)\cos(\theta - \theta*) - 1\right)) \tag{69}$$

with mean energy $\bar{E} = \int E d\theta d\nu$, mean frequency $\bar{\sigma} = \bar{E} / \int (E/\sigma) d\theta d\nu$, air,water density ρ_a, ρ_o, atmospheric friction velocity magnitude $u^\star$, with direction θ^*, deep water phase velocity $c = g/\sigma$, and the parameter $\beta_o = 2.56$. wim_simple carries only this WAMDI pair of input and dissipation. The input of Janssen and the WAM4 dissipation are options of wim_realistic. The mean frequency is clipped to the resolved spectral range,

$$\bar{\sigma} \leftarrow \min(\max(\bar{\sigma},\, \sigma_0),\, \sigma_{N_\nu - 1}) \tag{70}$$

The non-linear source term S_{nl} is the discrete interaction approximation (DIA) of Hasselmann et al. (1985), taken from WAM (cycle 7). A fortran version directly from the original WAM legacy code, and two python versions (one with jax/jit acceleration, one pure numpy) are available. The attribute "snonlin_backend" determines which one to use ('fortran'/'jax'/'numpy', or 'exact' for the reference integral described below). With the numpy backend of pyOM3 the default is 'fortran'. With the JAX backend the default is 'jax'. Under JAX the setting 'numpy' is replaced by 'jax', and the setting 'fortran' raises an error, since the Fortran code needs numpy arrays. The three DIA versions agree to round-off. The JAX version is a vectorised form of the WAM loop for the GPU. It is described in the section on the JAX backend below.

An exact scattering integral as reference

In addition to the DIA, an exact evaluation of the scattering integral is implemented (subpackage snl_exact, selected by "snonlin_backend" = 'exact'), intended as a reference for assessing the DIA and future parameterisations of S_{nl} rather than as a source term for production integrations. For the action density $A(\boldsymbol{k})$ the four-wave kinetic equation for deep water reads

$$S_{nl}(\boldsymbol{k}_1) \ = \ \int G\, \delta(\boldsymbol{k}_1 + \boldsymbol{k}_2 - \boldsymbol{k}_3 - \boldsymbol{k}_4)\, \delta(\sigma_1 + \sigma_2 - \sigma_3 - \sigma_4)\ (A_3 A_4 (A_1 + A_2) - A_1 A_2 (A_3 + A_4))\ d\boldsymbol{k}_2\, d\boldsymbol{k}_3\, d\boldsymbol{k}_4$$

with $A_i = A(\boldsymbol{k}_i)$ and the deep-water coupling coefficient $G(\boldsymbol{k}_1, \boldsymbol{k}_2, \boldsymbol{k}_3, \boldsymbol{k}_4)$ of Webb (1978). Here G is evaluated from the closed form of Webb's coupling coefficient. This closed form is ported from the routine xc_webb of the exact-NL code of van Vledder (xnl) in WAVEWATCH III, and its result is divided by g^2. The canonical four-wave kernel T of the Zakharov equation (Krasitskii, 1994) was derived independently from the water-wave Hamiltonian. It serves to verify the identity $G = 4\pi(2\pi)^4 T^2$ on resonant quadruplets, which holds to a relative accuracy of $3 \cdot 10^{-13}$.

The integral is evaluated with the Webb–Resio–Tracy (WRT) method: the wavenumber delta function eliminates $\boldsymbol{k}_2$, and the frequency delta function reduces the remaining integration to a line integral along the resonance locus $\sigma(\boldsymbol{k}_2) - \sigma(\boldsymbol{k}_2 + \boldsymbol{P}) + \sigma_1 - \sigma_3 = 0$ with $\boldsymbol{P} = \boldsymbol{k}_1 - \boldsymbol{k}_3$, weighted by the inverse gradient of the frequency mismatch. In deep water the loci form a one-parameter family under scaling and rotation. On the geometric frequency grid with uniform directions the locus geometry, coupling coefficients and interpolation stencils are computed once per grid and stored in a table for $\theta_1 = 0$. The table covers every frequency pair (i_1, i_3) of $\boldsymbol{k}_1$ and $\boldsymbol{k}_3$ and all direction differences $\theta_3 - \theta_1$. It is applied to all other directions θ_1 by integer shifts of the direction index, which is exact on the uniform direction grid. Same-frequency pairs, for which the locus degenerates to a straight line, are included separately. Action densities at the off-grid points $\boldsymbol{k}_2, \boldsymbol{k}_4$ are obtained by bilinear interpolation in $(\ln k, \theta)$. Before evaluation the backend appends by default "pad_hi" $= 8$ frequency rings of a parametric $E \propto \nu^{-5}$ tail above the model frequencies. In each direction this tail is matched to E at the last model frequency. Quadruplets with any member beyond the extended range are discarded entirely. This strict truncation preserves the equilibria of the operator on the extended grid exactly. For constant A on that grid the discrete transfer vanishes at machine precision, and for the Rayleigh–Jeans spectrum $A \propto 1/\sigma$ it vanishes at the level of the interpolation error, converging quadratically with the grid spacing. These equilibrium properties refer to the operator without the parametric tail ("pad_hi" $= 0$). With the default tail a model spectrum of constant A is continued by the ν^{-5} tail. The padded spectrum is then no equilibrium, and the transfer does not vanish.

Two discretisations of Eq. (71) are available. They are selected by the attribute "mode" of the backend object `pyOM3.wim_simple.snl_exact.snonlin_exact.Snonlin`, an instance of the class `_SnonlinExact`. "mode" is not an attribute of the model instance. Its default is 'conservative'. The 'direct' mode evaluates the collision integral for each $\boldsymbol{k}_1$ separately, in close analogy to other WRT implementations; the 'conservative' mode distributes each quadruplet contribution over all four wavenumbers with weights $(+, +, -, -)/4$ through the same interpolation stencils, which closes the total action budget of the discrete transfer to machine precision, including an explicit account of the action exchanged with the parametric tail. The two modes are independent discretisations of the same integral; their difference decreases with spectral resolution and serves as a built-in convergence diagnostic. For pointwise comparisons against reference transfers the 'direct' mode is preferable at moderate resolution, while the 'conservative' mode is appropriate whenever exact closure of the action budget matters.

The implementation was validated against the equilibria above, against the near-vanishing of the transfer for the Zakharov–Filonenko spectrum $E \propto \nu^{-4}$, against the WAM DIA for the overall magnitude, and against the independent exact-NL code of van Vledder (as distributed with WAVEWATCH III and SWAN) run on identical input: with the interaction filtering of that code disabled, the transfer lobes of the two implementations agree at the level of a few percent for typical single-peaked test spectra, with the exact-NL result bracketed by the 'direct' and 'conservative' modes. The interface of the exact backend matches the other S_{nl} backends, including the diagonal functional derivative $\partial S_{nl}/\partial E$ used by the

semi-implicit time stepping. In both modes this derivative is that of the 'direct' rate at $\boldsymbol{k}_1$. It is the derivative of the bracket in Eq. (71) with respect to A_1, accumulated over the table entries with $\boldsymbol{k}_1$ in the first role. With the default 'conservative' mode it is therefore not the exact derivative of the returned transfer. Since the cost of an exact evaluation grows rapidly with spectral resolution and applies per horizontal grid point, the backend evaluates at most "max_cells" = 64 spectra per call. A call with more cells raises an error unless "max_cells" is raised. Evaluation can be restricted to selected horizontal points through the attribute "points", a list of index pairs (i_x, i_y) of interior cells. Its default None selects all cells. Cells not in "points" get $S_{nl} = 0$ and $\partial S_{nl}/\partial E = 0$. Like "mode", the attributes "max_cells", "points" and "pad_hi" belong to the backend object, not to the model instance.

Stokes velocity

Stokes velocity is

$$\boldsymbol{u}_S(z) = \int\int 2(k/\sigma)e^{2kz}\boldsymbol{k}E(\boldsymbol{k})d\boldsymbol{k} = (2/g)\int_0^\infty \sigma^3 e^{2kz}\int_0^{2\pi} \boldsymbol{s}Ed\theta d\nu \tag{72}$$

with direction vector $\boldsymbol{s} = (\sin\theta, \cos\theta)$. In the code the frequency integral is a sum over all model frequency bins. With `mask_energy` this sum includes the diagnostic bins above the per-column cut-off, which hold the imposed ν^{-5} spectrum. The contribution by the high frequency tail is added to this sum. For the Stokes velocity the tail starts at the last model frequency bin. In Eq. (73) and Eq. (74) below, ν_c and $E(\nu_c, \theta)$ are therefore the frequency and the spectrum of that bin, which is also part of the resolved sum. The tail part is given by

$$\boldsymbol{u}_S^{hi} = (2/g)\int_{\nu_c}^\infty\int_0^{2\pi} \sigma^3 e^{2z\sigma^2/g}\boldsymbol{s}E(\nu_c,\theta)\nu_c^5\nu^{-5}d\theta d\nu \approx (2/g)\nu_c\int_0^{2\pi}\sigma_c^3\boldsymbol{s}E(\nu_c,\theta)d\theta \tag{73}$$

assuming that the uppermost model level thickness consumes the factor $e^{2z\sigma^2/g}$ and with $\sigma_c = 2\pi\nu_c$. Below the uppermost vertical level, $\boldsymbol{u}_S^{hi} = 0$ in this approximation. For small effective vertical decay scale of the tail contribution or large vertical levl thickness, this approximation gets worse. The frequency integral in Eq. (73) can in fact be carried out exactly, retaining the full depth dependence of the exponential. With $\sigma_c = 2\pi\nu_c$ the result is

$$u_S^{hi}(z) = \frac{2}{g}\,\nu_c\int_0^{2\pi}\mathbf{s}\,E(\nu_c,\theta)\,\sigma_c^3\left[e^{2z\sigma_c^2/g} - \sigma_c\sqrt{-\frac{2\pi z}{g}}\,\mathrm{erfc}\left(\sigma_c\sqrt{-\frac{2z}{g}}\right)\right]d\theta, \tag{74}$$

valid for $z \le 0$. The approximation Eq. (73) is recovered in the near-surface limit $\sigma_c^2|z|/g \ll 1$, where the erfc term vanishes and $e^{2z\sigma_c^2/g} \to 1$, leaving the bracket equal to σ_c^3 and hence the prefactor $\frac{2}{g}\nu_c\,\sigma_c^3$ of Eq. (73). Note that WAM (cycle 7) uses the approximated version; the difference grows with the ratio of the uppermost level thickness to the vertical decay scale of the tail.

Wave stress

The stress force in the ocean is given, with the dissipation energy source $\dot{E}_{out} = \gamma_{out} E$ in (θ, ν) coordinates, by

$$\partial_z \boldsymbol{\tau} = -(2/g) \int_0^\infty e^{2kz} \sigma^3 \int_0^{2\pi} \boldsymbol{s} \dot{E}_{out} d\theta d\nu \tag{75}$$

The stress itself is

$$\boldsymbol{\tau}_o = - \int_0^\infty \sigma \int_0^{2\pi} \boldsymbol{s} \dot{E}_{out} d\theta d\nu \tag{76}$$

The contribution by the high-frequency tail assuming the WAMDI dissipation scale is

$$\partial_z \boldsymbol{\tau}^{hi} = 2\nu_c^5 \beta_o g^{-5} \bar{E}^2 \bar{\sigma}^7 (2\pi)^5 \int_{\nu_c}^\infty e^{(8\pi^2/g) z \nu^2} d\nu \int_0^{2\pi} \boldsymbol{s} E(\nu_c, \theta) d\theta \tag{77}$$

with $E_{hi}(\nu) = E(\nu_c, \theta) \nu_c^5 \nu^{-5}$ and $\gamma_{out} = -\beta_o g^{-4} \bar{E}^2 \bar{\sigma}^7 \sigma^2$ and the finite integral

$$\int_a^\infty e^{-bx^2} dx = \frac{\sqrt{\pi} - \sqrt{\pi} \mathrm{erf}\left(a\sqrt{b}\right)}{2\sqrt{b}} \tag{78}$$

For the stress we find for the high-frequency tail

$$\boldsymbol{\tau}_o^{hi} = (2\pi)^3 \nu_c^4 \beta_o g^{-4} \bar{E}^2 \bar{\sigma}^7 \int_0^{2\pi} \boldsymbol{s} E(\nu_c, \theta) d\theta \tag{79}$$

In the code the resolved parts of Eq. (75) and Eq. (76) are sums over all model frequency bins. The frequency axis carries ghost (halo) bins on both sides. The tails Eq. (77) and Eq. (79) start at the first ghost bin above the last model frequency, and so do the tails Eq. (81) and Eq. (84) of the alternative formulation below. In these tails ν_c is the frequency of the ghost bin and $E(\nu_c, \theta)$ is the spectrum stored in it. The ghost bin holds the imposed ν^{-5} value only when `mask_energy` is set and the cut-off lies below the ghost-bin frequency. Otherwise the ghost bin holds zero. The tails of the stress, of the stress force and of the `'input'` booking then vanish, while the Stokes tail is still added.

An alternative formulation of the tail stress follows from the equilibrium assumption that underlies the imposed ν^{-5} shape itself: for $\nu > \nu_c$ the sources balance, $\dot{E}_{in} + \dot{E}_{nl} + \dot{E}_{out} = 0$, so the flux to the ocean, $-\dot{E}_{out}$, can be evaluated as $\dot{E}_{in} + \dot{E}_{nl}$ instead of extending the WAMDI dissipation scale (which is calibrated on the resolved spectrum and underestimates the dissipation that maintains the equilibrium range). The input part is

$$\boldsymbol{\tau}_o^{hi,in} = \int_{\nu_c}^\infty \sigma \, \gamma_{in} \, \nu_c^5 \nu^{-5} \, d\nu \int_0^{2\pi} \boldsymbol{s} \, E(\nu_c, \theta) \, d\theta \tag{80}$$

with γ_{in} from Eq. (69). Writing $28(u^\star/c)\cos(\theta-\theta^*) = a\nu$ with $a(\theta) = 28u^\star 2\pi\cos(\theta-\theta^*)/g$, the input is active only above $\nu_1 = \max(\nu_c, 1/a)$, and the frequency integral is elementary, $\int_{\nu_1}^{\infty}(a\nu^{-2} - \nu^{-3})\,d\nu = a/\nu_1 - 1/(2\nu_1^2)$, giving per direction

$$\boldsymbol{\tau}_o^{hi,in} = \frac{\rho_a}{\rho_o}\frac{(2\pi)^2\nu_c^3}{4}\int_0^{2\pi} \boldsymbol{s}\, E(\nu_c,\theta)\left\{\begin{array}{ll} a\nu_c - \frac{1}{2}, & a\nu_c \geq 1 \\ \frac{1}{2}\,(a\nu_c)^2, & 0 < a\nu_c < 1 \end{array}\right\} d\theta \tag{81}$$

(continuous at $a\nu_c = 1$; zero for $\cos(\theta-\theta^*) \leq 0$). The integrand decays like ν^{-2}, i.e. the integral converges exactly as benignly as the WAMDI form in Eq. (79); note that the corresponding integral with the dissipation of Janssen (1994; see also Komen et al. 1994, and the ECMWF IFS documentation, Part VII) diverges strongly because of its $(k/\bar{k})^4$ term – its $(k/\bar{k})^2$ term behaves like the WAMDI form – which is the reason the WAMDI scale is used in Eq. (79). The nonlinear part uses the conservation of the exact transfer over the full spectrum, $\int_0^\infty \sigma \int \boldsymbol{s}\,\dot{E}_{nl}\,d\theta\,d\nu = 0$, so that

$$\boldsymbol{\tau}_o^{hi,nl} = \int_{\nu_c}^{\infty}\sigma\int_0^{2\pi}\boldsymbol{s}\,\dot{E}_{nl}\,d\theta\,d\nu = -\int_0^{\nu_c}\sigma\int_0^{2\pi}\boldsymbol{s}\,\dot{E}_{nl}\,d\theta\,d\nu \tag{82}$$

i.e. the cascade flux into the tail equals minus the resolved-range integral of the computed nonlinear transfer (the "cascade credit"). For completeness, the resulting total stress on the ocean in this formulation is

$$\boldsymbol{\tau}_o = -\int_0^{\nu_c}\sigma\int_0^{2\pi}\boldsymbol{s}\left(\dot{E}_{out} + \dot{E}_{nl}\right)d\theta\,d\nu \;+\; \boldsymbol{\tau}_o^{hi,in} \tag{83}$$

where the first term is the resolved dissipation of Eq. (76) restricted to the prognostic range with the cascade credit Eq. (82) folded in, and the second term is the analytic tail input Eq. (81).

The depth-distributed version (the "stress work" of Eq. (75)) follows with the same replacement of the tail rate. With $-\dot{E}_{out} = \gamma_{in}E(\nu_c,\theta)\nu_c^5\nu^{-5}$ above ν_c and $\sigma^3\gamma_{in} = \frac{\rho_a}{\rho_o}\frac{(2\pi)^4}{4}\nu^4(a\nu-1)$,

$$\partial_z\boldsymbol{\tau}^{hi,in} = \frac{2}{g}\frac{\rho_a}{\rho_o}\frac{(2\pi)^4}{4}\nu_c^5\int_0^{2\pi}\boldsymbol{s}\,E(\nu_c,\theta)\int_{\nu_1}^{\infty}e^{(8\pi^2/g)z\,\nu^2}\left(a-\nu^{-1}\right)d\nu\,d\theta \tag{84}$$

with $\nu_1 = \max(\nu_c, 1/a)$ as before. The a–term is the finite integral Eq. (78); the second term requires the companion

$$\int_{a_0}^{\infty} x^{-1}e^{-bx^2}dx = \frac{1}{2}E_1(a_0^2\,b) \tag{85}$$

with the exponential integral E_1. The vertical integral of Eq. (84) recovers Eq. (81) exactly. The cascade contribution Eq. (82) is deposited at the surface: it is dissipated by tail waves with $\nu \geq \nu_c$, whose deposition scales $g/2\sigma^2$ are centimetres, far below any ocean grid spacing. Both formulations are implemented, selected by the switch `tail_flux` $\in$ {`'wamdi'`, `'input'`}; the discrete treatment, including the diagnostic bins inside the grid, is described below.

Discretisation of the wave–ocean coupling and the energy exchange

All three coupling quantities of Eq. (58) – the Stokes velocity $\boldsymbol{u}_S$, the stress force $\partial_z \boldsymbol{\tau}$ and the Doppler velocity $\boldsymbol{U}$ – involve the same vertical kernel $Q(k,z) = 2ke^{2kz}$ with $\int_{-\infty}^{0} Q\,dz = 1$: $\boldsymbol{u}_S$ and $\partial_z\boldsymbol{\tau}$ assign spectral quantities to a depth profile, while $\boldsymbol{U}$ projects the Lagrangian velocity back onto the spectral grid. The exchange term C of Eq. (57) closes because $\boldsymbol{U}$ is the adjoint of the profile assignment, i.e. because the same Q appears on both sides of the depth integral. The discretisation preserves this structure: one common discrete kernel is used in all three methods of `./pyOM3/wim_simple/interface.py` (`calc_stokes`, `calc_stress_work`, `calc_doppler_vel`), given by the cell integral of Q over each ocean cell of thickness Δz with centres z_k and faces $z_{k\pm1/2} = z_k \pm \Delta z/2$ (uppermost face at $z = 0$),

$$\bar{Q}_k \Delta z = \int_{z_{k-1/2}}^{z_{k+1/2}} Q\,dz = e^{2kz_{k+1/2}} - e^{2kz_{k-1/2}} \tag{86}$$

in place of the point value $Q(z_k)\Delta z$. The cell integral is essential: the decay scale $g/(2\sigma^2)$ of the kernel is of the order of centimetres for the high frequencies of the prognostic range and for the tail, far below any ocean level thickness, so that point sampling loses a large part of the coupling amplitude (about half of the stress force for $\Delta z = 1\,m$ and a developing wind sea). The weights Eq. (86) telescope over the column, $\sum_k \bar{Q}_k \Delta z = 1 - e^{-2kD}$ for column depth D, with three consequences that hold exactly for any Δz: the column-integrated stress force equals the surface stress $\boldsymbol{\tau}_o$ of Eq. (76), the discrete Stokes transport $\sum_k \boldsymbol{u}_S(z_k)\Delta z$ equals its analytic value, and a depth-independent (barotropic) current is projected onto the Doppler velocity without attenuation, $\boldsymbol{U} = \boldsymbol{u}_L(1 - e^{-2kD})$. The frequency derivative $\partial_\nu \boldsymbol{U}$ entering the radial refraction term Eq. (64) is taken as the analytic derivative of the discrete kernel, $\partial_\nu(\bar{Q}_k\Delta z) = (2k/\nu)\,2(z_{k+1/2}e^{2kz_{k+1/2}} - z_{k-1/2}e^{2kz_{k-1/2}})$, so that refraction sees the same Doppler projection as the advection.

For the high-frequency tails the same principle is applied to the exact tail profiles. The tail of the stress force Eq. (77) contains the frequency integral $W(t) = \int_{\nu_c}^{\infty} e^{-bt\nu^2} d\nu$ with $t = -z \geq 0$ and $b = 8\pi^2/g$, evaluated with Eq. (78); its antiderivative is

$$H(t) = \int_0^t W(t')dt' = \frac{\sqrt{\pi}}{b}\left[\sqrt{bt}\,\mathrm{erfc}\left(\nu_c\sqrt{bt}\right) + \frac{1 - e^{-\nu_c^2 bt}}{\nu_c\sqrt{\pi}}\right] \tag{87}$$

and each cell receives $\left(H(-z_{k-1/2}) - H(-z_{k+1/2})\right)/\Delta z$ in place of the point value. Since $H(\infty) = 1/(b\nu_c)$, the column sum of the tail force equals its exact depth integral $\boldsymbol{\tau}_o^{hi}$ of Eq. (79), using $2(2\pi)^5\nu_c^5 g^{-5} H(\infty) = (2\pi)^3\nu_c^4 g^{-4}$; the prefactor $(2\pi)^5$ in Eq. (77) is essential for this consistency. The Stokes tail Eq. (74) is treated the same way: with $a = 2\sigma_c^2/g$ its depth profile $B(t) = \nu_c[e^{-at} - \sqrt{\pi at}\,\mathrm{erfc}(\sqrt{at})]$ has the antiderivative ($S = \sqrt{t}$)

$$I(t) = \nu_c\left[\frac{1 - e^{-at}}{a} - 2\sqrt{\pi a}\left(\frac{S^3}{3}\mathrm{erfc}(\sqrt{a}S) + \frac{1 - (aS^2+1)e^{-aS^2}}{3\sqrt{\pi}\,a^{3/2}}\right)\right] \tag{88}$$

with $I(\infty) = \nu_c/(3a)$, so that the discrete tail transport equals the exact value $(2\pi/3)\nu_c^2 \int \boldsymbol{s} E(\nu_c, \theta) d\theta$. As stated above, ν_c in Eq. (87) is the frequency of the ghost bin, while ν_c in Eq. (88) is the frequency of the last model bin. In tests the column sums reproduce $\boldsymbol{\tau}_o$ and the Stokes transport to machine precision; for a shallow column below a developed wind sea, however, a substantial part of the retained fraction $1 - e^{-2kD}$ of the wave momentum flux can be deposited below the column bottom. For a closed momentum budget the attribute `closed_bottom` extends the integration of the bottom cell to $z \to -\infty$, in the shared kernel ($e^{2kz_{bot}} \to 0$ in Eq. (86)) as well as in both tail antiderivatives ($H, I \to H(\infty), I(\infty)$), so that $\sum_k \bar{Q}_k \Delta z = 1$ exactly: the bottom cell absorbs the deep part of all coupling profiles, no wave momentum is lost, and the barotropic Doppler projection becomes exact, $\boldsymbol{U} = \boldsymbol{u}_L$, at all frequencies (with the open bottom the low-frequency bins of a shallow column see an almost vanishing Doppler current). Since the kernel remains shared, the adjoint identity Eq. (89) below is unaffected. The closure amounts to assuming $\boldsymbol{u}_L$ constant below the column floor and is the recommended setting for shallow test columns; for deep columns ($2kD \gg 1$) it makes no difference.

The energy exchanged between ocean and waves is controlled by the same kernel. On the ocean side, with the adjoint pair Eq. (86) the discrete identity

$$\sum_k \boldsymbol{u}_h^L(z_k) \cdot \partial_t \boldsymbol{u}_S(z_k)\, \Delta z = \int \boldsymbol{U} \cdot \boldsymbol{k}\; \partial_t A\; d\boldsymbol{k} \tag{89}$$

holds exactly (to machine precision) for any spectral tendency $\partial_t A$ and any Δz, over the prognostic frequency range (the tails are diagnostic and remain outside this closure). The ocean therefore receives, per coupling step, exactly the energy $\int \boldsymbol{U} \cdot \boldsymbol{k}\, \partial_t A\, d\boldsymbol{k}$ corresponding to the actual discrete evolution of the wave field, and the handoff itself introduces no discretisation error.

On the wave side, the counterpart of this energy must be extracted from (or added to) the intrinsic wave energy $\int \sigma A\, d\boldsymbol{k}$ by the propagation operator. For steady $\boldsymbol{U}$ the rays conserve $\Omega = \sigma + \boldsymbol{U} \cdot \boldsymbol{k}$, so the continuous propagation operator satisfies $\int \Omega\, \partial_t A|_{prop}\, d\boldsymbol{k} = 0$ and the intrinsic energy change $\int \sigma\, \partial_t A$ balances the ocean gain $-\int \boldsymbol{U} \cdot \boldsymbol{k}\, \partial_t A$ identically. Discretely, the flux-form advection conserves the action $\int A\, d\boldsymbol{k}$ to machine precision, but not $\int \Omega A\, d\boldsymbol{k}$: the nonlinear Superbee limiter leaves a residual. The flux through the frequency face $i + 1/2$ between the bins i and $i + 1$ uses a frequency advection velocity $\dot{\nu}$ that the code evaluates with the bin-centre frequency ν_i and the Doppler gradients of bin i. This velocity is thus offset by half a bin from the face on which it is used. In tests with a wave packet refracted across a jet (moderate Doppler shift, $\max|\boldsymbol{U} \cdot \boldsymbol{k}|/\sigma \approx 0.3$) this Ω-conservation defect amounts to about 1 % of the gross local transfer rate $\int |\dot{\sigma} A|\, d\boldsymbol{k}$ for the Superbee scheme, consistent with the $\mathcal{O}(0.5\,\%)$ energy overshoot observed for such a packet. First-order upwind advection is far worse (about 12 %, since its frequency diffusion moves action across the σ gradient), an unlimited second-order flux reduces the defect to about 0.2 %. The x-sweep is exactly energy-neutral for a zonal jet; the defect resides in the y, ν and θ sweeps.

The residual is removed exactly by a conservative energy fixer (attribute `energy_consistent` in `propagation.py`). Writing the discrete advection in flux form per phase-space grid cell

i as $\partial_t(A_i\Delta_i) = -\sum_{faces}\delta F_i$, with the phase-space volume Δ_i of the cell and F the total (limited) action transport through a cell face in $\boldsymbol{x}$, ν or θ, the transports cancel pairwise in the sum over all cells and action is conserved to machine precision for any advection scheme. Multiplying by Ω_i, summing over all grid cells and shifting the sums equivalent to a discrete partial integration yields

$$\partial_t \sum_i \Omega_i A_i \Delta_i = \sum_i \sum_{faces} F_i\, \delta\Omega_i \equiv D \tag{90}$$

with the forward difference $\delta\Omega$ of Ω across each cell face. Boundary terms vanish for closed and periodic boundaries (not for reflective ones, for which no correction is made). With the option `open_coasts` the face terms at the coast carry the energy that leaves the domain. They are therefore left out of D, and the outflow is booked separately (see the section on boundary conditions). The possible leakage D is split into contributions $\sum_{j,k} D_{j,k} = D$ per water column at discrete position (j,k), with the shared horizontal face terms split half-half between neighbouring columns, and each $D_{j,k}$ is cancelled by a small corrective action transport $\tilde{F}_m$ across the frequency faces of that column,

$$\tilde{F}_m = \lambda\, \bar{A}_m\, \delta_\nu\Omega_m \quad , \quad \lambda(\boldsymbol{x}) = -\frac{D_{j,k}}{\sum_m \bar{A}_m\, (\delta_\nu\Omega_m)^2} \tag{91}$$

with m running over all ν faces and directions θ at position (j,k), and the weight $\bar{A}_m$ given by the face average $\bar{A}_m = (A_i + A_{i+1})/2$ of the action density A of the two adjacent cells i and $i+1$, not of their action content $A_i\Delta_i$. Being a transport, $\tilde{F}$ leaves the action conservation untouched, but by the same partial integration it changes the energy by $\sum_m \tilde{F}_m\, \delta_\nu\Omega_m = -D_{j,k}$. The frequency direction is chosen for the corrective flux since it is the only one in which $\delta\Omega$ does not vanish without currents ($\delta_\nu\Omega \approx \delta_\nu\sigma > 0$ for small Doppler shift), and it is where the leakage is generated by action crossing the frequency gradient of Ω. The factor $\bar{A}_m$ ties the correction to the level of action, so that it acts as a small correction of the frequency propagation velocity rather than as a source term and positivity is not threatened, and the factor $\delta_\nu\Omega_m$ makes the denominator sign-definite, so that λ exists whenever action is present at that position. The code applies a conditioning guard to each column. The correction is made only where the magnitude of the denominator of Eq. (91) exceeds $10^{-3}\,|D_{j,k}|\,\Delta t$, both in the normalisation used in `propagation.py`. Columns below this bound are left uncorrected with $\lambda = 0$, so that a near-zero denominator does not amplify the defect. Minimising the action-weighted amplitude $\sum_m \tilde{F}_m^2/\bar{A}_m$ subject to this constraint yields the same form by the standard Lagrange-multiplier condition: among all conservative frequency-flux corrections that restore the energy, Eq. (91) is the smallest one in the action-weighted least-squares sense, i.e. the correction perturbs the solution as little as possible.

The correction is local (one scalar per position $\boldsymbol{x}$, no global communication) and dormant without currents: for $\boldsymbol{U} = 0$ we have $\Omega = \sigma$, no frequency transports exist, and $D_{j,k} = 0$ so that $\lambda = 0$. In tests the correction amounts to about 1 % of the frequency transports, the tendency-level defect drops from 0.9 % of the gross transfer to machine

precision, and in long integrations of a wave packet through a jet the energy $\int \Omega A \, d\boldsymbol{k}$ is conserved exactly (Adams–Bashforth time stepping preserves this, since each stored tendency is corrected individually and Ω is constant while $\boldsymbol{U}$ is frozen between coupling updates), whereas without the fixer the spurious drift amounts to roughly half of the genuine exchange. The transfer from ocean to waves is thus exact on both sides; a structurally energy-conserving advection would achieve the same without an a posteriori correction and remains an option for future work.

The coupling to the ocean

Stokes velocity, wave stress, and wave stress force are calculated in `./pyOM3/wim_simple/interface.py`. The methods there may be called and results passed to the ocean model. The velocity variable of the coupled ocean model is interpreted as the three-dimensional Lagrangian mean velocity $\boldsymbol{u}_L = \boldsymbol{u} + \boldsymbol{u}_S$, with the Eulerian mean velocity $\boldsymbol{u}$ and the Stokes velocity $\boldsymbol{u}_S$, which is the most natural choice for energetic consistency (it also follows the example of interpreting the model's velocity as the isopycnal thickness weighted velocity to account for the effect of meso-scale eddies). Starting from the Craik–Leibovich equations of motion, the momentum equation for $\boldsymbol{u}_L$ becomes

$$\partial_t \boldsymbol{u}_L - \boldsymbol{u}_L \times \boldsymbol{\nabla} \times (\boldsymbol{u}_L - \boldsymbol{u}_S) + f\boldsymbol{z} \times \boldsymbol{u}_L = -\boldsymbol{\nabla}\pi + \partial_t \boldsymbol{u}_S + (\partial_z \boldsymbol{\tau}, 0) \tag{92}$$

with the three-dimensional operator $\boldsymbol{\nabla} = (\boldsymbol{\nabla}_{\boldsymbol{x}}, \partial_z)$ and a gradient force $\boldsymbol{\nabla}\pi$ collecting the pressure gradient and wave-driven gradient components. The ocean model thus receives the wave-driven force $\partial_t \boldsymbol{u}_S$, the vortex force

$$\boldsymbol{F} = -\boldsymbol{u}_L \times \boldsymbol{\nabla} \times \boldsymbol{u}_S \tag{93}$$

with $\boldsymbol{u}_L \cdot \boldsymbol{F} = 0$, and the wave momentum flux (stress), either as the surface stress $\boldsymbol{\tau}_o$ into the uppermost grid cell or as the interior stress force $\partial_z \boldsymbol{\tau}$; the tracers are advected by $\boldsymbol{u}_L$. Note that the Coriolis term acts on the full $\boldsymbol{u}_L$: with $w = 0$, identical results are obtained using $\boldsymbol{u}$ as model velocity and adding the Stokes–Coriolis force $-f\boldsymbol{z} \times \boldsymbol{u}_S$ instead of $\partial_t \boldsymbol{u}_S$, but with $w \neq 0$ the Lagrangian formulation is required to stay energetically consistent. The scalar product of Eq. (92) with $\boldsymbol{u}_L$ contains no transfer terms related to the vortex force and the Coriolis–Stokes force; the direct transfer to or from the waves is by $\boldsymbol{u}_L \cdot \partial_t \boldsymbol{u}_S$ and by the stress work $\boldsymbol{u}_h^L \cdot \partial_z \boldsymbol{\tau}$, which together form the exchange C of Eq. (57).

Discrete vortex force for horizontally uniform $\boldsymbol{u}_S$ (nh_simple)

For a horizontally uniform Stokes velocity $\boldsymbol{u}_S = (u_S(z), v_S(z), 0)$, as used in the non-hydrostatic model nh_simple, the wave-driven force of Eq. (93) becomes

$$\boldsymbol{F} = -\boldsymbol{u}_L \times \boldsymbol{\nabla} \times \boldsymbol{u}_S = \begin{pmatrix} w\,\partial_z u_S \\ w\,\partial_z v_S \\ -u\,\partial_z u_S - v\,\partial_z v_S \end{pmatrix} \quad , \quad \boldsymbol{u}_L \cdot \boldsymbol{F} = 0 \tag{94}$$

writing $\boldsymbol{u}_L = (u, v, w)$ in this section for brevity. On the C-grid, u, v and w are staggered at the cell faces $(i+\frac{1}{2}, j, k)$, $(i, j+\frac{1}{2}, k)$ and $(i, j, k+\frac{1}{2})$, and the vertical Stokes shear is placed on the w-levels,

$$S^u_{k+\frac{1}{2}} = \frac{u_{S,k+1} - u_{S,k}}{\Delta z} \quad , \quad S^v_{k+\frac{1}{2}} = \frac{v_{S,k+1} - v_{S,k}}{\Delta z} \tag{95}$$

with $S^u = S^v = 0$ at the rigid lid and the flat bottom. Introducing the two-point averages $\overline{\phi}^x_{i+\frac{1}{2}} = (\phi_i + \phi_{i+1})/2$ and analogous in y and z, the discrete force $\boldsymbol{F} = (F^u, F^v, F^w)$ is

$$F^u_{i+\frac{1}{2},j,k} = \overline{\overline{w}^x\, S^u}^z \quad , \quad F^v_{i,j+\frac{1}{2},k} = \overline{\overline{w}^y\, S^v}^z \quad , \quad F^w_{i,j,k+\frac{1}{2}} = -\,\overline{\overline{u}^x}^z S^u - \overline{\overline{v}^y}^z S^v \tag{96}$$

i.e. the products $\overline{w}^x S^u$ are formed on the cell edges $(i+\frac{1}{2}, j, k+\frac{1}{2})$ and averaged back to the u-points, while for F^w the horizontal velocities are averaged first to the cell centers and then to the w-levels where the shear is located. The averaging operators in Eq. (96) are pairwise transposed: each average that moves w towards u in F^u is mirrored by an average moving u towards w in F^w. With the summation-by-parts identity for the two-point average

$$\sum_i \psi_{i+\frac{1}{2}}\, \overline{\phi}^x_{i+\frac{1}{2}} = \sum_i \phi_i\, \overline{\psi}^x_i \tag{97}$$

which holds without boundary terms in the periodic horizontal directions, and in the vertical because the boundary contributions carry the factor $S^u = S^v = 0$ at the lid and bottom faces, the total work of the discrete force vanishes exactly,

$$\sum_{i,j,k} (u F^u + v F^v + w F^w) = 0 \tag{98}$$

for arbitrary $\boldsymbol{u}_L$ and arbitrary profiles $u_S(z)$, $v_S(z)$; this has been verified numerically to machine precision for random fields. Note that the work vanishes only in the sum.

The vanishing work is a property of the spatial operator and does not by itself guarantee numerical stability. Since $\boldsymbol{F}$ is perpendicular to $\boldsymbol{u}_L$, it turns the velocity vector without changing its magnitude, i.e. it acts locally as a rotation with angular frequency $|\partial_z \boldsymbol{u}_S|$, analogous to inertial oscillations with frequency f. An explicit third-order Adams–Bashforth time stepping scheme is stable for such neutral oscillations only if the angle covered per time step remains below $|\partial_z \boldsymbol{u}_S| \Delta t \lesssim 0.72$. Since the Stokes shear is largest in the uppermost grid cell and grows with the developing sea, this number should be monitored during integrations together with the advective Courant numbers.

Discrete vortex force in the hydrostatic case (pe_simple)

In the hydrostatic model pe_simple the Stokes velocity $\boldsymbol{u}_S = (u_S, v_S, 0)$ varies in all three dimensions, and the force Eq. (93) is truncated to

$$\boldsymbol{F} = \begin{pmatrix} -v\,\zeta_S \\ u\,\zeta_S \end{pmatrix} \quad , \quad \zeta_S = \partial_x v_S - \partial_y u_S \quad , \quad \boldsymbol{u}^L_h \cdot \boldsymbol{F} = 0 \tag{99}$$

with $\boldsymbol{u}_h^L = (u, v)$ in this section for brevity. The Stokes-shear terms $w\partial_z u_S$ and $w\partial_z v_S$, part of the first and second component of the total force $-\boldsymbol{u}_L \times \boldsymbol{\nabla} \times \boldsymbol{u}_S$, respectively, are deliberately omitted. This is because the scalar product of these terms with $\boldsymbol{u}_h^L$ is only cancelled by w times the vertical force component $-u\partial_z u_S - v\partial_z v_S$, which the hydrostatic model does not carry. Retained alone, the Stokes-shear terms would thus perform the uncancelled work $w\,(u\partial_z u_S + v\partial_z v_S)$ on $\boldsymbol{u}_L$. The omitted terms are the so-called Stokes shear force of Suzuki and Fox-Kemper (2016). Their neglect is also in agreement with quasi-geostrophic scaling where the Rossby number Ro becomes small: within the Stokes layer of depth h_S the vertical velocity of balanced flow is limited by the surface kinematic condition and its $O(Ro)$ divergence to $|w| \lesssim Ro\,U h_S/L$, such that $|w\partial_z u_S| \sim Ro\,U U_S/L$ independent of h_S and smaller by $O(Ro)$ than the retained vortex force.

On the C-grid, u_S and v_S are given at the cell centers and averaged to the u- and v-points, and the Stokes vorticity is formed at the cell corners $(i+\frac{1}{2}, j+\frac{1}{2})$ by centered differences,

$$\zeta_S = \frac{\overline{v_S}^{\,y}_{i+1} - \overline{v_S}^{\,y}_{i}}{\Delta x} - \frac{\overline{u_S}^{\,x}_{j+1} - \overline{u_S}^{\,x}_{j}}{\Delta y} \tag{100}$$

The products are formed at the corners first, and then averaged back to the velocity points,

$$F^u_{i+\frac{1}{2},j} = -\overline{\zeta_S\,\overline{v}^{\,x}}^{\,y} \quad , \quad F^v_{i,j+\frac{1}{2}} = \overline{\zeta_S\,\overline{u}^{\,y}}^{\,x} \tag{101}$$

with $\boldsymbol{F} = (F^u, F^v)$. Applying the summation-by-parts identity Eq. (97) once in y to the first and once in x to the second term of the work moves both averages onto the velocities, and the two contributions meet at the corners,

$$\sum uF^u + \sum vF^v = \sum_{\text{corners}} \zeta_S\,(-\overline{v}^{\,x}\,\overline{u}^{\,y} + \overline{u}^{\,y}\,\overline{v}^{\,x}) = 0 \tag{102}$$

i.e. the cancellation is not only global as in nh_simple but holds corner by corner. The boundary terms of Eq. (97) vanish because the velocities vanish on the channel walls and on land, so the identity holds in the presence of topography; this has been verified to machine precision for random fields on a masked channel.

Discrete aspects of the momentum coupling

The discretisation of the remaining coupling terms in the ocean model (C-grid) is chosen such that the energetic properties of the continuous system are preserved where possible, and monitored where not.

Stokes–Coriolis force. Continuously, the combined rotational forces are $(f+\zeta)\hat{z} \times \boldsymbol{u}_L$, which does no work against $\boldsymbol{u}_L$. For the Coriolis part this property carries over exactly to the discrete level if the Stokes velocity is first interpolated from T to the U/V points and then passed through the *same* discrete Coriolis operator C as the Eulerian velocity (the energy-conserving form of the main code, with f evaluated at T points inside the adjoint-paired averaging). By linearity the total discrete Coriolis force is then $C(\boldsymbol{u}_L)$, and

the antisymmetry of the operator gives $\sum \boldsymbol{u}_L \cdot C(\boldsymbol{u}_L) = 0$ to machine precision for arbitrary $f(y)$. Note that adding instead a separate Stokes–Coriolis term with its own (single) T-to-U/V averaging leaves an $O(\Delta x^2)$ residual in the $\boldsymbol{u}_L$ energy budget, since $\boldsymbol{u}_S$ then enters the budget once as a T-point field and once through a double interpolation; the difference is a commutator between the 1-2-1 smoothing and f.

Vortex force. With the discretisations of the previous section the $\boldsymbol{u}_L$-work of the applied vortex force vanishes exactly (globally in nh_simple, corner by corner in pe_simple). The coupled driver nevertheless computes $\sum \boldsymbol{u}_L \cdot \boldsymbol{F}$ (volume integrated, with the vortex force exactly as applied) every coupling step and stores it in the coupling output, so that it can be verified to remain at round-off level.

Stokes advection of tracers. The advection of buoyancy by $\boldsymbol{u}_S$ is added in flux form: $\boldsymbol{u}_S$ is averaged from T to the U/V faces, the vertical Stokes velocity w_S is diagnosed from the discrete Stokes continuity equation integrated upward from the bottom ($w_S(-D) = 0$), and the fluxes are differenced like the model's own tracer fluxes, with zero buoyancy flux through the surface face. This form is exactly conservative (the fluxes telescope, so $\int b\, dV$ is preserved to round-off) and constancy-preserving in the interior (a constant b yields zero tendency, since the diagnosed w_S closes the three-dimensional Stokes divergence discretely); both properties were verified to machine precision. In the top cell the surface no-flux condition places the convergence of the Stokes transport (the Stokes pumping w_S at the surface face) into the cell budget, which is the physically correct location: it is compensated by the Eulerian anti-Stokes flow driven by the Stokes–Coriolis force. Centered fluxes are used; the Stokes Courant number is $O(10^{-3})$, far below the regime where flux limiting matters.

Stress and vertical placement. The wave stress is kinematic (the spectrum is the variance density, so no density factor appears; the air–water density ratio enters through the growth rate of the input source term) and is applied either to the surface face of the top wet cell ($\tau/\Delta z$) or depth-distributed with the same cell-integrated vertical kernel that underlies the Stokes velocity and the Doppler projection, which makes the discrete energy-exchange identity (the C-closure of the previous section) hold to machine precision. Note that with a top-cell thickness large compared to the Stokes decay scale $1/(2k)$, the stored top-cell Stokes velocity is a cell average and correspondingly smaller than the surface value; resolving the near-surface Stokes shear requires Δz of a few metres.

High-frequency tail fluxes to the ocean (tail_flux switch). Above the prognostic cutoff ν_c the spectrum is diagnostic ($E \propto \nu^{-5}$, imposed by the tail parameterisation), and its momentum and energy fluxes to the ocean must be added analytically to the resolved-range dissipation integrals. Two formulations are implemented, selected by the attribute `tail_flux`:

(i) `'wamdi'` (default): the WAMDI dissipation rate continued over the tail, $\int_{\nu_c}^{\infty} \gamma_{out}\, E_{hi}\, \sigma\, d\nu$ with $\gamma_{out} \propto \bar{\sigma}(\sigma/\bar{\sigma})^2(\bar{\alpha}/\alpha_{PM})^2$. The integrand decays like ν^{-2} and converges; note that the corresponding integral with the dissipation of Janssen (1994) diverges strongly because of its $(k/\bar{k})^4$ term (its $(k/\bar{k})^2$ term behaves like the WAMDI form), which is why the WAMDI form is used for this term. However, γ_{out} is the whitecapping rate calibrated on the resolved

spectrum and underestimates the dissipation that actually maintains the ν^{-5} equilibrium range — the very reason the tail must be imposed diagnostically — so this form systematically underestimates the tail fluxes (in tests by a substantial factor in the momentum flux).

(ii) `'input'`: in the equilibrium range $S_{in}+S_{nl}+S_{out}=0$ holds, so the flux to the ocean, $-S_{out}$, equals $S_{in}+S_{nl}$. The input part is the analytic integral of the WAMDI growth rate over the tail: with $a(\theta)=28u_\star 2\pi\cos(\theta-\theta_w)/g$ (the input parameter evaluated at ν_c), the input being active only above $\nu_1=1/a$,

$$I_\tau(\theta)=\frac{\epsilon}{4}(2\pi)^2\nu_c^3\,E_c(\theta)\begin{cases}a\nu_c-\frac{1}{2}, & a\nu_c\geq 1\\ \frac{1}{2}(a\nu_c)^2, & 0<a\nu_c<1\end{cases}\qquad I_\Phi(\theta)=\frac{\epsilon}{4}(2\pi)\nu_c^2 g\,E_c(\theta)\begin{cases}\frac{a\nu_c}{2}-\frac{1}{3}, & a\nu_c\geq 1\\ \frac{1}{6}(a\nu_c)^3, & 0<a\nu_c<1\end{cases} \tag{103}$$

($\epsilon=\rho_a/\rho_w$, $E_c(\theta)=E(\nu_c,\theta)$; continuous at $a\nu_c=1$; zero upwind; the integrands again decay like ν^{-2}). The same booking applies to the *in-grid* diagnostic bins (between the per-column prognostic cut-off m_0 and the last grid frequency, where the ν^{-5} shape is imposed): there the flux is evaluated discretely as $\gamma_{in}E$ with the actual imposed spectrum, while the prognostic bins keep the true $\gamma_{out}E$ dissipation — booking the in-grid diagnostic range at γ_{out} would be inconsistent with the equilibrium assertion that the imposed tail itself encodes (and underestimates the flux substantially: the in-grid diagnostic range can carry several times the beyond-grid tail). The nonlinear part uses the conservation of the exact transfer over the full spectrum, $\int_{tail} S_{nl}\,d\Omega=-\int_{resolved} S_{nl}\,d\Omega$: the *prognostic*-range integrals of the stored S_{nl} (momentum weight σ, energy weight g) are credited to the ocean fluxes (the "cascade credit"; the in-grid diagnostic S_{nl} is part of the equilibrium booking there, and the algebra of the two ranges collapses exactly to the prognostic-masked integral). On the truncated grid the discrete DIA is not exactly conservative at the cutoff; the residual is deliberately included in the credit — whatever the discrete operator removes from the resolved range is deposited in the tail. For the depth-distributed stress (method `calc_stress_work`, as opposed to the surface stress of `calc_stress`) the tail is deposited with its exact profile: with $b=8\pi^2/g$, $t=-z$, $\beta_L=b\,t\nu_L^2$, the cumulative-from-surface tail stress is $V(t)=C_0[\,a/\nu_L-1/(2\nu_L^2)-(aJ_2-J_3)\,]$ with $J_2=e^{-\beta_L}/\nu_L-\sqrt{\pi bt}\,\mathrm{erfc}\sqrt{\beta_L}$ and $J_3=\frac{1}{2}(e^{-\beta_L}/\nu_L^2-bt\,E_1(\beta_L))$, so the cell weights $(V(t_{bot})-V(t_{top}))/\Delta z$ sum to the surface tail exactly; the cascade credit is deposited in the uppermost cell (its carrier waves have deposition scales $g/2\sigma^2$ of centimetres).

Dissipation energy flux to the ocean. The term $\int\sigma S_{out}\,d\boldsymbol{k}$ of the wave energy budget Eq. (57) is the energy the breaking waves hand to near-surface turbulence. It is diagnosed locally as

$$\Phi_{oc}(\boldsymbol{x})=-\,g\int\dot{E}_{out}\,d\nu\,d\theta\;\geq\;0 \tag{104}$$

(kinematic; $\rho_w\Phi_{oc}$ in $\mathrm{W\,m^{-2}}$; WAM's PHIOC), computed from the same stored dissipation rate γ_{out} and with the same spectral measures as the wave stress, and written to the coupling output both as the local field and as its domain integral in the energy-diagnostic record. Under `tail_flux = 'wamdi'` (the default) Φ_{oc} contains no tail contribution and

covers the model frequency bins only. Only `tail_flux = 'input'` adds the analytic tail integral I_Φ and the nonlinear cascade credit. The field is the surface boundary condition for the flux F of the turbulent kinetic energy (TKE) closure following Gaspar et al. (1990) in the ocean component ($\rho_w \Phi_{oc}$ as the surface TKE flux, in place of the $u_\star^3$-based parameterisation), which routes this term into the ocean's turbulent kinetic energy budget and thereby closes the coupled energy cycle. The TKE closure receives, besides the usual shear production by the Eulerian mean flow, a shear production term related to the mean Stokes velocity, $c_u K(\partial_z \bar{\boldsymbol{u}}) \cdot \partial_z \bar{\boldsymbol{u}}_S$, resulting from the down-gradient closure of the eddy stresses; the corresponding energy is exchanged with the mean Lagrangian kinetic energy.

Vertical friction and mixing. For consistency with the TKE closure, the vertical friction operator acts on the Eulerian mean velocity, $\partial_z A_v \partial_z (\boldsymbol{u}_L - \boldsymbol{u}_S) = \partial_z A_v \partial_z \boldsymbol{u}$: the work of this viscous term against $\boldsymbol{u}_L$ then exactly matches the shear production terms of the TKE closure, including the Stokes-shear production. Mixing the Lagrangian instead of the Eulerian momentum would break this energy consistency.

Coupling frequency and the exchange term. The two models exchange fields once per coupling interval Δt_c, chosen as a common multiple of the two time steps: the component with the smaller time step sub-cycles $\Delta t_c / \Delta t$ steps between exchanges. When the wave step is the smaller one, $\Delta t_c = \Delta t_{\text{ocean}}$ and the wave model sub-cycles; when $\Delta t_{\text{ocean}} < \Delta t_{\text{wave}}$ (e.g. a non-hydrostatic or very high resolution ocean) the roles reverse and the ocean sub-cycles, with $\Delta t_c = \Delta t_{\text{wave}}$. In either case all coupling fields are piecewise constant over the coupling interval, so the exchange work $C = \int \boldsymbol{u}_L \cdot \partial_t \boldsymbol{u}_S \, dV$ is evaluated with $\partial_t \boldsymbol{u}_S$ as the difference quotient across the coupling update, i.e. over Δt_c rather than over the time step of either component. The domain-integrated energy diagnostics — wind input to the waves $g \int \gamma_{\text{in}} E \, d\Omega \, dA$, the stress work on the ocean (computed with the identical stencils as the applied forcing), the exchange term C, and the vortex-force work — are written to the coupling output at every coupling step, so the discrete energy pathway of the coupled system is monitored throughout the integration rather than assumed. The choice of Δt_c is constrained by the physics rather than by stability: the coupling fields should be well sampled, i.e. Δt_c small compared to the evolution time scales of the exchanged quantities (translation of the wind systems across a grid cell for the stress and Stokes fields, and the evolution of the resolved ocean currents for the refraction terms).

Propagation, boundary conditions, land mask

Action is propagated since it avoids to calculate the energy exchange with the mean flow explicitely. Advection uses the second-order scheme with Superbee limiter (`tracer_flux_superbee`) taken from MITgcm. A first-order upwind option is also available (`tracer_flux_upwind`, selected by the attribute "f_limiter"="upwind"); it shares the identical finite-volume (face-flux) structure — the same land masking, halo exchange, closed frequency faces and reflection — so conservation and the energy fixer of Eq. (91) still apply, but the flux limiter is set to zero. Upwind is more diffusive (it smears sharp swell fronts) but robust to grid-scale noise. The refraction drift velocities $\dot{\nu}$ and $\dot{\theta}$ can optionally be clipped to a CFL bound (attribute `limit_fdot`, default off, with the bound set by `refraction_cfl`, default 0.5):

strong current shear can otherwise drive the spectral advection beyond its stability limit locally. The clipping breaks the exact ray equations where it engages and is meant as a safeguard, not a substitute for a proper time step.

Lateral boundary conditions in x and y are periodic (attribute "periodic_in_x"/"periodic_in_y"), closed, reflective, or open. Boundary conditions in frequency are always closed, in direction always periodic. Lateral reflection boundary conditions at coast lines are implemented the same way as in the IDEMIX2 in pyOM2. These are activated by setting the attribute "enable_reflection" to true value'. If undefined or false, the default of closed boundary are used where energy may pile up near the boundary. Open (radiating) coasts are activated by the attribute "open_coasts". A coastal face lies between a wet cell and a land cell, which includes the land ring of a closed outer boundary. On such a face the limited flux is replaced by the first-order upwind flux. Where the velocity points towards land, the upwind cell is the wet cell and action leaves the domain. Where it points away from land, the upwind cell is the land cell, where the action is zero, and nothing enters. "open_coasts" takes precedence over "enable_reflection". The outflow is booked with the same Adams–Bashforth weights that step the action. The method `coastal_loss()` returns the action, the intrinsic energy σA and the absolute energy ΩA that have left the domain since the start of the run. Land mask is set in the template method "set_topography" in `./pyOM3/wim_simple/main.py`.

Time stepping scheme

Each time step is split into an explicit propagation substep for the wave action A followed by an implicit source substep for the wave energy $E = \sigma A$. The propagation tendency $\dot{A}|_{\text{prop}}$ (the sum of the flux divergences in x, y, ν and θ) is advanced with a multi-step Adams–Bashforth scheme,

$$A^{n+1} = A^n + \Delta t \left(c_A \, \dot{A}^{\,n}|_{\text{prop}} + c_B \, \dot{A}^{\,n-1}|_{\text{prop}} + c_C \, \dot{A}^{\,n-2}|_{\text{prop}} \right). \tag{105}$$

The scheme used is the stabilised second- or third-order Adams–Bashforth pair

$$(c_A, c_B, c_C) = \left(\frac{3}{2} + \epsilon, \; -\frac{1}{2} - \epsilon, \; 0 \right), \qquad \epsilon \approx 0.05, \quad \text{for second order} \tag{106}$$

$$(c_A, c_B, c_C) = \left(\frac{23}{12}, \; -\frac{16}{12}, \; \frac{5}{12} \right) \quad \text{for third order} \tag{107}$$

both started with a forward Euler step $(1, 0, 0)$. The time loop uses the AB2+ϵ pair; the third-order coefficients are given for reference but are deliberately not used (see below). The recent tendencies are held in a rotating three-level buffer (the pointers "tau", "taum1", "taum2"). Only the propagation is integrated this way; the source terms are treated separately below. After the propagation update the action is subject to the lateral boundary conditions and, on land, is masked to zero.

The choice of the time scheme for the propagation is more delicate than usual because the advection uses a flux *limiter*. TVD limiters (Superbee, upwind) guarantee a monotone,

non-oscillatory update — in particular $A \geq 0$ — only when the limited flux is applied in a *single forward Euler step* (or a strong-stability-preserving Runge–Kutta combination of such steps). Multi-step Adams–Bashforth schemes combine tendencies from several time levels with *negative* weights, which destroys this guarantee at any CFL number: each stored tendency is limited with respect to its own state, but the negatively-weighted combination is free to over- and undershoot. For the third-order scheme (23/12, −16/12, 5/12) the negative weight is large and the violation is severe: a smooth, sign-definite action field develops undershoots ($A < 0$) in an alternating grid-scale ($2\Delta x$) checkerboard pattern at sharp fronts (storm edges, swell fronts). Unlike in the ocean model — where the same combination is harmless because tracers are not sign-definite, explicit (biharmonic and vertical) diffusion damps grid-scale noise every step, and no term amplifies small wiggles — the wave model has no explicit diffusion, its action must remain non-negative, and the source terms amplify any ± pattern strongly non-linearly (through $\alpha = \bar{E}\,\bar{\sigma}^4$ and the cubic DIA transfer), so the checkerboard grows over $\mathcal{O}(1)$ day until floating-point overflow. The AB2+ϵ coefficients Eq. (106) reduce the negative weight to $-(1/2 + \epsilon)$, which in practice (CFL $\lesssim 0.25$) is small enough for the limiter's built-in slack to absorb the violation: the action remains non-negative to machine precision in long integrations where AB3 fails within a day. This is an empirical rather than a rigorous property; forward Euler $(1, 0, 0)$ or an SSP Runge–Kutta scheme restores the formal TVD guarantee at the cost of temporal accuracy or an additional flux evaluation per step.

Two safeguards back this up. First, a positivity floor (attribute `floor_action`, default on) clips the action at zero after each propagation step, as is standard in WAM. The clipped mass — the action the floor removes, which equals the spurious action it injects and hence the energy-consistency cost of the clip — is measured *before* clipping and a warning is issued whenever it exceeds round-off scale relative to the total action, so a silently active floor cannot mask a genuine positivity violation; switching the floor off restores the raw scheme. With the numpy backend this check is made at every step. Under JAX it is made only every `floor_check_steps` steps (default 100), with the clip mass of that step, since the comparison requires a transfer from the device to the host. Second, $\min A$ is monitored automatically every $O(100)$ steps against a relative round-off threshold and a warning names the remedy (larger ϵ or forward Euler): any limiter/time-scheme violation shows up there long before a blow-up.

Writing the source terms as growth rates acting on the energy, the source substep integrates $\partial_t E = S$ with $S = \dot{E}_{nl} + (\gamma_{in} + \gamma_{out})E$, where $\dot{E}_{nl}$ is the non-linear (DIA) transfer. A fully explicit update would require a very small time step because the growth rate increases with frequency (this is also the reason for the high-frequency cut-off). Instead a linearised-implicit (semi-implicit) step is used: linearising S about the current state, $S(E^{n+1}) \approx S(E^n) + \partial_E S\,(E^{n+1} - E^n)$, and solving for E^{n+1} gives

$$E^{n+1} = E^n + \frac{S\,\Delta t}{1 - \Delta t\,\partial_E S}, \qquad \partial_E S = \gamma_{in} + \gamma_{out} + \partial_E \dot{E}_{nl}, \tag{108}$$

where the diagonal of the non-linear functional derivative $\partial_E \dot{E}_{nl}$ is returned by the DIA backend alongside the transfer itself. This is unconditionally stable for a purely dissi-

pative feedback ($\partial_E S < 0$), for which the denominator exceeds unity and the update is damped. The growth can additionally be bounded as in WAM by a growth limiter (attribute `limit_growth`, default off) that caps S (not $\partial_E S$) so that the energy cannot grow by more than a prescribed fraction per step,

$$S \leftarrow \min\!\big(S,\ 10^{-7}\, g\, \max(u_\star,\, u_\star^{\min})\, \bar{\sigma} \,/\, (2\pi\nu^4)\big)\,, \qquad u_\star^{\min} = 0.01\ \mathrm{m\,s^{-1}}. \tag{109}$$

The friction-velocity floor $u_\star^{\min}$ in the cap is essential and easy to overlook: without it, the cap vanishes wherever the wind vanishes and the limiter degenerates to $S \leftarrow \min(S, 0)$ in calm cells. This clips only the *positive* lobes of the nonlinear transfer, destroying the conservative $\pm$ structure of S_{nl} exactly there, and acts as a small but systematic negative-action source. In a calm region that accumulates swell without a sink (e.g. a walled basin with reflective coasts outside the wind band), the resulting violation grows linearly in time — independent of the time step and of the time-stepping scheme — until, after $O(100)$ days, the nonlinear source terms of the accumulated spectrum activate and the integration fails abruptly. The floor keeps the cap strictly positive everywhere while leaving the physics unchanged: $u_\star^{\min}$ is far below the input threshold of the slowest resolved wave ($c \approx 28\, u_\star$ requires $u_\star \gtrsim 0.06\ \mathrm{m\,s^{-1}}$ for the highest resolved frequency), so the wind input remains identically zero in floored cells; only the growth cap becomes a finite positive bound instead of zero. Because the limiter bounds only the numerator, the denominator $1 - \Delta t\, \partial_E S$ can still vanish or change sign where $\partial_E S$ is large and positive — most notably at high frequency during spin-up from a small seed, where the DIA derivative is steep. When this happens the update overshoots and the highest bins become unstable. Two safeguards are used: the high-frequency cut-off removes the steepest bins from the prognostic integration (replacing them by the diagnostic ν^{-5} tail), and, optionally, only the dissipative part of the feedback is kept in the implicit denominator,

$$\partial_E S \ \rightarrow\ \min(\partial_E S,\, 0)\,, \tag{110}$$

guaranteeing $1 - \Delta t\, \partial_E S \geq 1$. This is activated by setting the attribute `implicit_dissipation_only` to `True` and equals the standard WAM/WW3 construction — the stabilising dissipative feedback is treated implicitly while the growth is treated explicitly and bounded by the limiter. It leaves the equilibrium ($S = 0$) unchanged and therefore does not affect the energy balance; it only alters the transient integration of the growth phase.

After the source substep the energy is masked to zero on land. If `mask_energy` is set, the high-frequency tail is then re-imposed through `imphftail`. Finally the action is recovered as $A = E/\sigma$ for the next propagation step.

The time step of the propagation is limited by the advective CFL number based on the fastest group velocity, i.e. the lowest resolved frequency (Superbee is stable to CFL $\lesssim 0.5$, upwind to CFL < 1; in practice CFL $\approx$ 0.2–0.25 is used, which is also the regime in which the AB2+ϵ scheme preserves the non-negativity of the action, see the time-stepping section). Note that the grid-scale checkerboard described there is *not* a CFL violation — it appears at any Courant number when a multi-step scheme with a large negative weight (AB3) is combined with the flux limiter — so it cannot be cured by reducing the time

step; the cure is the time scheme. Substantially larger time steps than the advective CFL allows would require an implicit or semi-Lagrangian treatment of propagation, at the cost — for plain semi-Lagrangian — of the exact (face-flux) energy conservation.

The JAX backend

pyOM3 runs either with numpy, in parallel with MPI, or with JAX. The backend is chosen when pyOM3 is imported. The environment variable `PYOM3_BACKEND=jax` selects JAX also when mpi4py is installed. The JAX backend is meant for GPUs, with one process per GPU. The JAX results agree with the numpy results to round-off. The test `pyOM3/test/wim/jax_sharding.py` checks this to 10^{-12} for one, two and four shards. Notes on running on GPUs are collected in `pyOM3/README_jax_gpu.md`.

Domain decomposition. Under JAX the two horizontal axes y and x of all arrays are sharded over $n_{pes,x} \times n_{pes,y}$ devices (`decomposition_jax.py`). The spectral axes of direction and frequency are never sharded. Every shard carries its own halo ring. The global padded sizes are therefore $N_x = n_x + 2\,h\,n_{pes,x}$ and $N_y = n_y + 2\,h\,n_{pes,y}$, with h the number of halo points. A slice that removes the halo on the y or x axis therefore removes only the outer halo. Operations on the interior cells use the interior mask `maskI` instead. The halo exchange between neighbouring shards is done with `ppermute`.

Compiled time step. A time step consists of three compiled (jitted) parts. The first is the propagation (`_mk_advection` and `_add_propagation`). The second is the Adams–Bashforth update with the outflow booking and the positivity floor (`_ab_update`). The third is the source step (`_source_update`). The model instance is a static argument of these parts. Attributes read inside them are therefore constants of the compiled program. Fields that change in time, the wind and the ocean currents, are passed as arguments. The sharded constants of the run, such as the masks, are passed as the argument `consts`. The results are returned and written to the attributes outside the compiled parts. The check of the positivity floor needs a transfer from the device to the host. It is therefore made only every `floor_check_steps` steps (default 100).

Propagation. Under JAX the propagation uses the array form (`propagation_backend` = 'numpy'). The Fortran kernels need numpy arrays. Specular reflection at coasts uses a vectorised form of the reflection of the fluxes instead of the loop over the coastal cells. The compiled propagation contains optimisation barriers (`jax.lax.optimization_barrier`). They separate the flux scheme, the energy fixer, the coastal loss sums and the flux divergence. XLA may not fuse the pieces across a barrier. Without the barriers XLA fused all pieces into one program. For 384×384 cells and 24×25 spectral bins on four GPUs this program took minutes per step, while each piece alone takes milliseconds. With numpy the barriers do nothing.

Nonlinear transfer. Under JAX the DIA is evaluated by `snonlin_jax.py`. It has the same arithmetic as the WAM loop of `snonlin_np.py` and `Snonlin.f90`, but no loop and no scatter. The WAM loop runs over the centre frequency index M_C and the two mirror halves $K_H \in \{0, 1\}$ of the quadruplet. In the vectorised form these two loops become one group axis $g = 2(M_C - 1) + K_H$. There are $G = 2M_{LSTHG}$ groups, with M_{LSTHG} the

number of centre frequencies of WAM. The spectral values of all quadruplets are collected with index tables of shape (G, n_d). The quadruplet arithmetic then runs on arrays of shape (n_x, n_y, G, n_d). Each contribution is placed in its destination bin in two steps. First the directions are permuted into their destination direction bins. Then a matrix product over the group axis with a weight matrix of shape (G, n_ν) adds it to its destination frequency. The weight matrices also hold the validity masks of the interior, high- and low-frequency branches of the WAM loop. All tables are built once with numpy at the setup. Without a scatter the result is deterministic on the GPU. The sums over the groups run in a different order than in the loop. The result therefore agrees with the Fortran and numpy versions to round-off. For a spectrum with 25 frequencies and 24 directions the largest relative difference was $8 \cdot 10^{-16}$ in S_{nl} and $6 \cdot 10^{-16}$ in its derivative. On a CPU the loop forms are faster, since their slices per group stay in the cache. Under JAX the DIA is evaluated on all horizontal columns including the halo columns. In the shard layout a slice does not give the interior, and the DIA acts on each column separately. The exact scattering integral (`snonlin_backend` = 'exact') is a numpy code and cannot be used under JAX.

Model parameters and options

All parameters and options are attributes of the model instance. They are set either through the parameter dictionary passed at construction or by assigning the attribute on the instance before the model is stepped; options marked with a default may simply be left unset. The tables below list every attribute the wave model reads, with its default where one exists.

Required parameters (no default)

Attribute	Unit	Meaning
`nx`, `ny`	—	global horizontal grid size (cells, without halo points)
`dx`, `dy`	m	horizontal grid spacing
`nf`	—	number of frequency bins (local on every PE)
`nd`	—	number of direction bins (local on every PE)
`f0`	1/s	lowest frequency of the geometric frequency grid (WAM value 0.035)
`df0`	—	relative frequency increment $\Delta\nu/\nu$ of the geometric grid (WAM value 0.1)
`dt`	s	time step
`periodic_in_x`	—	lateral boundary in x: periodic if true, closed if false (the outermost ring of cells is then set to land)
`periodic_in_y`	—	as above for y

The constructor additionally takes the keyword arguments `n_pes_x`, `n_pes_y` (processor grid, default 1×1; their product must equal the communicator size), `halo_points` (default 1) and `comm_in` (default: the MPI world communicator). Under JAX the product of `n_pes_x` and `n_pes_y` must not exceed the number of devices, and `nx` and `ny` must be divisible by `n_pes_x` and `n_pes_y`.

Forcing and coupling fields

Attribute	Default	Meaning
`ustar`	required	friction velocity magnitude $u_\star$. A scalar is accepted and broadcast only in the default configuration. With `limit_growth` or `tail_flux` = `'input'` a horizontal field including the halo points, of shape `(Ny,Nx)`, is required
`udir`	required	wind direction θ^* (radians), scalar or field. With `tail_flux` = `'input'` a field of shape `(Ny,Nx)` including the halo points is required
`ocean_U`, `ocean_V`	unset	Doppler velocity $\boldsymbol{U}(\nu)$ provided by the coupler. If unset, all current effects (advection by currents, refraction, the energy fixer) are dormant
`ocean_Udf`, `ocean_Vdf`	0	frequency derivative $\partial_\nu \boldsymbol{U}$ of the Doppler velocity (radial group-velocity correction R and frequency refraction)

Options with defaults

Attribute	Default	Meaning
`snonlin_backend`	`'fortran'`	S_{nl} implementation. The DIA versions are `'fortran'` (original WAM code, double precision only), `'jax'` and `'numpy'`, which agree to round-off. `'exact'` selects the reference scattering integral. The default is `'fortran'` with the numpy backend and `'jax'` with the JAX backend. Under JAX `'numpy'` is replaced by `'jax'`, and `'fortran'` raises an error

Attribute	Default	Meaning
`propagation_backend`	`'fortran'`	propagation kernels. `'fortran'` uses the f2py kernels in `wim_simple/Propagation/` (double precision only), `'numpy'` the array form. Both give bit-identical results. The default is `'fortran'` with the numpy backend and `'numpy'` with the JAX backend, under which `'fortran'` raises an error
`grav`	9.81	gravitational acceleration
`rho_o`	1024	reference water density. Currently unused inside the wave model: the air–water density ratio of the source terms is fixed at 1.225/1000
`mask_energy`	off	prognostic cut-off ν_c per column with the imposed ν^{-5} diagnostic tail (`imphftail`); prerequisite for the equilibrium booking of `tail_flux = 'input'`
`limit_growth`	off	WAM growth limiter, Eq. (109)
`implicit_dissipation_only`	off	keep only the dissipative part of the source feedback in the implicit denominator, Eq. (110)
`floor_action`	on	positivity floor $A \leftarrow \max(A, 0)$ after the propagation step, with the clipped mass monitored and warned about
`floor_check_steps`	100	JAX only. Interval in time steps at which the clipped mass of the positivity floor is checked (with numpy at every step)
`f_limiter`	`'superbee'`	advection flux limiter: `'superbee'` (second order) or `'upwind'` (first order, diffusive, robust)
`energy_consistent`	off	conservative energy fixer of the propagation operator, Eq. (91)
`enable_reflection`	off	specular reflection of the advective fluxes at coastlines
`open_coasts`	off	radiating coasts. Every face between a wet and a land cell carries the first-order upwind flux, so action leaves the domain and nothing enters from land. Takes precedence over `enable_reflection`. The outflow is booked, and `coastal_loss()` returns the action, intrinsic energy and absolute energy lost since the start. These faces are left out of the defect of the energy fixer

Attribute	Default	Meaning
`limit_fdot`	off	clip the refraction velocities $\dot{\nu}, \dot{\theta}$ to a CFL bound
`refraction_cfl`	0.5	the CFL bound used by `limit_fdot`
`closed_bottom`	off	extend the bottom cell of the coupling kernel to $z \to -\infty$ (exact barotropic projection, no momentum lost below the column floor)
`tail_flux`	`'wamdi'`	booking of the diagnostic-tail fluxes to the ocean: `'wamdi'` (dissipation-rate continuation) or `'input'` (equilibrium-range γ_{in} integral plus nonlinear cascade credit)
`phioc_kernel`	`'stokes'`	vertical kernel of `calc_phioc_work`, which distributes Φ_{oc} of each model frequency bin over the ocean cells. `'stokes'` uses the shared kernel $Q = 2ke^{2kz}$ of Eq. (86). `'orbital'` uses ke^{kz}, with twice the decay depth. The tail and cascade credit of `tail_flux = 'input'` go into the top cell

Snapshot diagnostics

The netCDF snapshot class `snap_nc4` reads the attribute `snapint` (snapshot interval in seconds, required) and takes the setup arguments `snap_file_name` (default `'snap.cdf'`) and `show_each_time_step` (default on). The alternative class `snap_raw` writes the block of every PE as `.npy` files in float32 into a directory, without gathering. It reads `snapint` as well, and its `snap_file_name` is the directory name (default `'raw_snap'`). The attribute `snap_fields` is a tuple of the field names to write. By default it contains `E_mean`, `om_mean`, `Hs`, `E_f` and `E_d`, where `E_f` and `E_d` are the spectrum integrated over direction and over frequency. Any other 2-D or 4-D attribute of the model may be named as well. The full spectrum `E` is added on those snapshot steps that fall on a multiple of `snapint_full` seconds (default 0, which means never). The land mask is defined by overloading the template method `set_topography`; the default is an all-ocean domain, with closed outer boundaries generated automatically from `periodic_in_x`/`periodic_in_y`.

The spectral wave model wim_realistic

wim_realistic is wim_simple on the grid of pe_realistic. This text describes only the differences to wim_simple. Everything that is not mentioned here is as described in the documentation of wim_simple.

The differences are the following. The horizontal grid spacings may vary. The horizontal coordinates may be longitude and latitude on the sphere. The propagation then contains the metric factors and the turning of the rays on the sphere. The coupling to the ocean allows a vertical grid with varying cell thickness. The wind input of Janssen and the dissipation of WAM4 are available as an option.

Most methods of wim_realistic are those of wim_simple. They are inherited or assigned from wim_simple. wim_simple itself is not changed by wim_realistic. The source terms of wim_simple, the nonlinear transfer, the time stepping, the reflection, the open coasts and the JAX backend are therefore the same in both models.

Grid

The model domain consists of $n_x \times n_y$ cells. The horizontal coordinates x and y are either Cartesian coordinates in metres or longitude λ and latitude ϕ in degrees. The attribute `coord_degree` selects longitude and latitude. The spectral grid in frequency ν and direction θ is that of wim_simple.

The widths of the cells are given by the attributes `dxt` and `dyt`. Each of them is either a number or an array of length n_x or n_y without halo points. The numbers `dx` and `dy` of wim_simple are accepted instead. With `coord_degree` the widths are given in degrees.

The horizontal grid is built as in pe_realistic. A cell has its centre at a T point and its faces at U points. In x the cell i has the centre x_i^t, the width Δx_i^t and the eastern face x_i^u. The distance between the centres x_i^t and x_{i+1}^t is Δx_i^u. The same holds in y. The faces and centres follow from the widths by the recursion of pyOM2,

$$x_i^u = x_{i-1}^u + \Delta x_i^t \ , \quad x_{i+1}^t = 2x_i^u - x_i^t \ , \quad \Delta x_i^u = x_{i+1}^t - x_i^t \tag{111}$$

The first interior U point is placed at `x_origin` and `y_origin` (default 0).

The recursion Eq. (111) can give $\Delta x^u \leq 0$ when the widths jump. This holds also across the seam of a periodic direction. The widths must therefore vary smoothly.

With `coord_degree` the widths are converted to metres after the coordinates are built. The factor is $a\pi/180$ with the radius of the earth $a = 6370\,\mathrm{km}$ (attribute `radius`). The metric factors are $\cos\phi$ at the T rows, $\cos\phi$ at the U rows and $\tan\phi/a$ at the T rows. In Cartesian coordinates they are 1, 1 and 0. The area of a cell is $A^t = \cos\phi^t \Delta x^t \Delta y^t$ with Δx and Δy in metres.

The attributes `dx` and `dy` are set to None after the grid is built. Code from wim_simple that still uses them therefore stops with an error.

The grid has no treatment of the poles and no reduced grid. The domain must therefore end before the poles. The zonal width $\cos\phi\,\Delta x^t$ of a cell becomes small at high latitude.

The Courant number in x then grows with $1/\cos\phi$, and the time step has to be chosen accordingly.

Propagation

On the sphere the action equation of wim_simple becomes

$$\partial_t A + \frac{1}{a\cos\phi}\left(\partial_\lambda(\dot{x}A) + \partial_\phi(\cos\phi\,\dot{y}A)\right) + \partial_\theta(\dot{\theta}A) + \partial_\nu(\dot{\nu}A) = S \tag{112}$$

with λ and ϕ in radians. The velocities $\dot{x}$ and $\dot{y}$ are the eastward and northward velocities in m/s. They are those of wim_simple, $\dot{\boldsymbol{x}} = (c/2)\boldsymbol{s} + \boldsymbol{U} + R\boldsymbol{s}$. The direction θ is counted from north, with $\boldsymbol{s} = (\sin\theta, \cos\theta)$ as in wim_simple.

The discrete form is the flux form of wim_simple. The flux in y is multiplied by $\cos\phi^u$. The flux divergence is divided by $\cos\phi^t\Delta x^t$ and $\cos\phi^t\Delta y^t$, with the widths in metres. The fluxes in frequency and direction are those of wim_simple. The Courant numbers of the Superbee scheme are $|\dot{x}|\Delta t/(\cos\phi^t\Delta x^t)$ and $|\cos\phi^u\dot{y}|\Delta t/(\cos\phi^t\Delta y^t)$.

The refraction by the Doppler velocity $\boldsymbol{U}$ needs the horizontal gradients of $\boldsymbol{U}$. On the sphere these are $(a\cos\phi)^{-1}\partial_\lambda$ and $a^{-1}\partial_\phi$. The discrete differences of wim_simple are therefore divided by $\cos\phi^t\Delta x^u$ and Δy^u instead of Δx and Δy.

On the sphere a ray that keeps its direction relative to north does not follow a great circle. The direction therefore changes along the ray even without currents. With currents the frequency changes as well. Both effects add terms to $\dot{\theta}$ and $\dot{\sigma}$ of wim_simple,

$$\dot{\theta} = \dot{\theta}_{simple} + (c/2 + \boldsymbol{s}\cdot\boldsymbol{U})\sin\theta\frac{\tan\phi}{a} \tag{113}$$

$$\dot{\sigma} = \dot{\sigma}_{simple} - \frac{\sigma}{2}(\boldsymbol{U}\cdot\boldsymbol{s}_\perp)\sin\theta\frac{\tan\phi}{a} \tag{114}$$

with $\boldsymbol{s}_\perp = (\cos\theta, -\sin\theta)$ and $\dot{\nu} = \dot{\sigma}/(2\pi)$. Here $\dot{\theta}_{simple}$ and $\dot{\sigma}_{simple}$ are the expressions of wim_simple with the gradients on the sphere. The added term of $\dot{\theta}$ is evaluated at the faces in direction, as $\dot{\theta}$ of wim_simple. The added term of $\dot{\sigma}$ is evaluated at the centres in direction.

Without currents only the term $(c/2)\sin\theta\tan\phi/a$ remains. It turns the ray onto the great circle. The frequency is then constant along the ray.

The two terms follow exactly from the dispersion relation in spherical coordinates. The derivation is checked symbolically in `development/derive_spherical_rays.py`. The documentation of ECMWF (IFS Part VII, Eq. 2.25c) keeps only the term $(c/2)\sin\theta\tan\phi/a$. The terms with $\boldsymbol{U}$ are of the order $|\boldsymbol{U}|\tan\phi/a$. This is about $2\cdot10^{-7}\,\mathrm{s}^{-1}$ at 60° latitude for $|\boldsymbol{U}| = 1\,\mathrm{m/s}$.

The option `limit_fdot` of wim_simple also bounds the added terms.

The energy fixer is that of wim_simple. The defect D of wim_simple is here the integral over the area of the cells. The terms of the faces in x are multiplied by Δy^t. The terms of the faces in y are multiplied by Δx^t, since the flux already carries $\cos\phi^u$. The terms

of the faces in frequency and direction are multiplied by A^t. On a uniform Cartesian grid this reduces to the fixer of wim_simple.

The open coasts are those of wim_simple. The outflow through a face in x is weighted with Δy^t and through a face in y with Δx^t.

The Fortran kernels of the propagation (`propagation_backend` = 'fortran') are those of wim_simple. The metric factors and the added terms above are handed to them. They give the same result as the array form.

Coupling to an ocean with varying cell thickness

The coupling of wim_simple assumes ocean cells of constant thickness Δz. pe_realistic has cells of varying thickness Δz^t. wim_realistic therefore accepts Δz^t as an array in the coupling.

The Stokes velocity, the stress and the Doppler velocity of wim_simple use the kernel $Q = 2ke^{2kz}$ integrated over each ocean cell,

$$\int_{z_b}^{z_t} Q\, dz = e^{2kz_t} - e^{2kz_b}$$

with the top face z_t and the bottom face z_b of the cell. For an array Δz^t the faces are the partial sums of Δz^t from the surface downward. The array is given from the bottom to the surface, as in pe_realistic. The top face of a cell is then exactly the bottom face of the cell above. The sum of the kernel over the column is therefore exactly $1 - e^{-2kH}$ for a column of depth H. With `closed_bottom` the lowest cell also takes the rest e^{-2kH}, so that the sum is exactly 1, as in wim_simple.

`calc_stokes` and `calc_stress_work` divide the integrated kernel by the thickness Δz^t of each cell. The Doppler velocities need no division and are those of wim_simple. With a number Δz the arithmetic is that of wim_simple, bit for bit.

`calc_phioc_work` of wim_simple is not generalised. It works only with a number Δz.

Wind input of Janssen and dissipation of WAM4

The attribute `input_scheme` selects the source terms. The default 'wamdi' uses the input and dissipation of wim_simple. The value 'janssen' uses the wind input of Janssen (1991) and the dissipation of WAM4 (Bidlot et al. 2007). These are the source terms of ecWAM for `IPHYS` = 0. They are ported from the routines `sinput_jan`, `sdissip_jan`, `stresso`, `tau_phi_hf`, `z0wave`, `fkmean`, `femeanws` and `frcutindex` of ecWAM. The nonlinear transfer is that of wim_simple in both cases.

The friction velocity u_* and its direction θ_u are prescribed, as in wim_simple. ecWAM computes u_* from the wind speed and the roughness. This part of ecWAM is not ported.

The input is $S_{in} = \gamma_{in} E$ with the growth rate

$$\gamma_{in} = \frac{\beta_{max}}{\kappa^2} \frac{\rho_a}{\rho_w} \sigma\, x^2 \mu\, (\ln \mu)^4 \tag{115}$$

Here $x = (u_*/c + z_\alpha)\cos(\theta - \theta_u)$ and $\ln\mu = \ln(kz_0) + \kappa/x$. $\kappa = 0.4$ is the constant of von Kármán. The input is zero where $\cos(\theta - \theta_u) \le 0.01$ or $\ln\mu \ge 0$. The cells with input define the wind sea used below.

The roughness length depends on the momentum that the waves take from the wind,

$$z_0 = \frac{\alpha}{g}\frac{u_*^3}{\sqrt{u_*^2 - \tau_w}} \tag{116}$$

with the constant α of Charnock. The momentum flux τ_w into the waves is divided by the density of air ρ_a. It is the input of momentum over all frequency bins plus the analytic tail described below,

$$\tau_w = \frac{\rho_w}{\rho_a}\left|\int \gamma_{in} E\,\sigma\,\boldsymbol{s}\,d\theta\,d\nu + \boldsymbol{\tau}_{tail}\right|$$

It is limited to $u_*^2/(1+\epsilon_1)$. Without waves $\tau_w = 0$ and $z_0 = \alpha u_*^2/g$ is the roughness of Charnock.

z_0 and τ_w depend on each other. In each source step z_0, γ_{in} and τ_w are therefore computed `janssen_ncall` times in turn, as ecWAM does. The first pass starts from τ_w of the previous step.

The dissipation is $S_{out} = \gamma_d E$ with

$$\gamma_d = c_{dis}\,2\pi\nu_1\,E_{tot}^2\,\bar{k}^4\,X\,((1-\delta) + \delta X)\ ,\quad X = k/\bar{k} \tag{117}$$

It does not depend on direction. E_{tot} is the total energy, ν_1 the mean frequency weighted with ν and $\bar{k}$ the mean wavenumber from the moment with $\sqrt{k}$.

The means are sums over the frequency bins plus an analytic tail $E_N(\nu_N/\nu)^5$ beyond the last bin ν_N,

$$\begin{aligned}
E_{tot} &= \sum E_d\Delta\nu + \nu_N E_N/4 \\
\nu_1 &= \left(\sum \nu E_d\Delta\nu + \nu_N^2 E_N/3\right)/E_{tot} \\
\bar{k} &= \left(\left(\sum \sqrt{k}\,E_d\Delta\nu + \frac{2\pi}{\sqrt{g}}\nu_N^2 E_N/3\right)/E_{tot}\right)^2
\end{aligned}$$

with $E_d = \int E\,d\theta$ and E_N of the last bin.

The prognostic range ends at $2.5\max(\bar{\nu}, \bar{\nu}_{ws})$, at the nearest bin in $\ln\nu$. Here $\bar{\nu}$ is the mean frequency weighted with $1/\nu$ and $\bar{\nu}_{ws}$ the same mean over the wind sea. The lowest frequency bin is always prognostic. The term of Pierson and Moskowitz in the cut-off of wim_simple is not used, as in ecWAM.

Beyond the prognostic range the spectrum is $E_c(\nu_c/\nu)^5$, as in wim_simple. The stress and the energy flux of this tail are computed as in `tau_phi_hf` of ecWAM. The integral over the tail uses the rule of Simpson with `jtot_tauhf` = 19 points in $\ln(\omega\sqrt{z_0/g})$. It runs from $\max(\omega_c, x_0 g/u_*)$ to $\omega = \sqrt{g/z_0}$. x_0 is the value of u_*/c below which the growth rate

vanishes for the roughness of Charnock. The growth in the tail is the one along the wind times $\cos^2(\theta - \theta_u)$ for $\cos(\theta - \theta_u) > 0$.

The tail of τ_w starts at the last frequency bin of the model. The frequency ghost bin is still zero when the input is computed. The tails of the stress and of Φ_{oc} with `tail_flux` = 'input' start at the ghost bin, as in wim_simple. `calc_stress_work` puts the tail stress into the ocean cells point by point of the rule of Simpson, with the kernel $2k_j e^{2k_j z}$ of each point j.

The port differs from ecWAM in three points. The widths of the first and last frequency bins are those of wim_simple, while ecWAM uses half bins there. This changes the means by 0.05 to 0.5 %. The tail of τ_w starts at the last model bin, as stated above. The tail stress is booked along each direction $\boldsymbol{s}(\theta)$. Its component along the wind is the stress of ecWAM, and ecWAM drops the component across the wind.

The following parts of ecWAM are not ported. These are the gustiness, the negative input, the normalisation of the growth rate, the roughness of gravity-capillary waves, the sheltering and the viscous dissipation. The viscous term is in the code with $c_{disvis} = 0$, as in ecWAM for `IPHYS` $= 0$.

τ_w and z_0 are carried from step to step. They are not part of the restart of the model. A restart starts with $\tau_w = 0$, i.e. with the roughness of Charnock. The iteration of z_0 and τ_w corrects this within the first step.

The port is compared with the routines of ecWAM compiled with f2py. The test is `janssen_ref_test.py` in `development/`. All quantities agree to round-off.

Model parameters and options

The parameters and options of wim_simple are also those of wim_realistic. The tables list only the differences.

Attribute	Default	Meaning
`dxt`, `dyt`	required	widths of the cells, a number or an array without halo. Degrees with `coord_degree`, metres otherwise. Replace `dx` and `dy` of wim_simple, which are still accepted
`coord_degree`	off	longitude and latitude in degrees instead of Cartesian coordinates
`x_origin`, `y_origin`	0	position of the first interior U point
`radius`	6370 km	radius of the earth (m)
`input_scheme`	'wamdi'	source terms. 'wamdi' as in wim_simple, 'janssen' the input of Janssen and the dissipation of WAM4

With `input_scheme` = 'janssen' the following constants are read. Their defaults are those of ecWAM for `IPHYS` $= 0$.

Attribute	Default	Meaning
`betamax`	1.20	β_{max} of the input
`zalp`	0.008	z_α of the input
`alpha_charnock`	0.0065	constant α of Charnock
`kappa`	0.40	constant of von Kármán
`cdis`	-1.33	c_{dis} of the dissipation
`delta_sdis`	0.5	δ of the dissipation
`cdisvis`	0	viscous dissipation
`rnu`	$1.5 \cdot 10^{-6}$	kinematic viscosity (m^2/s) of the viscous dissipation
`eps1`	10^{-5}	ϵ_1 in the limit of τ_w
`tailfactor`	2.5	factor of the prognostic cut-off
`janssen_ncall`	2	number of passes of z_0, γ_{in} and τ_w per step
`jtot_tauhf`	19	number of points of the tail integral
`rho_a`	1.225	density of air (kg/m^3)
`rho_w`	1000	density of water (kg/m^3)

The densities `rho_a` and `rho_w` act only in the source terms of Janssen. The input of wim_simple and the tails of the coupling use 1.225/1000.

Snapshot diagnostics

The snapshot classes are those of wim_simple. They write the coordinates x^t and y^t of the grid, in degrees with `coord_degree`. The fields are those of wim_simple.

Tests

The tests are in `pyOM3/wim_realistic/development/`. `reduction_test.py` compares wim_realistic on a uniform Cartesian grid with wim_simple. The action agrees bit for bit, and to 10^{-15} with the energy fixer. `sphere_test.py` tests the propagation on the sphere. The total action is conserved to 10^{-16} over three days. A wave packet follows the great circle to 0.15° in position and 0.6° in direction after 36° of arc. The error of a uniform state decreases with the second power of the grid spacing. The absolute energy closes to $2 \cdot 10^{-16}$ with a sheared current. `janssen_ref_test.py` compares the source terms of Janssen with ecWAM, see above.

Limits

- The domain must end before the poles.
- The widths of the cells must vary smoothly, also across a periodic seam.
- The terms with $\boldsymbol{U}$ in the turning on the sphere are not tested by a dynamic test.

- `calc_phioc_work` works only with a number Δz.
- With `input_scheme` = 'janssen' and `tail_flux` = 'wamdi' the tails of the stress use the dissipation of wim_simple.
- τ_w and z_0 of the scheme of Janssen are not part of the restart.

The exact scattering integral S_{nl} in wim_simple: formulation, discretisation and validation

This note documents the exact evaluation of the non-linear transfer S_{nl} implemented in the sub-package snl_exact of wim_simple ("snonlin_backend" = 'exact'). It complements the model documentation, which contains only a summary, with the underlying conventions, the derivation chain of the coupling coefficient, the geometry of the resonance loci, the discretisation on the spectral grid, the discrete conservation properties, and the validation against independent implementations. Only deep water is considered.

Conventions and the kinetic equation

The kinetic equation below is written for the wave action spectral density $A(\boldsymbol{k})$ per d^2k. It is related to the surface elevation spectrum $F(\boldsymbol{k})$, with $\langle\zeta^2\rangle = \int F\, d\boldsymbol{k}$, by

$$A \;=\; \frac{g}{\sigma}\,F \quad , \quad \sigma(\boldsymbol{k}) = \sqrt{g|\boldsymbol{k}|} \tag{118}$$

The model carries its spectra as functions of cyclic frequency ν and direction θ. Its spectral variables are the variance density $E(\nu,\theta)$ per Hz and radian and the action E/σ, which is also a density per Hz and radian. This model action is not the density A per d^2k of Eq. (118). The model hands E to the exact backend. With $k = (2\pi\nu)^2/g$ and $dk/d\nu = 8\pi^2\nu/g$ the backend converts E to the action density per d^2k by

$$A(k,\theta) \;=\; \frac{g\,E(\nu,\theta)}{\sigma\,k\,dk/d\nu} \tag{119}$$

and converts the computed transfer back to E with the same factor.

The evolution of A by resonant four-wave interactions is the kinetic equation of Hasselmann (1962), written here in the Zakharov form

$$S_{nl}(\boldsymbol{k}_1) = 4\pi \int T^2_{1234}\,\delta(\boldsymbol{k}_1+\boldsymbol{k}_2-\boldsymbol{k}_3-\boldsymbol{k}_4)\,\delta(\sigma_1+\sigma_2-\sigma_3-\sigma_4)\;\mathcal{B}\;d\boldsymbol{k}_2\,d\boldsymbol{k}_3\,d\boldsymbol{k}_4 \tag{120}$$

with the density bracket

$$\mathcal{B} \;=\; A_3A_4(A_1+A_2) - A_1A_2(A_3+A_4) \tag{121}$$

Two properties of $\mathcal{B}$ are used repeatedly below: it vanishes identically for constant A, and it vanishes on the resonance manifold for the Rayleigh–Jeans equilibrium $A \propto 1/\sigma$, since

$$\mathcal{B}|_{A=c/\sigma} \;=\; c^3\,\frac{(\sigma_1+\sigma_2)-(\sigma_3+\sigma_4)}{\sigma_1\sigma_2\sigma_3\sigma_4} \;=\; 0 \tag{122}$$

Both properties are preserved by the discretisation (see below) and provide sharp correctness tests.

The kernel T_{1234} in Eq. (120) is the canonical four-wave kernel of the Zakharov equation

$$i\,\partial_t b_1 \;=\; \int T_{1234}\, b_2^* b_3 b_4\, \delta(\boldsymbol{k}_1+\boldsymbol{k}_2-\boldsymbol{k}_3-\boldsymbol{k}_4)\, d\boldsymbol{k}_2\, d\boldsymbol{k}_3\, d\boldsymbol{k}_4 \tag{123}$$

in the Fourier convention without 2π prefactors, for which $\langle b(\boldsymbol{k}) b^*(\boldsymbol{k}')\rangle = A(\boldsymbol{k})\,\delta(\boldsymbol{k}-\boldsymbol{k}')$ holds with the physical action density Eq. (118) directly, and $T(\boldsymbol{k},\boldsymbol{k},\boldsymbol{k},\boldsymbol{k}) = k^3$. In the convention of Krasitskii (1994), with a factor $1/2\pi$ per transform, the same kernel reads $T^K = T/(2\pi)^2$ and $T^K(\boldsymbol{k},\boldsymbol{k},\boldsymbol{k},\boldsymbol{k}) = k^3/4\pi^2$; the collision prefactor 4π in Eq. (120) is form-invariant under this change of convention. The combination

$$G \;=\; 4\pi\, T^2 \;=\; 4\pi(2\pi)^4 \left(T^K\right)^2 \tag{124}$$

is the deep-water coupling coefficient of Webb (1978).

Derivation and verification of the kernel

The kernel is not transcribed from the literature but derived from the water-wave Hamiltonian, truncated at fourth order in wave steepness,

$$H_3 \;=\; \frac{1}{2}\int \zeta\left((\boldsymbol{\nabla}\psi)^2 - (G_0\psi)^2\right) d\boldsymbol{x} \tag{125}$$

$$H_4 \;=\; \frac{1}{2}\int \zeta\,(G_0\psi)\left(G_0(\zeta\, G_0\psi) + \zeta\,\nabla^2\psi\right) d\boldsymbol{x} \tag{126}$$

with the surface potential ψ, the flat-surface Dirichlet–Neumann operator G_0 (symbol $|\boldsymbol{k}|$ in deep water), and normal variables defined by

$$\hat{\zeta}(\boldsymbol{k}) = \sqrt{\frac{k}{2\sigma}}\left(a_{\boldsymbol{k}} + a^*_{-\boldsymbol{k}}\right) \quad , \quad \hat{\psi}(\boldsymbol{k}) = -i\sqrt{\frac{\sigma}{2k}}\left(a_{\boldsymbol{k}} - a^*_{-\boldsymbol{k}}\right) \tag{127}$$

(here written in the 2π-free convention). Both truncated Hamiltonians were verified exactly against the analytic kinetic energy of a prescribed harmonic flow, with residuals scaling as the neglected orders ε^3, ε^4, ε^5. This check was made during the development. It is not part of the tests shipped with the sub-package (test_kernel.py, test_locus.py, test_assembly.py and test_physics.py). The canonical (normal-form) transformation then removes the non-resonant cubic terms; on the four-wave resonance manifold the two classes of induced quartic contributions coincide pairwise, which is exploited by keeping both with weight $1/4$ so that the $(12)\leftrightarrow(34)$ symmetry of T is manifest also off the manifold.

The resulting kernel satisfies, at machine precision, the on-shell Hermiticity and index symmetries, rotation invariance, cubic homogeneity $T(\lambda\boldsymbol{k}_i) = \lambda^3 T(\boldsymbol{k}_i)$, and independence of g. It reproduces the two-wave degenerate-quartet interaction coefficients of Longuet-Higgins/Leblanc (as compiled by Stuhlmeier, 2024) to about 10^{-6}. These coefficients are values of T at degenerate quadruplets such as $(\boldsymbol{k},\boldsymbol{k},\boldsymbol{k},\boldsymbol{k})$. There the expression for T contains vanishing difference wavenumbers and cannot be evaluated directly. The tests

therefore evaluate T at nearby quadruplets $(\boldsymbol{k}_i, \boldsymbol{k}_j, \boldsymbol{k}_i + \boldsymbol{d}, \boldsymbol{k}_j - \boldsymbol{d})$ with a small offset $\boldsymbol{d}$ of length 10^{-5} or 10^{-6} times a fixed vector of order one. The agreement of about 10^{-6} is set by this offset, not by the kernel. The deviation decreases linearly with the offset and is $1.2 \cdot 10^{-9}$ for $(\boldsymbol{k}, \boldsymbol{k}, \boldsymbol{k}, \boldsymbol{k})$ at an offset factor of 10^{-8}. During the development the kernel also reproduced the Stokes second-harmonic and mean-flow response and the bichromatic super- and sub-harmonic amplitudes. These checks are not part of the shipped tests. Finally, the identity

$$\text{xc_webb} \;=\; 64\pi^5 g^2 \left(T^K\right)^2 \;=\; g^2\, G \tag{128}$$

connecting Eq. (124) to the coupling routine of the exact-NL code of van Vledder (mod_xnl4v5, as distributed with WAVEWATCH III and SWAN) was verified to a relative accuracy of $3{\cdot}10^{-13}$ on resonant quadruplets, so that the absolute normalisation of Eq. (120) is anchored to an independent production code at the level of the coupling coefficient.

WRT reduction and the resonance loci

Following Webb (1978) and Tracy and Resio (1982), the wavenumber delta function in Eq. (120) eliminates $\boldsymbol{k}_2$, and for fixed $(\boldsymbol{k}_1, \boldsymbol{k}_3)$ the frequency delta function reduces the remaining $\boldsymbol{k}_2$-plane integral to a line integral along the resonance locus

$$z(\boldsymbol{k}_2) \;=\; q + \sigma(\boldsymbol{k}_2) - \sigma(\boldsymbol{k}_2 + \boldsymbol{P}) \;=\; 0 \quad , \quad \boldsymbol{P} = \boldsymbol{k}_1 - \boldsymbol{k}_3 \;,\; q = \sigma_1 - \sigma_3 \tag{129}$$

with $\boldsymbol{k}_4 = \boldsymbol{k}_2 + \boldsymbol{P}$, so that

$$S_{nl}(\boldsymbol{k}_1) \;=\; \int d\boldsymbol{k}_3 \oint_{z=0} \frac{ds}{|\boldsymbol{\nabla}_{\boldsymbol{k}_2} z|}\, G\, \mathcal{B} \tag{130}$$

Note that $z(\boldsymbol{k}_3) = 0$ identically: the trivial quadruplet $\boldsymbol{k}_2 = \boldsymbol{k}_3$, $\boldsymbol{k}_4 = \boldsymbol{k}_1$ lies on every locus, with bounded G; the vanishing of the integrand there resides entirely in $\mathcal{B}$.

In deep water the loci form a one-parameter family: scaling lengths by $|\boldsymbol{P}|$ and frequencies by $\sqrt{g|\boldsymbol{P}|}$, the locus shape depends only on

$$\hat{q} = \frac{q}{\sqrt{g|\boldsymbol{P}|}} \in (0, 1) \tag{131}$$

(the labels $1, 3$ are chosen such that $q > 0$; $\hat{q} < 1$ holds for every pair since $(\sqrt{k_1} - \sqrt{k_3})^2 < k_1 - k_3 \le |\boldsymbol{k}_1 - \boldsymbol{k}_3|$). In the normalised frame with $\hat{\boldsymbol{P}} = (1, 0)$ the two intersections of the locus with the symmetry axis are analytic,

$$x_+ = \left(\frac{1 - \hat{q}^2}{2\hat{q}}\right)^2 \quad , \quad x_- = -\left(\frac{\sqrt{2 - \hat{q}^2} - \hat{q}}{2}\right)^2 \tag{132}$$

which provide exact anchors for the tracing. The code contains two tracers. Both describe the closed curve by its radius $r(\theta)$ about the interior midpoint of Eq. (132). Here θ is the

polar angle about this midpoint, not the wave direction. Both compute the derivative of the radius along the curve analytically from the gradient,

$$r'(\theta) = -\frac{\partial z/\partial\theta}{\partial z/\partial r} \quad , \quad ds = \sqrt{r^2 + r'^2}\, d\theta \tag{133}$$

The operator uses the routine trace_batch of assembly.py. It places the n_{pts} points of a locus uniformly in θ, with spacing $2\pi/n_{\mathrm{pts}}$, and finds the radius of each point by bisection along its ray. The periodic trapezoid rule in θ then yields spectrally accurate quadrature weights $w_s = (ds/d\theta)\,(2\pi/n_{\mathrm{pts}})/|\boldsymbol{\nabla} z|$. The routine trace_locus of locus.py is used in the locus tests. It traces the curve densely by radial continuation from ray to ray and resamples it to equal arc length Δs, with the weights $w_s = \Delta s/|\boldsymbol{\nabla} z|$. For $q = 0$ (equal frequencies, $\boldsymbol{k}_1 \neq \boldsymbol{k}_3$) the locus degenerates to the perpendicular bisector $\{\boldsymbol{k}_2 : \boldsymbol{k}_2 \cdot \boldsymbol{P} = -|\boldsymbol{P}|^2/2\}$; these same-frequency interactions are unbounded in extent but their line integral converges through the decay of the spectrum, and they are included with a separate straight-line quadrature. The dimensional line measure follows from the scale invariance,

$$\frac{ds}{|\boldsymbol{\nabla} z|} \;=\; \frac{|\boldsymbol{P}|^{3/2}}{\sqrt{g}}\, \frac{d\hat{s}}{|\hat{\boldsymbol{\nabla}}\hat{z}|} \tag{134}$$

and the coupling scales as $G \propto |\boldsymbol{k}|^6$ by the homogeneity of T. Combining the scalings of Eq. (130), the transfer obeys the deep-water self-similarity

$$S_{nl}\,(\boldsymbol{k}; \lambda A(\cdot/\mu)) \;=\; \lambda^3 \mu^{19/2}\, S_{nl}\,(\boldsymbol{k}/\mu; A) \tag{135}$$

which the discretisation reproduces on the geometric grid for the closed loci in the interior of the grid (see below).

Discretisation

The spectral grid of wim_simple is geometric in frequency and uniform in direction, hence geometric in k with uniform spacing h in $\ln k$. Two consequences are exploited. First, a rotation of the pair $(\boldsymbol{k}_1, \boldsymbol{k}_3)$ by a multiple of the direction spacing $\Delta\theta$ rotates its locus by the same angle. The locus geometry, the coupling values, the quadrature weights and the interpolation stencils are therefore tabulated for $\theta_1 = 0$ only. The table covers every pair (i_1, i_3) of frequency indices and every direction-index difference $j_3 - j_1$. Each of these configurations is traced once, exactly. In the stencils the $\ln k$ indices are absolute grid indices, and the direction indices are offsets relative to j_1. The table is applied to every θ_1 by shifting all direction indices by j_1. This renders the discrete operator exactly equivariant under rotations of the spectrum by grid directions. The size of the table scales as $n_f^2\, n_d\, n_{\mathrm{pts}}$, with n_f frequencies, n_d directions and n_{pts} points per locus. There is no interpolation in $\hat{q}$; the discrete grid realises finitely many configurations, and all of them are solved. Second, $\hat{q}$ depends only on the ratio k_3/k_1 and the direction difference. Let ρ be the ratio of neighbouring frequencies, so that $h = 2\ln\rho$. A shift of i_1 and i_3 by one ring multiplies

all wavenumbers of the quadruplets by ρ^2, and the stencils move by one index in $\ln k$. In the interior of the grid this renders the closed-locus part of the operator equivariant under shifts along the geometric k-axis — the discrete form of Eq. (135). Near the grid boundaries this equivariance is broken. There the strict truncation described below discards different quadruplets for a spectrum and for its shifted copy. The same-frequency line loci break it as well. Each line is cut at $|\boldsymbol{k}_2| = 1.02\, k_{max}$, with k_{max} the largest wavenumber of the grid, and not at a fixed multiple of $|\boldsymbol{P}|$. The shipped test of Eq. (135) therefore leaves out the line loci and evaluates only interior rings.

Action densities at the off-grid locus points $\boldsymbol{k}_2, \boldsymbol{k}_4$ are obtained by bilinear interpolation in $(\ln k, \theta)$. Locus points within a small fraction of $|\boldsymbol{P}|$ of the trivial point $\boldsymbol{k}_2 = \boldsymbol{k}_3$ are removed: there the exact bracket Eq. (121) vanishes but its interpolant does not; the computed transfer is insensitive to the filter width at the percent level, which is verified as part of the test battery. Above the discrete frequency range the spectrum is extended by a parametric $E \propto \nu^{-5}$ tail over a number of additional geometric rings before evaluation (following WRT practice in WAVEWATCH III), and the transfer is restricted to the original band afterwards; quadruplets with any member beyond the extended range are discarded entirely (strict truncation). Strict truncation is what preserves the equilibria: every retained quadruplet carries all four action values, so $\mathcal{B} \equiv 0$ termwise for constant A, and Eq. (122) holds up to interpolation error for $A \propto 1/\sigma$. This holds for the operator itself, evaluated without the tail. The backend in wim_simple appends the tail by default, with eight rings (pad_hi = 8, see below). The tail continues an equilibrium spectrum of the band by $E \propto \nu^{-5}$, so the extended spectrum is no equilibrium. The backend therefore returns a nonzero transfer for an equilibrium spectrum of the band. For constant A on a band of 20 rings this transfer reaches about 0.2 of the one-sided (gross) rate, also in the interior of the band.

With cell areas $\Delta_i = k_i\, \Delta k_i\, \Delta\theta$, where $\Delta k_i = k_i\,(\rho - 1/\rho) = 2k_i \sinh(h/2)$ is the width of ring i, the discrete form of Eq. (130) for one quadruplet instance $(i_1 j_1, i_3 j_3, s)$ carries the measure

$$\mu_q \;=\; \Delta_3\, w_s\, G_s \tag{136}$$

and two assemblies of the same integral are provided. In the *direct* mode the rate $\mu_q \mathcal{B}$ is accumulated at $\boldsymbol{k}_1$ only — the discrete analogue of Eq. (130) and of other WRT implementations. In the *conservative* mode each instance deposits the total-action increments

$$(\delta\mathcal{A}_1, \delta\mathcal{A}_2, \delta\mathcal{A}_3, \delta\mathcal{A}_4) \;=\; (+1, +1, -1, -1)\, \frac{\Delta_1 \mu_q \mathcal{B}}{4} \tag{137}$$

distributed over the grid through the same bilinear stencils used for the gather. Dividing the deposit at the interpolated legs by the effective cell area $\bar{\Delta}$, interpolated in $\ln k$ with the same weights, conserves the deposited action exactly, since the stencil weights w_t and the θ-independent areas satisfy $\sum_t w_t \Delta_{c(t)} = \bar{\Delta}$ identically. The factor $1/4$ in Eq. (137) averages the four role-estimators of the enumeration over ordered pairs; the total action budget of the discrete transfer, including the action exchanged with the parametric tail, then closes

at machine precision by construction. The two modes are independent discretisations of Eq. (130); their difference is $O(h)$, decreases under grid refinement, and serves as a built-in convergence diagnostic. Energy and momentum are conserved by the continuous quadruplets and hence to interpolation accuracy by the discrete transfer. The code does not compute their residuals. The backend stores action diagnostics of the last evaluated cell in the dictionary Snonlin.diag. In the conservative mode these are action_rate, the sum of Δ_i times the transfer over the extended grid, and leak, the action of deposits that fall outside the grid. In both modes they are band_action_rate and tail_action_rate, the same sum over the band and over the tail rings.

For the semi-implicit time stepping a diagonal derivative is accumulated alongside,

$$\frac{\partial S_{nl}(\boldsymbol{k}_1)}{\partial A_1} \;=\; \sum_q \mu_q \left(A_3 A_4 - A_2(A_3 + A_4)\right) \tag{138}$$

It is the derivative of the direct rate at $\boldsymbol{k}_1$ with respect to A_1, with A_2, A_3 and A_4 held fixed. The code returns it in both modes. In the conservative mode, the default, it is not the derivative of the returned transfer. There A_1 enters also through the deposits of the instances in which $\boldsymbol{k}_1$ plays the role 2, 3 or 4, and the role-1 deposit carries the factor 1/4. At the peak cell of a JONSWAP spectrum and at a cell above the peak the derivative of the conservative transfer, estimated by finite differences, differs from the sum above by about a factor 2. In the direct mode the two differ by about 15% at these cells, since A_1 also enters through the interpolated A_2 and A_4. The sum serves as the diagonal factor of the semi-implicit step. It is invariant under the conversion between A and E on the diagonal and is used directly in the update $E \leftarrow E + S\,\Delta t/(1 - \Delta t\,\partial S/\partial E)$.

The backend in wim_simple

The model selects the exact transfer by "snonlin_backend" = 'exact'. In each source step the model hands the variance density E of a cell to the backend. The backend returns the transfer S_{nl} of E and the diagonal factor $\partial S/\partial E$. The settings of the backend are attributes of the object Snonlin in the module pyOM3.wim_simple.snl_exact.snonlin_exact. They must be set before the setup of the model, since that setup builds the interaction table. The attributes and their defaults are

- g = 9.81, the gravitational acceleration of the backend. It is not taken from the attribute grav of the model. A model with another value of grav needs the same value here.
- npts = 24, the number n_{pts} of points per closed locus. The 128 points per locus quoted for the locus tests below refer to the routine trace_locus, not to the operator.
- npts_line = None, which gives $4\,n_{\mathrm{pts}}$ points per same-frequency line locus.
- pad_hi = 8, the number of tail rings.

- dfilt = 0.025, the radius of the trivial-point filter in units of $|\boldsymbol{P}|$.
- cache = None. A file name here stores the table in that file. A later setup with the same grid and the same settings reloads it.
- mode = 'conservative', the assembly mode. The other choice is 'direct'.
- points = None, the list of cells (i_x, i_y) to evaluate. None means all cells.
- max_cells = 64, the largest number of cells per call. With more cells the backend stops with an error.
- verbose = True, progress output during the table build.

Cells not in points get $S_{nl} = 0$ and $\partial S/\partial E = 0$. With the numpy backend of pyOM3 the indices (i_x, i_y) count the interior cells of the local MPI rank, without halo. The frequency axis of the model must be geometric. The setup of the backend checks this and stops otherwise.

Validation

The implementation is validated layer by layer; the quoted accuracies refer to the test battery shipped with the sub-package (test grids of 28–30 frequencies and 24 directions).

- *Kernel*: degenerate two-wave interaction coefficient to $4 \cdot 10^{-6}$ at the test offsets; on-shell symmetries, rotation, homogeneity and g-independence to 10^{-13}; identity Eq. (128) with the WAVEWATCH III coupling routine to $3 \cdot 10^{-13}$.
- *Loci*: residuals $|z|$ at returned points $4 \cdot 10^{-14}$; crossings Eq. (132) exact; resonance closure of dimensional quadruplets 10^{-14} for both signs of q; line-delta quadrature against a brute-force two-dimensional integral to $5 \cdot 10^{-5}$ (grid limited); measure scaling Eq. (134) to $4 \cdot 10^{-15}$; quadrature of trace_locus converged to 10^{-9} with 128 points per locus.
- *Assembly*: action budget (transfer + leak) closed to 10^{-15}; rotation equivariance, isotropy and cubic homogeneity at machine precision; discrete self-similarity Eq. (135) to $1.2 \cdot 10^{-7}$ for the closed loci on interior rings; trivial-filter insensitivity 0.6% between filter widths 0.025 and 0.05.
- *Equilibria*: constant A null at $2 \cdot 10^{-19}$ of the one-sided (gross) rate; Rayleigh–Jeans null at $7 \cdot 10^{-5}$ with $O(h^2)$ convergence; near-vanishing transfer for the Zakharov–Filonenko spectrum $E \propto \nu^{-4}$.
- *Magnitude and shape*: mean JONSWAP transfer with the classic $(+, -, +)$ lobe structure. The maximum of the positive low-frequency lobe is 0.8 times that of the WAM DIA. The DIA was itself tuned against exact computations (Hasselmann et al., 1985). This agreement rules out any residual power of 2π or g in the normalisation chain.

- *Independent exact code*: on identical input, and with the default interaction filtering of that code disabled, the transfer lobes agree with the exact-NL code of van Vledder (WAVEWATCH III / SWAN) at the level of a few percent (positive lobe ratio 0.96, negative lobe ratio 1.02 in the reference case), with the independent result bracketed between the direct and conservative modes, as expected for consistent discretisations of the same integral.

References

Hasselmann, K., 1962: On the non-linear energy transfer in a gravity-wave spectrum. Part 1. *J. Fluid Mech.*, 12, 481–500.

Hasselmann, S., K. Hasselmann, J.H. Allender and T.P. Barnett, 1985: Computations and parameterizations of the nonlinear energy transfer in a gravity-wave spectrum. Part II. *J. Phys. Oceanogr.*, 15, 1378–1391.

Krasitskii, V.P., 1994: On reduced equations in the Hamiltonian theory of weakly nonlinear surface waves. *J. Fluid Mech.*, 272, 1–20.

Stuhlmeier, R., 2024: Deterministic wave forecasting with the Zakharov equation. arXiv:2401.03539.

Tracy, B.A. and D.T. Resio, 1982: Theory and calculation of the nonlinear energy transfer between sea waves in deep water. WES Report 11, US Army Corps of Engineers.

van Vledder, G.Ph., 2006: The WRT method for the computation of non-linear four-wave interactions in discrete spectral wave models. *Coastal Engineering*, 53, 223–242.

Webb, D.J., 1978: Non-linear transfers between sea waves. *Deep-Sea Res.*, 25, 279–298.

Zakharov, V.E., 1968: Stability of periodic waves of finite amplitude on the surface of a deep fluid. *J. Appl. Mech. Tech. Phys.*, 9, 190–194.